

Above the Cut

Sequent Calculus as a High-Level Programming Language

PAUL DOWNEN, University of Massachusetts Lowell, USA

For almost a century, the sequent calculus has proven to be an indispensable theoretical tool for studying logic, proofs, and now, computation. More recently, it also provided new insights about practical concerns of compilation and program optimization. But what would it take to actually *program* with the sequent calculus directly without cutting yourself on low-level concerns? Here, we aim to address questions around designing a practical, high-level language based on the sequent calculus. This involves several solutions to usability issues when writing sequent calculus programs, including the notion of duality between values and contexts, choosing between implicit versus explicit control flow, and weaving between different ways to react to a value or a context. In the end, we provide a higher-level interface surrounding a sequent calculus core that lets the programmer harness its full, uncompromised expressiveness while still rising above the low-level *Cuts*.

Additional Key Words and Phrases: Sequent Calculus, Duality, Polarity, Codata, Copatterns, Continuations

1 Introduction

In the world of logics, natural deduction and the sequent calculus are twins—born at the same time in the same paper [33]—long known to be equivalent: two different views of the same structure. However, when it comes to computation, these two have had very different impacts in the landscape of programming languages. For a long time, natural deduction, with its correspondence with the λ -calculus, has heavily influenced the way programming languages are designed and implemented—especially functional languages—from source code to type systems to the final executable. Though its theoretical basis for computation is more recent [19, 37, 39], the sequent calculus, too, has also found purchase in discovering, understanding, and expressing aspects of programs under the level of the original source code. Several compiler intermediate representations [28, 49, 51] designed for optimizing various low-level behaviors are based on the sequent calculus, and it has even been used for the lowest possible level isomorphic to theoretical [50, 52] and real [41, 56] machine code.

So we might then ask: what should a *high-level language*, for programmers to use directly, based on the sequent calculus look like? What would it be useful for? And what difficulty is standing in the way of directly programming in the existing sequent-based compiler intermediate representations?

One advantage—which was an original motivation for investigating sequent calculus as an intermediate representation—is to give control over non-functional control flow with a structural approach to continuations and direct jumps. A functional style of programming with pattern matching can make for beautifully simple functions over lists and trees. However, this same style can sometimes pigeon-hole us into only certain well-worn paths, making us blind to other possibilities. For example, consider this definition of the *filter* function—keeping only the list elements that pass a predicate *p*—written in Standard ML:

```
fun filter p [] = []
  | filter p (x::xs) = if p x then x :: filter p xs
                      else filter p xs;
```

While this definition is straightforward to write and reason about using structural induction, it has some undesirable operational behavior during real execution. In particular, this *filter* function will *always* allocate an entirely new list, even if it is exactly the same as before (if it happens that *p x* is true for every element). Ideally, we would be able to share a common suffix of the original list that doesn't change—and avoid allocating any new list cells entirely if nothing changes—through some

well-timed short-circuiting. Doing so without re-checking every step needs non-functional control flow to manage *multiple continuations*, which is possible using multi-return function calls [57] that are expressible with structured continuations in a sequent calculus representation [53].

A second advantage is an intrinsic notion of *duality*. Following the fundamental duality between logical truth and falsehood, the same reflection between polar opposites appears in languages based on the sequent calculus. The aforementioned expressiveness over control flow is a consequence of the fact that control flow is dual to information flow, and information flow has a rich range of tools for expressive power. It is second nature to store values in variables and combine multiple values into one, but surprising when we do the same with continuations. This same pattern repeats across various aspects of language design. Algebraic data types are reflected [24, 61] as codata types [32, 36]. Evaluation orders are dual to one another, connecting the familiar call-by-value and call-by-name [19, 60], as well as deriving new ones [9]. Taking this duality to its logical conclusion blurs the line between values and evaluation contexts, making them appear perfectly isomorphic.

So then, what makes the sequent calculus difficult to program in directly? When compared to something like the λ -calculus, it offers a trade: new expressive power over control flow, structured continuations, and duality described above in exchange for the ability to simply *use* something and implicitly receive the results. This is due to the lack of *eliminations* and resemblance between sequent style and continuation-passing style. For example, a purely sequent calculus representation—perhaps surprisingly—lacks any notion of function *application* $f(x)$. Instead, one must *construct* a call-stack (often after capturing and naming the current continuation as the base of the stack) and put it in contact with the function to call. No matter the selling points, it is hard to make a serious case for giving up something so natural and ubiquitous as direct function composition $f(g(x))$.

A second wrinkle is that, in order to take full advantage of all the dualities available, a sequent-based language needs to mix call-by-name and call-by-value evaluation orders using *polarity* [3, 42]. This is a common practice for languages based on the sequent calculus [47, 62], especially when representing rich, user-defined (co)data types [25]. However, it imposes either restrictions, redundancies, or additional annotations more appropriate for compiler intermediate representations than a source language, similar to the way call-by-push-value [43] explicates details useful for a compiler but often distracting for the programmer.

This paper, then, is about having your cake and eating it, too. The aim is to explore and promote the design of high-level, source programming languages based on the sequent calculus in a way that provides its advantages—without compromise—but avoids the pitfalls. We present a new high-level programming language, *Coo*, which exposes the full power of a sequent calculus abstract machine based on the logical *Cut* rule to the programmer. At the same time, a primary design goal is to let the programmer rise above the low-level *Cuts*, writing ordinary programs in a “natural” way (almost) exactly as they would in a functional language. Our contributions to this design space are:

- A hybrid, high-level programming language subsuming both sequent calculus and λ -calculus in one, and expressing both traditional functional programming idioms alongside a stateless style of object-oriented programming in terms of codata [15, 29] instead of classes.
- An *invisible duality*, similar to involutive negation [34], as a language feature. Captured call stacks are usable as data structure values, and vice versa. Functions *are* continuations.
- A novel approach to polarity in a source language, with implicit, untyped *polarity correction* that statically fixes polarity mismatches rather than merely checking for them. Polarity correction is compatible with both static types (as it occurs during compile time, fixing types before type checking) and dynamic types (as it requires no typing information).
- A unifying *reaction system* which combines multiple different methods of matching on or responding to some structured interface into a single language construct.

Moreover, we present the following specific technical results our sequent-based source language:

- Section 3 gives a new, minimal sequent-based intermediate language, generalizing [28] to be suitable for fully symmetric languages and not just the purely functional sub-language. This intermediate language enjoys both type safety and an equational theory, while being directly interpretable as an abstract machine.
- Section 4 presents a formalization of the *Coo* source programming language, its (optional) type system, equational theory, and polarity correction transformation on source programs.
- Section 5 defines the translation from the source to the intermediate language, and proves its correctness in terms of preservation of static typing and their equational theories.
- Section 6 describes additional aspects of the reference implementation, including integrating resumable control into an interactive interpreter, such that the interpreter prompt follows the (potentially non-functional) control-flow of incrementally written programs.

The reference implementation of the source programming language Coo, extended examples, and proofs of all the theorems below are provided in the supplemental materials.

2 Programming With and Without the Sequent Calculus

To illustrate what it is like to program with the sequent calculus in *Coo*, let's consider some specific examples showing what is the same, what is different, and what is possible now that wasn't before.

2.1 Simple Programs in a Natural Style

Here are functions for projecting out of a pair, and making a pair of duplicate elements in *Coo*:

```
fun fst(x,y) = x;           fun snd(x,y) = y;           fun dup x = (x,x);
```

What is remarkable about these definitions is how utterly unremarkable they are. This shows one of the main design principles of the language: ordinary programs should look natural, without unnecessary ceremony. In fact, all three of these definitions are also legal as-is in Standard ML, too, and with the same semantics. For example, if we compose *dup* with itself, we will get a 4-by-4 nested pair, such as $\text{dup}(\text{dup } 4) = ((4, 4), (4, 4))$. Similarly, we can project out a specific element of a tuple sequence using first and second projection as $\text{fst}(\text{snd}(\text{snd}(1,2,3,4))) = 3$, since pairing associates to the right. The two *snd* projections returns $(3,4)$, whose *fst* projection is 3.

Often, a function has to decide between multiple different alternative results depending on its inputs. One way to do this is with *guards*—introduced by an *if* clause—which adds a precondition to a response. Multiple alternative conditional options are introduced and separated by a *|* vertical bar. For example, the absolute value function *abs* can be defined with a boolean guard, like so

```
fun abs n if | n < 0 = 0 - n
              |      = n;
```

where the first option (negate *n*) is only returned when $n < 0$. If that test fails, the next option (simply *n*) is always returned, since it has no precondition between *|* introducing the option and *=* marking the right-hand side. This way, both *abs*(10-3) and *abs*(3-10) return a positive 7.

More generally, functions can pattern-match on their arguments—just like the ML and Haskell family of languages—to decide the result. Using the usual representation of lists as either the constant *Nil* or the newly-constructed cell *Cons*[*x*,*xs*] with one element *x* before the rest of the list *xs*, here is the recursive function summing up all the numbers in a list by pattern-matching:

```
fun | sum Nil           = 0
    | sum Cons[x,xs] = x + sum xs;
```

2.2 Evaluation Order and Code Versus Data

By default, the arguments to functions are evaluated before being passed to the function, and parameters to constructors (like `Cons[x, xs]`) are evaluated before the structure is built. This has the advantage of being similar to the familiar call-by-value evaluation order used by most programming languages, but is not always desirable. For example, defining a boolean `and` as a function like

```
fun and_eager x y if | x = y
                  |   = False;
```

does not have the useful short-circuiting property that most programmers expect. For example, if we call `and_eager (z /= 0) (10/z > 2)` when `z` is 0, then both arguments are evaluated first before `and_eager` gets a chance to decide, and `10/z` causes a division by zero error. To delay evaluation only when it is really needed, we can instead pass arguments as *thunks* of unevaluated code like so:

```
fun and_lazy {x} {y} if | x = y
                    |   = False;
```

Notice that the only change to the function is that the parameters have been wrapped in braces `{...}` denoting the introduction of a thunk. This matches the exact same shape of the call site. When we call `and_lazy {z /= 0} {10/z > 2}`, the braces delay evaluation of the arguments. The function `and_lazy` unwraps these braces, and then the contents are automatically evaluated only as needed when they are used in a strict context. By the definition, `x` is evaluated first in the guard, and only when it returns `True` does `y` get evaluated, skipping the division by zero error from before.

This is Coo's first major deviation from traditional functional languages. *Functions are not values!* Instead, functions of any arity (including 0, as in `and_lazy`) can be *converted* to values—a prerequisite to function calling or data construction—by wrapping them in braces. This has the biggest impact on higher-order functions, such as following definition of `map`:

```
fun | map {f} Nil          = Nil
    | map {f} Cons[x, xs] = Cons[f x, map {f} xs];
```

Similarly, on the call site, passing a function as a value requires braces as well, such as `map {abs}`. Forgetting to convert the function into a value, and just writing `map abs`, leads to an error because this operationally means “call the `abs` function with no arguments *now* and pass the result to `map`.”¹

Another change is that there is more flexibility in deciding how to react to a function call *during* the match, because reactions can be nested. We already saw the first example with boolean guards, but other types of reactions can be nested, too. Recursive functions like `map` often have initial parameters that never change, leading to repetitions of the same call prefix (e.g., `map {f}`). We can factor out this common prefix by nesting another *inner* recursive function definition inside the original one. For example, `map {f}` factors out of the recursive function loop like so:

```
fun map {f} where # from here, we have: loop = map {f}
  | loop Nil      = Nil
  | loop Cons[x, xs] = Cons[f x, loop xs];
```

We're not restricted to introducing a recursive function in just named definitions. Anonymous function expressions can be simply written as `fn x y => (x+1)*y`, but also support pattern-matching and boolean guards with multiple clauses like named function definitions, too. More interestingly, we can define *anonymous recursive functions* by introducing recursive equations with **where** as above. For example, we can define and pass the factorial function to `map` in-place like so:

¹Incidentally, this operational reading *does* coincide with the scripting language Ruby, wherein parentheses are also optional for function calls. Because `f()` and `f` are the same, calling a higher-order method like `.map(abs)` with a unary function argument `abs` results in a run-time arity error since `abs` is called first with 0 arguments. Instead, the correct method call similarly wraps the function in braces as `.map[|n| abs(n)]`. The differences in Coo are that the η -expansion is never necessary, and the evaluation order is decided *statically*, not *dynamically*, so the error can be reported before the code runs.

```
map {fn where | fact 0 = 1
      | fact n = n * fact(n-1)}
    Cons[1, Cons[2, Cons[3, Cons[4, Cons[5, Nil]]]]]
```

2.3 Codata and Copatterns

And now we get to one of the main motivations behind using the sequent calculus as a programming language: to be able to talk about codata objects in multiple ways. The first way, which may be familiar already to Agda programmers, is to use *copatterns* [1] to distinguish between different destructors. For instance, the common example of infinite streams is modeled with two destructors: *Head* projects out the first element and *Tail* returns the rest of the stream after it. As simple examples, the stream which is always the same element *x*, and the stream of repeated applications of a next function can be defined like so:

```
fun | always(x).Head = x          fun repeat {next} where
    | always(x).Tail = always(x); | from(x).Head = x
                                | from(x).Tail = from(next(x));
```

A more interesting example is to merge two streams *xs* and *ys* in a zigzag fashion, alternating between their elements with *xs* at even positions and *ys* at odd positions.

```
fun | zigzag {xs} {ys} .Head      = xs.Head
    | zigzag {xs} {ys} .Tail.Head = ys.Head
    | zigzag {xs} {ys} .Tail.Tail = zigzag {xs.Tail} {ys.Tail};
```

This involves nested destructors to handle the case of the initial *.Head* element at index 0, the *.Tail.Head* at index 1, and the remaining *.Tail.Tail* after those two, updating each stream.

But we can do much more than just simple streams. Consider the object-oriented model of stepping through a range of numbers between two end points. The most basic interface is to have a *Lower* bound, an *Upper* bound, and then a *Step* function that increments to a smaller range like so:

```
fun range(x, y) where
  | self.Lower      = x
  | self.Upper      = y
  | self.Done       = x > y
  | self.Step{make} = make{range(x+1, y)};
```

The prefix *range(x, y)* serves as the “constructor” of the object which is named *self* in the following lines. Since it has no internal mutable state, we can use equational reasoning to understand it. *range(20,30).Lower* = 20 and *range(20,30).Upper* = 30 by substitution and matching. The extra parameter on *Step* explains how to make the incremented range object. In a simple case, we can just pass a function which runs the object directly, so that *range(20,30).Step{fn {r} => r}.Lower* = 21.

But this interface is too basic to be of much use on its own! We need to layer at least a little more functionality on top. One of the simplest methods derivable from the others is to calculate its *Size*.

```
fun sizing{obj} where
  | self.Size = self.Upper - self.Lower + 1
  | self      = obj;
```

The *sizing* function takes a range object with an *Upper* and *Lower* method, and extends it with a *Size* method on top; any other message is handled by *obj* itself. So *sizing{range(20,30)}.Size* = 10, and *sizing{range(20,30)}* behaves like *range(20,30)* in any other context. A more interesting operation to add is a *Collect{f}* method, which builds a list of values by applying the function *f* to each number in the range like so:

```
fun collecting{obj} where
  | self.Collect{f} if self.Done = Nil
  | self.Collect{f} = Cons[f(self.Lower), self.Step{collecting}.Collect{f}]
  | self           = obj;
```

So that `collecting{range(1,10)}.Collect{fn x => x*x}` returns a list of the first ten squares from $1 = 1*1$ to $100 = 10*10$. Note that this is the first time we are using the underlying Step function; to make sure the next step still knows how to Collect, we pass the collecting modifier along.

2.4 Sequential and Non-Functional Control

The second major motivation is to be able to exploit the underlying control flow of a classical sequent calculus representation. And the simplest version of this is straightforward, sequential, structured programming. Thus far, we have only considered the purely functional aspects of the Coq programming language made up of essentially just expressions and reactive function definitions. However, it contains an imperative side, too, with sequential statements. For example, a stand-alone **if** conditional is a statement—not an expression. If we wanted to intentionally write the one-step collatz function with an **if** statement, it could look like:

```
fun collatz(n) = do => {
  if (n%2 == 0) => go n/2;
  go n*3;
};
```

The right-hand side of a **fun** definition in this form is an expression, and one possibility is a **do**-expression wrapping a sequence of statements. Inside a **do** => { ... }, the statement **go** x jumps out of the **do** with x as its returned value. So for collatz(n), if it happens that n is even, then the right-hand side of the conditional runs and n/2 is the result. Otherwise, we move on to the next statement, which sends out n*3 as the answer of the **do**.

As a more interesting example, we can continue the range examples. Here, we extend a range object obj with an imperative-style Each{f} method; f is called on each number in the range:

```
fun iterating{obj} where
| self.Eeach{f} = do => {
  if self.Done => go ();
  f(self.Lower);
  go self.Step{iterating}.Each{f};
}
| self = obj;
```

If the range is already Done, then the Each{f} returns immediately with a unit value (), otherwise the function f is called with the Lower number in the range, and we go to continue iterating over the next Step. For example, `iterating{range(1,10)}.Each{ pprint }` will pretty-print (with the pprint function) the numbers between 1 and 10.

Remark 2.1. Coq is based on the classical sequent calculus as part of its core, so it can define encodings representing axioms of classical logic, most famous of which is Scheme's call/cc operator, corresponding to Peirce's law [7, 35]. The shortest definition of callcc in Coq is:

```
fun callcc {h} = do => go h {fn x to k' => go x};
```

Note that the implicit **go** x goes to the *lexically* nearest **do** => ...—the one starting callcc's body—just like a static binder. Furthermore, we have a new kind of reaction in the function passed to h: **fn** x **to** k' => **go** x means a function taking one argument x that binds and *throws away* the rest of its calling context, instead choosing to **go** here with the answer x. For clarity, we can explicate the implicit **do** => ... **go** ... into an equivalent, explicit label and jump pair like so:

```
fun callcc {h} = do k => go h {fn x => do k' => go x to k} to k
```

Now, the **do**-expression introduces an explicit label k, which is used as a target for a **go** **to** command twice: once immediately after callcc is called, and later in the function passed to h. Note the similar use of **to** to introduce an evaluation context in both the jump form **go** ... **to** ... as well

as the function abstraction $\text{fn } x \text{ to } k' \Rightarrow \dots$. This corresponds exactly to the usual encoding of `call/cc` [23] in terms of lower-level primitives based on Parigot's $\lambda\mu$ -calculus [54], where a `go to` corresponds to a labeled jump command, and `do` corresponds to a μ -abstraction.

Moreover, we can emphasize the equational meaning of `callcc` by expanding the left-hand side to match on an entire `go ... to ...` form all at once in another equivalent definition:

```
fun go callcc {h} to k = go h {fn x to k' => go x to k} to k;
```

Other classical axioms can be defined, too, like the devilish interpretation of the law of excluded middle as an effectful sum $A + \neg A$ [60]—with an initial `lie` that we retract if ever called out—as well as the more purely angelic one in terms of linear logic's negative disjunction $A^\perp \wp A$ [34].

```
fun devil's_lem = do => {
  fun go lie x to k' = go Yes[x];
  go No[{lie}];
};
fun go angel's_lem to x, k = go x to k;
```

We can also make full use of duality to program more interesting control operators, too. For example, subtraction [19] is the dual to functions and can express delimited control [4], which happens to be connected to Ruby's `yield` operation [40]. A subtraction-based `yield` ($A \rightarrow (A - B)^\perp \rightarrow B$) is:

```
fun go yield x cxt to k = go (x, k) to cxt;
```

2.5 Unusual Filter Functions

Now, let's bring all this together for two unusual takes on the filter function. The first—a *parsimonious* filter on lists—was already described in the introduction. The goal is to filter a list such that (1) the longest unchanged suffix is reused instead of reallocated, and (2) the recursive result is returned without re-checking (ruling out a sum type of the two possible cases). Here is the algorithm based on a function with two return paths [53, 57] written with the sequent calculus:

```
fun go parsimoniousFilter {p} xs to {start} = {
  val () = filterPass {p} xs {start};
  go xs to start
};
fun filterPass {p}
| go fp Nil to {diff}, {same} = go to same
| go fp Cons[x,xs] to {diff}, {same}
  if | p x = go fp xs
      to {do xs' => go Cons[x,xs'] to diff},
      {same}
  | = go fp xs
      to {diff},
      {do () => go xs to diff};
```

The main entry point is `parsimoniousFilter`, which might return to the `{start}` caller in two different ways: if something is different, the helper `filterPass` will return a list directly to the first continuation, but if everything is the same, it will return a unit value `()` to the second one so we can just pass the original list to the `start`. This logic can be seen in the recursive `filterPass` definition. An empty list cannot change, so we just `go to` the same return path. On `Cons[x,xs]`, whether there is a change depends on whether `p x` is `True`. If it is, nothing changed here, so remember to add back `x` if something is different later. If it's not, then this is a change; if this is the *first* change (*i.e.*, the second path is taken) then send `xs` to the `diff` path, otherwise just skip it.

This function gives the same answer as the naïve one, but with different allocation behavior. Of note, `parsimoniousFilter {fn x => True}` never allocates a single `Cons` cell. Moreover, we can

visually check that `filterPass` is always *half tail-recursive*: no matter the `p x` guard, one of the two return paths is exactly reusing a given one as-is, and the other is allocated (*i.e.*, a non-tail call).

Now let's shift attention to filtering streams. Compared to finite lists, stream filtering infinite streams is more dangerous because if the predicate is always `False`, then filtering becomes an unproductive infinite loop searching for the first `True` case. But theoretically, there is a safe notion of always-productive filtering on infinite streams if we're not picky. No matter what, there will always be an infinite number of `True` cases *or* an infinite number of `False` cases (sometimes both). So we could just choose to return an infinite stream of *either* all successes *or* all failures.

But that can't be computable, can it? Deciding at the start which one we will have infinite examples of seems like it would take infinite time. After all, there is no finite number of checks we can do before we *know* there will be infinite similar examples. A predicate could fail the first one billion elements and then succeed forever after. Determining which case we're in ahead-of-time would require some kind of infinite prescience. So let's write an infinitely prescient filter function!

```
fun go prescientFilter {p} {xs} to {start}
  = go filterSwap {p} {xs} to start, start;

fun filterSwap {p} where
  | go filter {xs} to Head[{this}], that = go xs.Head to {this}
  | go filter {xs} to Tail[{this}], that if {ys} <= xs.Tail
    if | p(xs.Head) == p(ys.Head)      = go filter {ys} to this, that
    |                                = go filter {ys} to that, this;
```

This function juggles two continuations, just like the parsimonious filter, but now they are *structured messages* for the stream codata object. The entry function `prescientFilter` duplicates the start continuation which is a sequence of `Tail` projections ended by a `Head`. By reasoning in terms of the sequent calculus, a projection `go xs.Tail to k` is exactly the same as sending a structure as `go xs to Tail[k]`. The `filterSwap` helper function then juggles two possible futures: the first one where the predicate `p`'s judgment is the *same* as this stream, and the second for the opposite cases. The `Head` element must then be the same as this one, and when the `Tail` is requested, we have to check who gets the next answer. Call the `Tail` of `xs` as `ys` (set by the *pattern guard* [30] `if {ys} <= xs.Tail`, which generalizes boolean guards to any pattern-match); if `p` is the same on the `Head` of both, then continue `filter {ys}` to `this`, otherwise swap and `filter {ys}` to `that`.

The surprising thing about the prescient filter function is that—despite the impossibility of an exhaustively infinite pre-check—it maintains a *consistency* across all observations of the same stream, while exploring the least amount of its input as possible. For example, suppose we try filtering an arbitrary predicate across the stream of natural numbers:

```
fun | count(n).Head = n          val {xs} = prescientFilter
  | count(n).Tail = {count(n+1)}; {fn | 2 = False
val nats = {count(0)};           | n => n < 5}
                                nats;
```

If we only ever look at the first four elements of `xs`, we will see 0, 1, 3, and 4 in that order. But the second we look for element five and beyond, it turns out that `xs` begins with 2, 5, 6, 7, 8, and so on. The trick is essentially the same idea as `devil's_lem`, swapping when a contradiction is found, but done *incrementally* as the program pulls elements out of `xs`, by juggling multiple future paths.

3 Dual Core: A Sequent Calculus Intermediate Language

To understand how to build the source programming language on top of the sequent calculus, we now introduce *Dual Core*, a small subset of the full *Coo* programming language with only a small number of features that nevertheless suffice to encode everything else we've seen so far.

$$\begin{aligned}
\text{Command} \ni \quad J &::= \text{go } M \text{ as } I \text{ to } N \\
(\text{Co}) \text{Term} \ni M, N &::= x \mid \text{do } x \Rightarrow J \mid \text{do } \{ \mid \text{as } x \text{ to } P \Rightarrow J \dots \} \mid V \\
(\text{Co}) \text{Value} \ni V, W &::= x \mid () \mid (V, W) \mid X[V] \mid \{ M \} \\
\text{Pattern} \ni \quad P &::= x \mid () \mid (P, P') \mid X[P] \mid \{ x \} \\
\text{Interaction} \ni \quad I &::= \text{cbv} \mid \text{cbn}
\end{aligned}$$

Syntactic sugar:

$$\begin{aligned}
X &= X[()] & \langle M, N \rangle &= \text{do } z \Rightarrow \text{go } M \text{ as cbv to do } y \Rightarrow \\
\langle X[M] \rangle &= \text{do } x \Rightarrow \text{go } M \text{ as cbv to do } y \Rightarrow & \text{go } N \text{ as cbv to do } z \Rightarrow \\
&\text{go } X[y] \text{ as cbv to } x & \text{go } (y, z) \text{ as cbv to } x
\end{aligned}$$

Fig. 1. Syntax of the Dual Core calculus.

3.1 Syntax

Dual Core is a polarized, focused sequent calculus whose syntax is given in fig. 1, along with some syntactic sugar for writing data structures more easily. The first thing to notice is that the notation reflects the plain-text keywords of the source language. For aficionados of the sequent calculus lineage of $\bar{\lambda}\mu\tilde{\mu}$ [19], the term $\text{do } x \Rightarrow J$ denotes either a $\mu x.J$ term or a $\tilde{\mu}x.J$ coterm. The only command is a *jump* $\text{go } M \text{ as } I \text{ to } N$ denoting a cut $\langle M \mid I \mid N \rangle$, with an extra parameter I in the middle signing the evaluation strategy of the interaction as in [25]. The binary $\bar{\lambda}\mu\tilde{\mu}$ form of cut $\langle v \parallel e \rangle$ is written as the shorthand $\text{go } v \text{ to } e$, where the strategy annotation is left implicit.

Beyond these superficial differences in notation, Dual Core compares to other polarized and focused core calculi and intermediate languages [25, 46, 48, 58, 61] as follows:

- (Co)terms (and similarly (co)values) have been collapsed, and share the exact same syntax.
- A (co)term’s polarity is not an intrinsic part of its syntax for generic forms like x or $\text{do } x \Rightarrow J$; their polarity is instead inferred from context (discussed later in section 3.3).
- Focusing is enforced syntactically, as components of data structures—like (V, W) or $X[V]$ —must be (co)values already. The non-(co)value constructions are given as syntactic sugar.
- The possible constructions include nullary $()$ and binary (V, W) tuples, tagged values $X[V]$ of a sum type, and the thunk $\{ M \}$ of a polarity shift. The only other value is a variable x .
- These constructions are destructured with $\text{do } \{ \mid \text{as } x_1 \text{ to } P_1 \Rightarrow J_1 \mid \dots \mid \text{as } x_n \text{ to } P_n \Rightarrow J_n \}$, a pattern-matching abstraction sometimes notated with a μ or $\tilde{\mu}$ binder over a set of pattern-command responses in other sequent calculi [46, 48, 58]. In addition to the pattern $\text{to } P$, Dual Core also allows for a self-referential binder $\text{as } x$ to model recursion similar to [27], but without any termination-enforcing restriction. If the self-referential variable is never used, we write $P \Rightarrow J$ as shorthand for $\text{as } x \text{ to } P \Rightarrow J$ where $x \notin FV(J)$.

3.2 Operational Semantics

The operational semantics of the Dual Core calculus is given in fig. 2. Unlike an expression-based functional language based on the λ -calculus—which are interested in reducing terms to a final value—the computational unit of the Dual Core sequent calculus is the command. Also like other sequent calculi languages [5], this operational semantics simultaneously serves as an abstract machine, because the next operational step is *always* at the top of the whole command.

There are only four rules in the operational semantics. The first rule expresses one of the main results of the sequent calculus as a programming language: the duality between call-by-value and

$$\begin{aligned}
(\beta^\circ) \quad & \text{go } M \text{ as } \text{cbn} \text{ to } N \mapsto \text{go } N \text{ as } \text{cbv} \text{ to } M \\
(\beta_{fun}) \quad & \text{go } (\text{do } x \Rightarrow J) \text{ as } \text{cbv} \text{ to } M \mapsto J[M/x] \\
(\beta_{val}) \quad & \text{go } V \text{ as } \text{cbv} \text{ to } (\text{do } x \Rightarrow J) \mapsto J[V/x] \\
(\beta_{DC}) \quad & \text{go } P_j[\sigma] \text{ as } \text{cbv} \text{ to } M \mapsto J_j[\sigma, M/x_j] \\
& (M = \text{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \}, \text{Dom}(\sigma) = \text{FV}(P_j))
\end{aligned}$$

Fig. 2. Operational semantics of the Dual Core calculus.

call-by-name execution [19, 60]. Writing the duality operation as $(_)^\circ$, it says, operationally

$$\text{go } M \text{ as } I \text{ to } N = \text{go } N^\circ \text{ as } I^\circ \text{ to } M^\circ \quad \text{cbv}^\circ = \text{cbn} \quad \text{cbn}^\circ = \text{cbv}$$

Fortunately, since we’ve reused exactly the same syntax for both terms and coterms, $M^\circ = M$, which makes it easy to express this operational equivalence directly as a reduction step, cutting the rest of the cases in half. The choice of which evaluation strategy to favor is arbitrary due to symmetry, so the β° converts every call-by-name (**cbn**) command into an equivalent call-by-value (**cbv**) one by swapping the two sides of a jump. After that, we only have to handle **cbv** reductions.

The next two operational steps are β_{fun} and β_{val} for substituting into a generic $\text{do } x \Rightarrow J$ abstraction according to the priority of the evaluation strategy. In call-by-value, only values can be substituted into the parameter of the continuation on the right, whereas anything can be substituted in for the x in a $\text{do } x \Rightarrow J$ computation on the left. This resolves [19] the classical critical pair

$$\text{go } (\text{do } x_1 \Rightarrow J_1) \text{ as } \text{cbv} \text{ to } (\text{do } x_2 \Rightarrow J_2) \mapsto J_1[(\text{do } x_2 \Rightarrow J_2)/x] \quad (\beta_{fun})$$

so that β_{fun} has higher priority over β_{val} , since $\text{do } x_1 \Rightarrow J_1$ is not in $(\text{Co})\text{Value}$. The last reduction rule is β_{DC} which handles pattern-matching between structured values and reactive branching **do** abstractions. In addition to decomposing a value into a pattern and substitution to plug into the parameters of the command, β_{DC} also substitutes the entire pattern-matching abstraction in for the self-referential parameter $\text{as } x_j$, unfolding the recursion one step.

3.3 Type System

Dual Core’s biggest departure from other similar focused, polarized sequent calculi is the type system, given in fig. 3. There are only four different connectives in the simple type system, one matching each shape of pattern and written in a similar way. The nullary and binary tuple types are written $()$ and (A, B) , respectively. Tagged unions, written $[X : A\dots]$, list the types of payload y in a matching tagged structure $X[y]$, à la a sum type. A thunk $\{ M \}$ has a shifted type $\{ A \}$.

Usually, each type has a specific polarity given by the outer connective, which introduces an extra complication when combining polarized types in ways that don’t align with connectives’ expectations. One typical solution is to restrict the well-formed types [62] similar to call-by-push-value [43] so that, for example, (A, B) is a positive type, and requires both A and B to be positive types, too. Another solution [46] is to insert implicit shifts at a mismatch, so that if A is negative in (A, B) , it *really* means that A is shifted to positive. Both solutions add complications—restrictions add additional kind-checking constraints statically, and invisible shifts add complications dynamically to handle unstated, implicit changes in evaluation order not spelled out by constructors.

Instead, each type just signifies an *interface* or *shape* of pattern, and is combined with a kind—either **val** (i.e., *positive*) or **fun** (i.e., *negative*)—to determine the direction of flow. For example, if we say $x : () : \text{val}$, that means x is a *value* of the form $()$ from the unit type $()$. In contrast, $x : () : \text{fun}$ is also allowed, and means that x is a *functional process* which takes an input of the shape $()$. This

$$\begin{aligned}
& \text{Type} \ni A, B ::= () \mid (A, B) \mid [X : A \dots] \mid \{A\} & \text{Kind} \ni K ::= \text{val} \mid \text{fun} \\
& \text{go}(\text{cbv}) = \text{val} & \text{to}(\text{cbv}) = \text{fun} & \text{go}(\text{cbn}) = \text{fun} & \text{to}(\text{cbn}) = \text{val} \\
& \text{Core typing rules: } \boxed{\Gamma \vdash J} \text{ and } \boxed{\Gamma \vdash M : A : K} \\
& \frac{\Gamma \vdash M : A : \text{go}(I) \quad \Gamma \vdash N : A : \text{to}(I)}{\Gamma \vdash \text{go } M \text{ as } I \text{ to } N} \text{Cut} & \frac{}{\Gamma, x : A : K \vdash x : A : K} \text{Id} & \frac{\Gamma, x : A : K^\circ \vdash J}{\Gamma \vdash \text{do } x \Rightarrow J : A : K} \text{Act} \\
& \text{Rules for (co)terms of (co)data types: } \boxed{\Gamma \vdash M : A : K} \\
& \frac{\Gamma \vdash V : A ;}{\Gamma \vdash V : A : \text{val}} \text{Dat} & \frac{\Delta_i \vdash P_i : A ;; \quad \Gamma, x_i : A : \text{fun}, \Delta_i \vdash J_i \dots^i. \quad \{P_i \dots^i\} \text{ covers } A}{\Gamma \vdash \text{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \dots^i \} : A : \text{fun}} \text{Cod} \\
& \text{Rules for focused (co)value structures: } \boxed{\Gamma \vdash V : A ;} \\
& \frac{}{\Gamma, x : A : \text{val} \vdash x : A ;} \text{IdV}; & \frac{\Gamma \vdash M : A : \text{fun}}{\Gamma \vdash \{M\} : \{A\} ;} \{ \} V; \\
& \frac{}{\Gamma \vdash () : () ;} ()V; & \frac{\Gamma \vdash V : A ; \quad \Gamma \vdash W : B ;}{\Gamma \vdash (V, W) : (A, B) ;} (,)V; & \frac{\Gamma \vdash V : A_j ;}{\Gamma \vdash X_j[V] : [X_i : A_i \dots^i] ;} [[]]V; \\
& \text{Rules for linear (co)patterns: } \boxed{\Delta \vdash P : A ;;} \\
& \frac{}{x : A : \text{val} \vdash x : A ;;} \text{IdV}; & \frac{}{x : A : \text{fun} \vdash \{x\} : \{A\} ;;} \{ \} V; \\
& \frac{}{\bullet \vdash () : () ;;} ()V; & \frac{\Delta \vdash P : A ;; \quad \Delta' \vdash P' : B ;;}{\Delta, \Delta' \vdash (P, P') : (A, B) ;;} (,)V; & \frac{\Delta \vdash P : A_j ;;}{\Delta \vdash X_j[P] : [X_i : A_i \dots^i] ;;} [[]]V; \\
& \text{Coverage of a type by a set of patterns: } \boxed{\{P \dots\} \text{ covers } A} \\
& \{P_i \dots^i\} \text{ covers } A \iff \forall (x : B : \text{fun} \dots \vdash V : A ;), \quad \exists P_j \in \{P_i \dots^i\}, \sigma \in \text{Subst}_{FV(P_j)}, \quad P_j[\sigma] = V
\end{aligned}$$

Fig. 3. Simple type system for the Dual Core calculus.

is repeated for all the other types, so that the **val** side of a type describes the values produced as outputs, and the **fun** side of the same type describes all the possible inputs the function can handle. This alignment is enforced by the *Cut* rule: in any command, one side will always be a **val** type and the other a **fun** type depending on the strategy of the interaction I as **cbv** or **cbn**. Shift types $\{A\}$ are the most subtle. A shifted value, like $\{M\} : \{A\} : \text{val}$, produces a value wrapped in a thunk, but the contents M is a *function* taking input of type A . Dually, a shifted function, like $\text{do } \{ \mid \{x\} \Rightarrow J \} : \{A\} : \text{fun}$, takes an input of the shape $\{x\}$, but passes a *value* of type A to x .

There are several advantages to this style of polarity in an implementation: (1) *every* combination of connectives is well formed, so there is no need to kind-check simple types, (2) *every* change of evaluation strategy is signaled by an explicit shift in the program, so that we can simply assume that all components of non- $\{_ \}$ data structure values are eagerly evaluated, and (3) we only need *half* as many connectives, instead of positive-negative dual pairs of type constructors. Despite these simplifications, we still get type safety for commands without free value variables.

THEOREM 3.1 (PROGRESS). *If $x : A : \text{fun}, \dots \vdash J$ then either $J \mapsto J'$ for some J' or J is a final command of the form $\text{go } V \text{ as } \text{cbv} \text{ to } x$ for some $x : A : \text{fun} \in \Gamma$.*

THEOREM 3.2 (PRESERVATION). *If $J \mapsto J'$ then $\Gamma \vdash J$ implies $\Gamma \vdash J'$.*

$$\begin{array}{ll}
(\eta_{act}) & \text{do } x \Rightarrow \text{go } M \text{ as } I \text{ to } x = M \quad (x \notin FV(M)) \\
(\eta_{react}) & \text{do } \{ \mid P_i \Rightarrow \text{go } M \text{ as } \text{cbn to } P_i.i. \} = M \quad (\forall i, \emptyset = FV(P_i) \cap FV(M)) \\
(\eta_{rec}) & \text{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow \text{go do } \{ \mid P'_{ij} \Rightarrow J_{ij}.i. \} \text{ as } \text{cbn to } P_i.i. \} = \text{do } \{ \mid P_i \Rightarrow \text{go do } \{ \mid \text{as } x_i \text{ to } P'_{ij} \Rightarrow J_{ij}.i. \} \text{ as } \text{cbn to } P_i.i. \}
\end{array}$$

Fig. 4. Additional axioms of the Dual Core calculus equational theory.

In addition, it is sometimes useful to check a programs' *polarity*—the distinction between positive **val** and negative **fun**—but not its *types*. For example, even though the **cbv** or **cbn** tag in commands resolves the critical pair locally, confluence is still lost in ill-polarized programs [25]. For this, we derive a polarity system from the type system by erasing types A, B from each of the rules in fig. 3 (shown in appendix C). This way, being well-polarized is always implied by being well-typed.

LEMMA 3.3 (TYPED POLARITY). *All well-typed commands, terms, values, patterns are well-polarized.*

3.4 Equational Theory

Finally, an equational theory is a useful alternative to the operational semantics (equivalently, the abstract machine semantics) to prove properties about program behavior. This will come in handy later for reasoning about translations. The Dual Core calculus includes the *symmetric, reflexive, transitive*, and *compatible* closure of the operational β rules from fig. 2 (meaning the rules can be applied forward and backward, repeatedly, in *any* context). In addition to the β rules, the equational theory also includes the η rules in fig. 4 for reasoning about extensional properties, too. Note that, like the operational steps (theorem 3.2), these η axioms preserve types from left to right. So for equating typed terms, it suffices to check that the left-hand side of the axiom is well-typed; if so, the right-hand side will have the same type.

The η_{react} axiom is the sequent calculus rephrasing of the familiar η axiom of the λ -calculus for functions, generalized to work with other (co)data types. The η_{act} axiom is the general form that works for any type, and surrounds any (co)term in a simple labeled jump. One example of its consequences is that an unfocused construction of values reduces down to a constructed value.

LEMMA 3.4 (VALUE RECONSTRUCTION). $\langle V, W \rangle = (V, W)$ and $\langle X[V] \rangle = X[V]$.

The last axiom η_{rec} is new here, and is useful for reasoning about recursive loops by moving the point of nested recursion inside an η -expanded pattern-match.

4 Coo: Combining Natural and Sequent Calculus Programs

Having seen the small Dual Core sequent calculus that can be used as an intermediate language, let's now consider how to grow a fuller, more usable, high-level programming language around it.

4.1 Syntax

Figure 5 gives a subset of the Coo source programming language syntax with syntactic sugar on top of the basic forms for common shorthands used in the real examples from section 2. Right away, we can see that the syntax combines both features from the sequent calculus (which has introductions and the **go to** *Cut* commands, but *never* an elimination) and natural deduction (which allows both introduction and eliminations, but eschews *Cuts*). Here, we have all three side-by-side, with the common elimination syntax $M N$ for function application and $M.X$ to project out the tagged X of

$Command \ni J ::= \text{go } M \text{ as } O \text{ to } N \mid \text{go } M \text{ as } O \mid \text{fun } R; J \mid \text{val } P \Leftarrow M \text{ else } R; J \mid \text{match } M R; J \mid \varepsilon$
 $(Co)Term \ni M, N ::= x \mid \text{do } R \mid M N \mid M.X \mid () \mid (M, N) \mid X[M] \mid \{M\} \mid !M$
 $Pattern \ni P ::= x \mid () \mid (P, P') \mid X[P] \mid \{x\}$
 $CoPattern \ni Q ::= x \mid Q P \mid Q.X$
 $Reaction \ni R ::= \varepsilon \mid \{R \mid R'\} \mid \text{as } Q R \mid \text{to } P R \mid \text{if } P \Leftarrow M R \mid \Rightarrow M \mid \Rightarrow J$
 $Annotation \ni O ::= I : A \mid I \mid A \mid \varepsilon$

$Interaction \ni I ::= \text{cbv} \mid \text{cbn}$

Syntactic sugar for (co)terms and statements (where $G ::= P \mid .X$):

$\text{do } P R = \text{do to } P R$ $\text{val } P \Leftarrow M; J = \text{val } P \Leftarrow M \text{ else } \varepsilon; J$ $\text{go } M \text{ to } N = \text{go } M \text{ as } \varepsilon \text{ to } N$
 $\text{fn } G \dots R = \text{do as } (_ G \dots) R$ $\text{if } R; J = \text{match } () \text{ with } () \text{ if } R; J$ $\text{go } M = \text{go } M \text{ as } \varepsilon$

Syntactic sugar for reactions:

$\text{with} = \text{to} \quad \text{to } \{ \mid P R \mid \dots \} = \{ \text{to } P R \mid \dots \} \quad \text{if } \{ \mid P \Leftarrow M R \mid \dots \} = \{ \text{if } P \Leftarrow M R \mid \dots \}$
 $\text{where} = \text{as} \quad \text{as } \{ \mid Q R \mid \dots \} = \{ \text{as } Q R \mid \dots \} \quad \text{if } M R = \text{if True} \Leftarrow M R$

Fig. 5. Syntax of the Coos source programming language.

a product type. In addition, there is a shift elimination $!M$ which means to evaluate a function of type $\{A\}$ —which needs to more arguments and has no more projections—to get a value of type A .

Moving on, the syntax of patterns is *exactly* the same as before, because these reflect exactly the focused introduction rules, which are the same in sequent calculus and natural deduction. What's new are the copatterns that reflect the new elimination rules for application and projection. One way to understand copatterns is their decomposition $Q = E_{\text{copat}}[x]$ into a special subset of evaluation contexts E_{copat} (which has the form of either $E'_{\text{copat}} P$ or $E'_{\text{copat}} X$) surrounding a self-referential binder x . In that sense, if a copattern-matching term is found in one of these contexts $E_{\text{copat}}[M]$, then the match is made and M itself is substituted for the root variable x .

Next, there are a larger variety of commands—which are now sequences of zero or more basic statements—for common sequential programming tasks including function definitions (**fun** $R; J$), a destructuring assignment (**val** $P \Leftarrow M$ **else** $R; J$), and a pattern-matching statement (**match** $M R; J$) which generalizes the basic boolean conditionals (**if** $R; J$). Note that, because the pattern P might not match M 's given value in **val** $P \Leftarrow M$, an **else** clause can be given to explain how to react if the assignment fails and the next statement cannot proceed, similar to Idris' pattern-matching bind [17, 18]. By default, this alternative path **else** clause is left empty, stopping the program with a match failure. Even the basic jump command has more options, including an annotation O that may or may not include a type A or an interaction strategy I ; if none are given, we simply omit the annotation in **go** M **to** N . The **to** N may also be omitted.

Finally, we have the most unusual part of the syntax: the reactions. These generalize and unify many pattern- and copattern-matching features that would appear in subtly different ways in various contexts. Each of these is a special mode of use for the general, compositionally nested forms. First of all, multiple reactions are combined because each one might fail to trigger; the empty reaction ε fails immediately while the alternatives $\{R \mid R'\}$ tries R first and if it fails falls back to R' . The equivalent reactions **as** $Q R$ or **where** $Q R$ allow for a potentially recursive copattern-based equation, where if Q matches the context we perform the substitution and continue as R . This is the default mode for recursive **fun** definitions and anonymous **fn**-abstractions. The equivalent reactions **to** $P R$ or **with** $P R$ match their *entire* input against the pattern and continue as R ; this

$$\begin{array}{c}
\Xi ::= \mathbf{whole} \mid \mathbf{part} \qquad \Psi ::= \bullet \mid A : K \qquad \Phi ::= \Xi \Psi \\
\text{Axioms for deciding the compatibility of goto annotations:} \\
\langle A : \mathbf{go}(I) \mid A : I \mid A : \mathbf{to}(I) \rangle \qquad \langle A : K \mid A \mid A : K^\circ \rangle \\
\langle A : \mathbf{go}(I) \mid I \mid A : \mathbf{to}(I) \rangle \qquad \langle A : K \mid \varepsilon \mid A : K^\circ \rangle \\
\text{Types of commands: } \boxed{\Gamma \vdash J \mid \Phi} \\
\frac{\Gamma \vdash M : A : K \mid \Psi \quad \langle A : K \mid O \mid A' : K' \rangle \quad \Gamma \vdash N : A' : K' \mid \Psi}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \mathbf{to} N \mid \Xi \Psi} \text{GoTo} \\
\frac{\Gamma \vdash M : A' : K' \mid A : K \quad \langle A' : K' \mid O \mid A : K^\circ \rangle}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \mid \Xi A : K} \text{Go} \quad \frac{}{\Gamma \vdash \varepsilon \mid \mathbf{part} \Psi} \text{End} \\
\frac{\Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi \quad \Gamma, \text{name}(R) : A : \mathbf{fun} \vdash J \mid \Xi \Psi}{\Gamma \vdash \mathbf{fun} R; J \mid \Xi \Psi} \text{Fun} \\
\frac{\Gamma \vdash M : A : \mathbf{val} \mid \Psi \quad \Delta \vdash P : A; \quad \Gamma, \Delta \vdash J \mid \Xi \Psi \quad \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi}{\Gamma \vdash \mathbf{val} P \Leftarrow M \mathbf{else} R; J \mid \Xi \Psi} \text{Val} \\
\frac{\Gamma \vdash M : A : \mathbf{val} \mid \Psi \quad \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{part} \Psi \quad \Gamma \vdash J \mid \Xi \Psi}{\Gamma \vdash \mathbf{match} M R; J \mid \Xi \Psi} \text{Match} \\
\text{Types of (co)terms: } \boxed{\Gamma \vdash M : A : K \mid \Psi} \\
\frac{}{\Gamma, x : A : K \vdash x : A : K \mid \Psi} \text{Id} \quad \frac{\Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi}{\Gamma \vdash \mathbf{do} R : A : \mathbf{fun} \mid \Psi} \text{React} \\
\frac{\Gamma, x : A : \mathbf{fun} \vdash J \mid \mathbf{whole} \Psi}{\Gamma \vdash \mathbf{do} x \Rightarrow J : A : \mathbf{val} \mid \Psi} \text{Act} \quad \frac{\Gamma \vdash J \mid \mathbf{whole} A : \mathbf{val}}{\Gamma \vdash \mathbf{do} \Rightarrow J : A : \mathbf{val} \mid \Psi} \text{Act'} \\
\frac{\Gamma \vdash M : A : \mathbf{val} \mid \Psi \quad \Gamma \vdash N : B : \mathbf{val} \mid \Psi}{\Gamma \vdash (M, N) : (A, B) : \mathbf{val} \mid \Psi} (.)VI \quad \frac{\Gamma \vdash M : (A, B) : \mathbf{fun} \mid \Psi \quad \Gamma \vdash N : A : \mathbf{val} \mid \Psi}{\Gamma \vdash M N : B : \mathbf{fun} \mid \Psi} (.)FE \\
\frac{\Gamma \vdash M : A_j : \mathbf{val} \mid \Psi}{\Gamma \vdash X_j[M] : [X_i : A_i..] : \mathbf{val} \mid \Psi} [[]]VI \quad \frac{\Gamma \vdash M : [X_i : A_i..] : \mathbf{fun} \mid \Psi}{\Gamma \vdash M.X_j : A_j : \mathbf{fun} \mid \Psi} [[]]FE \\
\frac{}{\Gamma \vdash () : () : \mathbf{val} \mid \Psi} ()VI \quad \frac{\Gamma \vdash M : A : \mathbf{fun} \mid \Psi}{\Gamma \vdash \{M\} : \{A\} : \mathbf{val} \mid \Psi} \{ \}VI \quad \frac{\Gamma \vdash M : \{A\} : \mathbf{fun} \mid \Psi}{\Gamma \vdash !M : A : \mathbf{val} \mid \Psi} \{ \}FE
\end{array}$$

Fig. 6. Simple type system for the Coo source language: commands and (co)terms.

is the default to a **do**-term and is given explicitly in **match** M **with** $\dots$. Guards **if** $P \Leftarrow M R$ only succeed if M 's value matches P , and are ubiquitous in all the previous modes of use. Finally, we reach a right-hand side, either evaluating a term $\Rightarrow M$ or executing a command $\Rightarrow J$.

4.2 Type System

The typing rules for source commands and (co)terms are given in fig. 6, where the syntax of types is *exactly* the same as in Dual Core (fig. 3). Structurally, though, there are some additions to the typing judgments. The judgment for commands has the form $\Gamma \vdash J \mid \Xi \Psi$ where Ψ is an *optional* return type—used to determine the type of M in a **go** M without the **to**—that may or may not be present. Additionally, Ξ describes the *extent* of a command: a **whole** command *must* end in a final

Types of reactions: $\boxed{\Gamma \vdash R : \Psi \mid \Phi}$

$$\begin{array}{c}
\frac{}{\Gamma \vdash \varepsilon : \Psi \mid \Phi} \text{Cede} \quad \frac{\Gamma \vdash R : \Psi \mid \Phi \quad \Gamma \vdash R' : \Psi \mid \Phi}{\Gamma \vdash \{R \mid R'\} : \Psi \mid \Phi} \text{Alt} \\
\frac{\Delta \vdash P : A ;; \quad \Gamma, \Delta \vdash R : \bullet \mid \Phi}{\Gamma \vdash \text{to } P \text{ } R : A : \text{fun} \mid \Phi} \text{To} \quad \frac{\Delta ;; A \vdash Q : B \quad \Gamma, \Delta \vdash R : B : \text{fun} \mid \Phi}{\Gamma \vdash \text{as } Q \text{ } R : A : \text{fun} \mid \Phi} \text{As} \\
\frac{\Gamma \vdash M : A : \text{val} \mid \Psi' \quad \Delta \vdash P : A ;; \quad \Gamma, \Delta \vdash R : \Psi \mid \Xi \Psi'}{\Gamma \vdash \text{if } P \Leftarrow M \text{ } R : \Psi \mid \Xi \Psi'} \text{If} \\
\frac{\Gamma \vdash J \mid \Phi}{\Gamma \vdash \Rightarrow J : \bullet \mid \Phi} \text{Exec} \quad \frac{\Gamma \vdash J \mid \Xi A : \text{fun}}{\Gamma \vdash \Rightarrow J : A : \text{fun} \mid \Xi \Psi} \text{Exec'} \\
\frac{\Gamma \vdash M : A : \text{fun} \mid \Psi}{\Gamma \vdash \Rightarrow M : A : \text{fun} \mid \Xi \Psi} \text{Eval} \quad \frac{\Gamma \vdash M : A : \text{fun} \mid A : \text{fun}}{\Gamma \vdash \Rightarrow M : \bullet \mid \Xi A : \text{fun}} \text{Eval'}
\end{array}$$

Typing rules for patterns $\boxed{\Delta \vdash P : A ;;}$ are as given in fig. 3.

Typing rules for copatterns: $\boxed{\Delta ;; A \vdash Q : A}$

$$\begin{array}{c}
\frac{}{x : A : \text{fun} ;; A \vdash x : A} \text{IdF};; \\
\frac{\Delta ;; A \vdash Q : A' \rightarrow B' \quad \Delta' \vdash P : A' ;;}{\Delta, \Delta' ;; A \vdash Q \text{ } P : B'} \rightarrow F;; \quad \frac{\Delta ;; A \vdash Q : [X_i : B_{i..}] }{\Delta ;; A \vdash Q.X_j : B_j} [[]]F;;
\end{array}$$

Fig. 7. Simple type system for the Coos source language: reactions and copatterns.

go-jump in every branching path, whereas as **partial** command may end with an empty ε that will automatically continue to the “next” statement in some larger control flow. The typing judgment for (co)terms also has an optional second return type Ψ —in case it contains a **go** N without the **to** outside the scope of an implicit **do** $\Rightarrow J$ —but no Ξ , since all (co)terms are effectively **whole**. Note that the operation $\text{name}(R)$ gives the self-referential identifier name at the root of the copattern Q in the first (leftmost, outermost) **as** $Q \text{ } R$ reaction.

One thing might be surprising when reading the types and typing rules for (co)terms: there are no function arrows $A \rightarrow B$! That’s because ordinary function types are actually *the same thing* as tuple types. In the source language, arrows are treated as an alternative notation for pairs:

$$A \rightarrow B = (A, B)$$

This may seem shocking, but it is essentially the same encoding as in linear logic [34] ($A \multimap B = A^\perp \wp B$) but with even more dualities collapsed. Because the directional polarity is separated from a type interface, whether something of type $A \rightarrow B$ is actually the function process (transforming an A into a B) or the call stack for such a process depends on whether it is a **fun** or a **val** of type $A \rightarrow B$. Thus, the more intuitive elimination rule for arrows below is the same as $(\cdot)FE$ in fig. 6,

$$\frac{\Gamma \vdash M : A \rightarrow B : \text{fun} \mid \Psi \quad \Gamma \vdash N : A : \text{val} \mid \Psi}{\Gamma \vdash M N : B : \text{fun} \mid \Psi} \rightarrow FE$$

The typing rules for source reactions and copatterns are given in fig. 7. The typing judgment for reactions carries the same side information $\Phi = \Xi \Psi$ as a command—because a right-hand side of a reaction might be a command—and it *also* has an optional main type Ψ for itself. That main type $R : \Psi$ of a reaction will be non-empty when $R : A : \text{fun}$ still has more input to match on,

$$\begin{array}{c}
\Gamma, \Delta ::= \bullet \mid \Gamma, x :: K \quad \text{Direction} \ni D ::= \text{go} \mid \text{to} \\
\Psi ::= \bullet \mid K \quad \text{Action} \ni R_{act} ::= x \Rightarrow J \mid \Rightarrow J \\
\\
\text{Polarity inference of (co)terms: } \boxed{\Gamma \vdash M \rightsquigarrow M' :: K \mid \Psi} \text{ or } \boxed{\Gamma \vdash M \rightsquigarrow \bullet \mid \Psi} \\
\\
\frac{}{\Gamma, x :: K \vdash x \rightsquigarrow x :: K \mid \Psi} Id \quad \frac{}{\Gamma \vdash \text{do } R_{act} \rightsquigarrow \bullet \mid \Psi} Act\bullet \quad \frac{R \notin \text{Action} \quad \Gamma \vdash R :: \text{fun} \rightsquigarrow R' \mid \Psi}{\Gamma \vdash \text{do } R \rightsquigarrow \text{do } R' :: \text{fun} \mid \Psi} React \\
\\
\frac{}{\Gamma \vdash () \rightsquigarrow () :: \text{val} \mid \Psi} ()VI \quad \frac{\Gamma \vdash M :: \text{go cbv} \rightsquigarrow M' \mid \Psi \quad \Gamma \vdash N :: \text{go cbv} \rightsquigarrow N' \mid \Psi}{\Gamma \vdash (M, N) \rightsquigarrow (M', N') :: \text{val} \mid \Psi} (,)VI \\
\\
\frac{\Gamma \vdash M :: \text{go cbn} \rightsquigarrow M' \mid \Psi \quad \Gamma \vdash N :: \text{go cbv} \rightsquigarrow N' \mid \Psi}{\Gamma \vdash M N \rightsquigarrow M' N' :: \text{fun} \mid \Psi} (,)FE \\
\\
\frac{\Gamma \vdash M :: \text{go val} \rightsquigarrow M' \mid \Psi}{\Gamma \vdash X_j[M] \rightsquigarrow X_j[M'] :: \text{val} \mid \Psi} [[]]V \quad \frac{\Gamma \vdash M :: \text{go cbn} \rightsquigarrow M' \mid \Psi}{\Gamma \vdash M.X_j \rightsquigarrow M'.X_j :: \text{fun} \mid \Psi} [[]]FE \\
\\
\frac{\Gamma \vdash M :: \text{go cbn} \rightsquigarrow M' \mid \Psi}{\Gamma \vdash \{M\} \rightsquigarrow \{M'\} :: \text{val} \mid \Psi} \{\}VI \quad \frac{\Gamma \vdash M :: \text{go cbn} \rightsquigarrow M' \mid \Psi}{\Gamma \vdash !M \rightsquigarrow !M' :: \text{val} \mid \Psi} \{\}FE \\
\\
\text{Polarity correction of (co)terms: } \boxed{\Gamma \vdash M :: D I \rightsquigarrow M'} \\
\\
\frac{\Gamma, x :: D(I)^\circ \vdash J \rightsquigarrow J' \mid \Phi}{\Gamma \vdash \text{do } x \rightsquigarrow J :: D I \rightsquigarrow \text{do } x \Rightarrow J' \mid \Phi} Act \quad \frac{\Gamma \vdash J \rightsquigarrow J' \mid D(I)}{\Gamma \vdash \text{do} \Rightarrow J :: D I \rightsquigarrow \text{do} \Rightarrow J' \mid \Phi} Act' \\
\\
\frac{\Gamma \vdash M \rightsquigarrow M' :: K \mid \Phi}{\Gamma \vdash M :: D I \rightsquigarrow \text{shift}_I(D M' :: K) \mid \Phi} Shift \\
\\
\text{Auto-shifting conversion:} \\
\\
\begin{array}{ll}
\text{shift}_{cbv}(\text{go } M :: \text{fun}) = !M & \text{shift}_{cbv}(\text{to } M :: \text{val}) = \text{force } M \\
\text{shift}_{cbn}(\text{go } M :: \text{val}) = \text{return } M & \text{shift}_{cbn}(\text{to } M :: \text{fun}) = \{M\} \\
\text{shift}_I(D M :: K) = M & (\text{if } D(I) = K)
\end{array}
\end{array}$$

Fig. 8. Automatic polarity correction for the source Coq language: (co)term inference and checking.

and will be empty $R : \bullet$ when the entire input has already been consumed by a $\text{to } P R$. Note that these reactions will *always* be a **fun** kind of some type—if present—because they always match and consume their input. Guards have the same typing rule in both the empty and non-empty case, whereas the right-hand side depends on the combination of whether there is remaining input and if it leads to a command or (co)term. The rules for copatterns mimic (co)term eliminations.

4.3 Automatic Drive: Static Polarity Inference and Correction

And now we get to the system for determining polarity for the source language. Rather than just passively *checking* that the **val** and **fun** kinds are consistent like in the Dual Core calculus, the source language uses an active notion of polarity *correction* that fixes mismatches instead of merely reporting them. This is reflected by the binary nature of the polarity judgments, like $\Gamma \vdash J \rightsquigarrow J' \mid \Phi$, where the original code (e.g., J) is modified into a well-polarized form (e.g., J'). The rules for automatically inferring and correcting polarity are given in figs. 8 to 9.

There's just one difficulty: we don't have enough shifts! In particular, the rules we have regarding the shift type $\{A\}$ in fig. 6 can only introduce or eliminate the $\{A\}$ to make a *value*, but neither can

Polarity correction of commands: $\boxed{\Gamma \vdash J \rightsquigarrow J' \mid \Psi}$

$$\begin{array}{c}
\frac{}{\Gamma \vdash \varepsilon \rightsquigarrow \varepsilon \mid \Psi} \text{End} \quad \frac{\Gamma \vdash R :: \mathbf{fun} \rightsquigarrow R' \mid \Psi \quad \Gamma \vdash J \rightsquigarrow J' \mid \Psi}{\Gamma \vdash \mathbf{fun} R; J \rightsquigarrow \mathbf{fun} R'; J' \mid \Psi} \text{Fun} \\
\frac{\Gamma \vdash M :: \mathbf{go cbv} \rightsquigarrow M' \mid \Psi \quad \Delta \vdash P ;; \quad \Gamma \vdash R :: \mathbf{fun} \rightsquigarrow R' \mid \Psi \quad \Gamma \vdash J \rightsquigarrow J' \mid \Psi}{\Gamma \vdash \mathbf{val} P \Leftarrow M \mathbf{else} R; J \rightsquigarrow \mathbf{val} P \Leftarrow M' \mathbf{else} R'; J' \mid \Psi} \text{Val} \\
\frac{\Gamma \vdash M :: \mathbf{go cbv} \rightsquigarrow M' \mid \Psi \quad \Gamma \vdash R :: \mathbf{fun} \rightsquigarrow R' \mid \Psi \quad \Gamma \vdash J \rightsquigarrow J' \mid \Psi}{\Gamma \vdash \mathbf{match} M R; J \rightsquigarrow \mathbf{match} M' R'; J' \mid \Psi} \text{Match} \\
\frac{\Gamma \vdash M :: \mathbf{go strat}(O) \rightsquigarrow M' \mid \Psi \quad \Gamma \vdash N :: \mathbf{to strat}(O) \rightsquigarrow N' \mid \Psi}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \mathbf{to} N \rightsquigarrow \mathbf{go} M' \mathbf{as} O \mathbf{to} N' \mid \Psi} \text{GoTo}\Rightarrow \\
\frac{\Gamma \vdash N \rightsquigarrow N' :: \mathbf{to}(I) \mid \Psi \quad \Gamma \vdash M :: \mathbf{go} I \rightsquigarrow M' \mid \Psi}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \mathbf{to} N \rightsquigarrow \mathbf{go} M' \mathbf{as} I : O \mathbf{to} N' \mid \Psi} \text{GoTo}\Leftarrow \\
\frac{\Gamma \vdash N \rightsquigarrow \bullet \mid \Psi \quad \Gamma \vdash M \rightsquigarrow M' :: \mathbf{go}(I) \mid \Psi \quad \Gamma \vdash N :: \mathbf{to} I \rightsquigarrow N' \mid \Psi}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \mathbf{to} N \rightsquigarrow \mathbf{go} M' \mathbf{as} I : O \mathbf{to} N' \mid \Psi} \text{GoTo}\Rightarrow \\
\frac{\Gamma \vdash M :: \mathbf{go strat}(O) \rightsquigarrow M' \mid \mathbf{go}(\mathbf{strat}(O))}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \rightsquigarrow \mathbf{go} M' \mathbf{as} O \mid \mathbf{go}(\mathbf{strat}(O))} \text{Go}\Rightarrow \quad \frac{\Gamma \vdash M :: \mathbf{go} I \rightsquigarrow M' \mid \mathbf{go}(I)}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \rightsquigarrow \mathbf{go} M' \mathbf{as} I : O \mid \mathbf{go}(I)} \text{Go}\Leftarrow
\end{array}$$

Adding an explicit interaction strategy I to an existing annotation O :

$$I : (\varepsilon) = I \quad I : (A) = I : A \quad I : O \text{ undefined otherwise}$$

Polarity correction of reactions: $\boxed{\Gamma \vdash R :: \Psi \rightsquigarrow R' \mid \Psi'}$

$$\begin{array}{c}
\frac{}{\Gamma \vdash \varepsilon :: \Psi \rightsquigarrow \varepsilon \mid \Psi'} \text{Cede} \quad \frac{\Gamma \vdash R_1 :: \Psi \rightsquigarrow R'_1 \mid \Psi' \quad \Gamma \vdash R_2 :: \Psi \rightsquigarrow R'_2 \mid \Psi'}{\Gamma \vdash \{R_1 \mid R_2\} :: \Psi \rightsquigarrow \{R'_1 \mid R'_2\} \mid \Psi'} \text{Alt} \\
\frac{\Delta \vdash P ;; \quad \Gamma, \Delta \vdash R :: \bullet \rightsquigarrow R' \mid \Psi}{\Gamma \vdash \mathbf{to} P R :: \mathbf{fun} \rightsquigarrow \mathbf{to} P R' \mid \Psi} \text{To} \quad \frac{\Delta ;; \vdash Q \quad \Gamma, \Delta \vdash R :: \mathbf{fun} \rightsquigarrow R' \mid \Psi}{\Gamma \vdash \mathbf{as} Q R :: \mathbf{fun} \rightsquigarrow \mathbf{as} Q R' \mid \Psi} \text{As} \\
\frac{\Gamma \vdash M :: \mathbf{go val} \rightsquigarrow M' \mid \Psi' \quad \Delta \vdash P ;; \quad \Gamma, \Delta \vdash R :: \Psi \rightsquigarrow R' \mid \Psi'}{\Gamma \vdash \mathbf{if} P \Leftarrow M R :: \Psi \rightsquigarrow \mathbf{if} P \Leftarrow M' R' \mid \Psi'} \text{If} \\
\frac{\Gamma \vdash J \rightsquigarrow J' \mid \Psi}{\Gamma \vdash \{\Rightarrow J\} :: \bullet \rightsquigarrow \{\Rightarrow J'\} \mid \Psi} \text{Exec} \quad \frac{\Gamma \vdash J \rightsquigarrow J' \mid \mathbf{fun}}{\Gamma \vdash \{\Rightarrow J\} :: \mathbf{fun} \rightsquigarrow \{\Rightarrow J'\} \mid \Psi} \text{Exec}' \\
\frac{\Gamma \vdash M :: \mathbf{go cbn} \rightsquigarrow M' \mid \Psi}{\Gamma \vdash \{\Rightarrow M\} :: \mathbf{fun} \rightsquigarrow \{\Rightarrow M'\} \mid \Psi} \text{Eval} \quad \frac{\Gamma \vdash M :: \mathbf{go cbn} \rightsquigarrow M' \mid \mathbf{fun}}{\Gamma \vdash \{\Rightarrow M\} :: \bullet \rightsquigarrow \{\Rightarrow M'\} \mid \mathbf{fun}} \text{Eval}'
\end{array}$$

Polarity inference of patterns $\boxed{\Delta \vdash P ;;}$ are as given in fig. 14.

Polarity inference of copatterns: $\boxed{\Delta ;; \vdash Q}$

$$\frac{}{x :: \mathbf{fun} ;; \vdash x} \text{IdF};; \quad \frac{\Delta ;; \vdash Q \quad \Delta' \vdash P ;;}{\Delta, \Delta' ;; \vdash Q P} \rightarrow F;; \quad \frac{\Delta ;; \vdash Q}{\Delta ;; \vdash Q.X_j} [[]]F;;$$

Fig. 9. Automatic polarity correction for the source Coq language: commands, reactions, and (co)patterns.

make a shifted *function*. For example, if we try to write $\text{fn } x \Rightarrow ()$, then the right-hand side needs to be some type of function, but $()$ is a value. That's no problem, though, because we can *define* the missing two shifting operations by pattern-matching on the braces $\{ _ \}$. These correspond closely to familiar operations from call-by-push-value [43], and are defined internally in Coq like so:

$$\begin{array}{ll} \text{fun return} : A \rightarrow \{ A \} & \text{fun go return } x \text{ as cbn to } \{ k \} = \text{go } x \text{ as cbv to } k \\ \text{fun force} : \{ A \} \rightarrow A & \text{fun go force } \{ x \} \text{ as cbn to } k = \text{go } x \text{ as cbn to } k \end{array}$$

Or more concisely for forcing a thunk, $\text{fun force } \{ x \} = x$. In other words, in Dual Core only,

$$\text{return} = \text{do } \{ \mid (x, \{ k \}) \Rightarrow \text{go } x \text{ as cbv to } k \} \quad \text{force} = \text{do } \{ \mid (\{ x \}, k) \Rightarrow \text{go } x \text{ as cbn to } k \}$$

This gives us admissible rules for the missing introduction and elimination of the shift braces

$$\frac{\Gamma \vdash M : A : \text{val} \mid \Psi}{\Gamma \vdash \text{return } M : \{ A \} : \text{fun} \mid \Psi} \{ \} FI \quad \frac{\Gamma \vdash M : \{ A \} : \text{val} \mid \Psi}{\Gamma \vdash \text{force } M : A : \text{fun} \mid \Psi} \{ \} VE$$

which are enough to plug the remaining gaps where polarities can be inferred.

In particular, the polarity correction system can infer most polarities from context; either the constraints of the introductions, eliminations and commands themselves, or from the way variables are introduced by (co)patterns. The one place it gets stumped is at an unannotated, under-specified **go to** between two generic **do**-abstractions of the form $\text{do } x \Rightarrow J$ or $\text{do} \Rightarrow J$. Unambiguous inference for such jumps is actually *impossible* in the worse case of $\text{go } (\text{do } _ \Rightarrow J) \text{ to } (\text{do } _ \Rightarrow J')$ where neither binder is used. This *could* be interpreted as either a **cbv** or a **cbn** command, and so polarity correction gives up in cases like these instead of making an arbitrary choice.

Otherwise, polarity correction is a deterministic procedure that gives an inherent meaning for being well-polarized like before, and is more forgiving than being well-typed. Specifically, the polarity-correction procedure can be used on untyped terms *before* types are known, which is a crucial feature: it can also correct type mistakes before the type checker gets caught by them.

Definition 4.1 (Polarity Elaboration & Well-Polarized). A (co)term, command, or reaction is *polarity elaborated* if it contains no annotations O of the form ε or A —that is to say, they all contain an explicit evaluation order of the form I or $I : A$. Moreover, it is *well-polarized* if it remains unchanged after polarity correction in some environment (e.g., $\Gamma \vdash J \rightsquigarrow J \mid \Psi$ for some Γ and Ψ).

LEMMA 4.2 (TYPED POLARITY). *If a command, (co)term, or reaction is well-typed and polarity elaborated, then it is well-polarized.*

LEMMA 4.3 (POLARITY CORRECTION).

- (1) J is well-polarized iff $\Gamma \vdash J' \rightsquigarrow J \mid \Psi$ for some Γ and Ψ .
- (2) M is well-polarized iff $\Gamma \vdash M' :: D I \rightsquigarrow M \mid \Psi$ for some Γ, D, I , and Ψ .
- (3) R is well-polarized iff $\Gamma \vdash R' :: \Psi \rightsquigarrow R \mid \Psi'$ for some Γ, Ψ , and Ψ' .

LEMMA 4.4 (POLARITY DETERMINISM).

- (1) If $\Gamma \vdash J \rightsquigarrow J_1 \mid \Psi$ and $\Gamma \vdash J \rightsquigarrow J_2 \mid \Psi$ then $J_1 = J_2$.
- (2) If $\Gamma \vdash R :: \Psi \rightsquigarrow R_1 \mid \Psi'$ and $\Gamma \vdash R :: \Psi \rightsquigarrow R_2 \mid \Psi'$ then $R_1 = R_2$.
- (3) If $\Gamma \vdash M :: D I \rightsquigarrow M_1 \mid \Psi$ and $\Gamma \vdash M :: D I \rightsquigarrow M_2 \mid \Psi$ then $M_1 = M_2$.
- (4) If $\Gamma \vdash M \rightsquigarrow M_1 :: K_1 \mid \Psi$ and $\Gamma \vdash M \rightsquigarrow M_2 :: K_2 \mid \Psi$ then $M_1 = M_2$ and $K_1 = K_2$.

As a consequence, polarity correction is idempotent: its output is well-polarized by lemma 4.3 meaning that it corrects back to exactly itself, and this is the only possible output by lemma 4.4.

	Type annotation erasure:
(<i>erase</i>)	$\text{go } M \text{ as } I : A \text{ to } N = \text{go } M \text{ as } I \text{ to } N$
	Eliminating eliminations:
($\kappa_{@}$)	$\text{go } M \ M' \text{ as cbn to } N = \text{go } M \text{ as cbn to } (M', N)$
(κ_X)	$\text{go } M.X \text{ as cbn to } N = \text{go } M \text{ as cbn to } X[N]$
($\kappa_!$)	$\text{go } !M \text{ as cbv to } N = \text{go } M \text{ as cbn to } \{ N \}$
	Constructing structures:
($\zeta_{@}$)	$\text{go } (M, M') \text{ as cbv to } N = \text{go } M \text{ as cbv to do } x \Rightarrow$ $\text{go } M' \text{ as cbv to do } y \Rightarrow$ $\text{go } (x, y) \text{ as cbv to } N$
(ζ_X)	$\text{go } X[M] \text{ as cbv to } N = \text{go } M \text{ as cbv to do } y \Rightarrow$ $\text{go } X[y] \text{ as cbv to } N$
	Commanding statements:
(goto_{fun})	$\text{fun } R; J = \text{go do } R \text{ as cbn to do } \text{name}(R) \Rightarrow J$
(goto_{val})	$\text{val } P \Leftarrow M \text{ else } R; J = \text{go } M \text{ as cbv to do } \{ P \Rightarrow J \mid R \}$
(goto_{match})	$\text{match } M \text{ R; } J = \text{go } M \text{ as cbv to do } (R \ ++ \ J)$
Where appending one command onto another, or onto a reaction, is defined as:	
	$(S_1; \dots; S_n; \varepsilon) \ ++ \ J = S_1 \dots; S_n; J$
	$(\Rightarrow J') \ ++ \ J = (\Rightarrow (J' \ ++ \ J)) \quad (\text{as } Q \ R) \ ++ \ J = \text{as } Q \ (R \ ++ \ J) \quad (\text{to } P \ R) \ ++ \ J = \text{to } P \ (R \ ++ \ J)$
	$(\Rightarrow M) \ ++ \ J = (\Rightarrow M) \quad (R_1 \mid R_2) \ ++ \ J = (R_1 \ ++ \ J) \mid (R_2 \ ++ \ J) \quad \varepsilon \ ++ \ J = \varepsilon$

Fig. 10. Additional axioms for equational reasoning about Coo commands and terms.

4.4 Equational Theory

The equational theory of the Coo source language includes every axiom already given for the Dual Core calculus—which is a sub-syntax of the full source language—including the operational steps in fig. 2 and the extensionality axioms in fig. 4. But this is not enough to understand the new forms of syntax. We need additional rules to give meaning to the additional programs.

4.4.1 Eliminating eliminations. Additional axioms for reasoning equationally about source commands and (co)terms are given in fig. 10. The *erase* rule lets us ignore type annotations on commands. The κ rules simulate the familiar action of “pushing” arguments and projections onto the continuation stack. In our case, those forms of continuations are just ordinary introduction rules for pairs and tagged values, except on the **to**-side of a jump. Intuitively, the $\kappa_!$ rule says that to interpret the $!M$ evaluation of a function call in a continuation N , we must build a thunk closure $\{ N \}$ to delay the continuation to pass it as the “return pointer” into M . The ζ rules correspond to Wadler’s [60] rules of the same name, and expand out unfocused constructions. Finally, the *goto* rules rewrite the new forms of commands into more primitive forms of **go to** expressible in the Dual Core calculus.

4.4.2 Reacting to the context. Additional axioms for reasoning equationally about source reactions are given in fig. 11. The first group of axioms simplifies special cases of right-hand sides with commands to other forms. The second group of axioms describes the algebraic properties of binary

Reducing special cases of right-hand sides:

$$\begin{aligned}
 (\text{eval}) \quad & \text{to } x \Rightarrow \text{go } M \text{ as } O \text{ to } x = \{ \Rightarrow M \} & (x \notin FV(M)) \\
 (\text{eval}_\bullet) \quad & \Rightarrow \text{go } M \text{ as } O = \{ \Rightarrow M \} \\
 (\text{exec}_\bullet) \quad & \text{to } x \Rightarrow J[(\text{go } M \text{ as } O \text{ to } x)/(\text{go } M \text{ as } O)] = \{ \Rightarrow J \} & (x \notin FV(J))
 \end{aligned}$$

Associativity, identity, distributivity, and annihilation of alternatives:

$$\begin{aligned}
 (\text{assoc}) \quad & R_1 \mid \{ R_2 \mid R_3 \} = \{ R_1 \mid R_2 \} \mid R_3 & (\text{id}_L) \quad \varepsilon \mid R = R & (\text{id}_R) \quad R \mid \varepsilon = R \\
 (\text{distr}_{as}) \quad & \text{as } Q \{ R_1 \mid R_2 \} = \{ \text{as } Q R_1 \} \mid \{ \text{as } Q R_2 \} & (\text{annih}_{as}) \quad \text{as } Q \varepsilon = \varepsilon \\
 (\text{distr}_{to}) \quad & \text{to } P \{ R_1 \mid R_2 \} = \{ \text{to } P R_1 \} \mid \{ \text{to } P R_2 \} & (\text{annih}_{to}) \quad \text{to } P \varepsilon = \varepsilon
 \end{aligned}$$

Reassociating and composing copatterns:

$$\begin{aligned}
 (\text{asto}_@) \quad & \text{as } (Q P') \text{ to } P R = \text{as } Q \text{ to } (P', P) R & (\text{asto}_X) \quad \text{as } (Q.X) \text{ to } P R = \text{as } Q \text{ to } X[P] R \\
 (\text{self}_{alias}) \quad & \text{as } Q \text{ as } x R = \text{as } Q R[Q/x] & (\text{self}_{anon}) \quad \text{as } x R = R & (x \notin FV(R))
 \end{aligned}$$

Catch-all cases:

$$\begin{aligned}
 (\text{commit}_{exec}) \quad & \{ \Rightarrow J \} \mid R = \{ \Rightarrow J \} & (\text{commit}_{eval}) \quad \{ \Rightarrow M \} \mid R = \{ \Rightarrow M \} \\
 (\text{commit}_{error}) \quad & \text{do } \{ R \mid _ \Rightarrow \text{go as cbv to } \underline{error} \} \rightarrow \text{do } \{ R \}
 \end{aligned}$$

Fig. 11. Additional axioms for equational reasoning about Coo reactions.

and empty reaction alternatives; intuitively, they form an almost semiring-like structure with alternation ($\mid$) as addition and ε as its unit, and the **as** and **to** forms as multiplication-like operations that distribute over them. The third group of axioms allows us to merge chains of **as** and **to** abstractions together into one. The final group of axioms handles the base cases, where we commit to a particular right-hand side, or spelling out that a whole set of alternatives will result in an error state—where error denotes a primitive continuation that signals an error and ends the program—if none of them match. Taken together, these axioms let us simplify any reaction down to a form analogous to Dual Core—as long as it contains no **if** pattern guards.

Definition 4.5 (Guardless). A source reaction, command, or (co)term, is *guardless* if it does not contain any **if** $P \Leftarrow M R$ reactions anywhere in it.

LEMMA 4.6 (REACTION ENUMERATION).

- (1) If $\Gamma \vdash R : A : \text{fun} \mid \Phi$ is guardless, then $R = \varepsilon \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \mid \dots$
- (2) If $\Gamma \vdash R : \bullet \mid \Phi$ is guardless, then either $R = \{ \Rightarrow J \}$ or $R = \varepsilon$.

While these axioms are somewhat “low-level,” we can still derive common high-level equalities from them. For example, the β axiom of (call-by-value/-push-value) λ -calculus is derived like so:

$$\begin{aligned}
 (\text{fn } x \Rightarrow M) V &= \text{do } \{ \text{as } (_ x) \Rightarrow M \} V & (\text{sugar}) \\
 &= \text{do } k \Rightarrow \text{go do } \{ \text{as } (_ x) \text{ to } k' \Rightarrow \text{go } M \text{ as cbn to } k' \} V \text{ as cbn to } k & (\eta_{act}, \text{eval}) \\
 &= \text{do } k \Rightarrow \text{go do } \{ \text{as } _ \text{ to } (x, k') \Rightarrow \text{go } M \text{ as cbn to } k' \} \text{ as cbn to } (V, k) & (\kappa_@, \text{asto}_@) \\
 &= \text{do } k \Rightarrow \text{go } M[V/x] \text{ as cbn to } k = M[V/x] & (\beta_{DC}, \eta_{act})
 \end{aligned}$$

5 Go To Coup: Translation to the Dual Core Sequent Calculus

Having specified the source programming language Coo, we now show how to translate it to the intermediate language using only features from the Dual Core sequent calculus. This translation is given in figs. 12 to 13, which assumes that the input program is polarity-elaborated definition 4.1.

Translating introduction (co)terms:

$$\begin{aligned} \llbracket x \rrbracket &= x & \llbracket X[M] \rrbracket &= \langle X[\llbracket M \rrbracket] \rangle & \llbracket \{ M \} \rrbracket &= \{ \llbracket M \rrbracket \} \\ \llbracket () \rrbracket &= () & \llbracket (M, N) \rrbracket &= \langle \llbracket M \rrbracket, \llbracket N \rrbracket \rangle \end{aligned}$$

Translating elimination (co)terms:

$$\begin{aligned} \llbracket M N \rrbracket &= \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \langle \llbracket N \rrbracket, \underline{\text{here}} \rangle \\ \llbracket M.X \rrbracket &= \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } X[\underline{\text{here}}] \\ \llbracket !M \rrbracket &= \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \{ \underline{\text{here}} \} \end{aligned}$$

Translating **do**-expressions

$$\begin{aligned} \llbracket \text{do } x \Rightarrow J \rrbracket &= \text{do } x \Rightarrow \llbracket J \rrbracket & \llbracket \text{do } \Rightarrow J \rrbracket &= \text{do } \underline{\text{exit}} \Rightarrow \llbracket J \rrbracket \\ \llbracket \text{do } R \rrbracket &= \text{do } \{ | \text{ as } \underline{\text{self}} \text{ to } \underline{\text{here}} \Rightarrow \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_1) \text{ as cbv} & \text{(otherwise)} \\ & \quad \text{to } (\text{do } _ \Rightarrow \text{go } () \text{ as cbv to } \underline{\text{error}}) \} \end{aligned}$$

Translation of commands:

$$\begin{aligned} \llbracket \text{go } M \text{ as } O \text{ to } N \rrbracket &= \text{go } \llbracket M \rrbracket \text{ as strat}(O) \text{ to } \llbracket N \rrbracket \\ \llbracket \text{go } M \text{ as } O \rrbracket &= \text{go } \llbracket M \rrbracket \text{ as strat}(O) \text{ to } \underline{\text{exit}} \\ \llbracket \text{fun } R; J \rrbracket &= \text{go } \llbracket \text{do } R \rrbracket \text{ as cbn to do name}(R) \Rightarrow \llbracket J \rrbracket \\ \llbracket \text{val } P \Leftarrow M \text{ else } R; J \rrbracket &= \text{go } \llbracket M \rrbracket \text{ as cbv to } \llbracket \text{do } \{ P \Rightarrow J \mid R \} \rrbracket \\ \llbracket \text{match } M R; J \rrbracket &= \text{go } (\text{do } \underline{\text{next}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbv to } \llbracket \text{do } R \rrbracket) \\ & \quad \text{as cbv to do } \{ | () \Rightarrow \llbracket J \rrbracket \} \\ \llbracket \varepsilon \rrbracket &= \text{go } () \text{ as cbv to } \underline{\text{next}} \end{aligned}$$

Extracting a strategy from an annotation:

$$\begin{aligned} \text{strat}(I) &= I & \text{strat}(I : A) &= I & \text{strat}(O) &\text{undefined otherwise} \end{aligned}$$

Fig. 12. Translation of source Coo (co)terms and commands to Dual Core calculus.

The first part of the translation, shown in fig. 12, covers (co)terms and commands. Following essentially the same translation from natural deduction to sequent calculus from Gentzen [33] in 1935, the introductory (co)terms can be left the same, while the elimination forms follow a η_{act} expansion and a κ reassociation to push the elimination to the other side of the cut as an equivalent introduction. This requires binding a new local variable to refer to the current continuation, which we will systematically name here throughout. The only other (co)terms are **do**-abstractions. An general **do** $x \Rightarrow J$ is left unchanged, an implicit **do** $\Rightarrow J$ introduces an unstated static variable—which we will systematically name exit—to be referenced later by a **go** without a **to**. Finally, all other complex reactions **do** R are (potentially) recursive forms, so we introduce a standardized self-referential name self, bind the current continuation to here, and bind the alternative continuation if R fails to other, which will go to an error state signaling pattern-match failure.²

The rules for translating commands essentially follow the equations in fig. 10 to convert every command into some **go to**, to the point where it replaces every other statement in the language. Types are erased from annotations O , and the implicit **go** without a **to** is given the explicit target **to** exit, which is statically bound by the nearest lexical $\llbracket \text{do } \Rightarrow J \rrbracket$. Finally, the reaction in a **match**

²This is a place where we can generate and pass useful error messages pointing to the origin of the failure for debugging.

$$\begin{aligned}
\llbracket \varepsilon \rrbracket_1 &= \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}} & \llbracket \varepsilon \rrbracket_0 &= \text{go } () \text{ as cbv to } \underline{\text{other}} \\
\llbracket R_1 \mid R_2 \rrbracket_n &= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket R_1 \rrbracket_n) \text{ as cbv to } (\text{do } \underline{\text{here}} \Rightarrow \llbracket R_2 \rrbracket_n) \\
\llbracket \text{if } P \Leftarrow M \rrbracket_n &= \text{go } \llbracket M \rrbracket \text{ as cbv to do } \{ \mid P \Rightarrow \llbracket R \rrbracket_n \mid _ \Rightarrow \llbracket \varepsilon \rrbracket_n \} \\
\llbracket \text{as } (_ P) \rrbracket_1 &= \text{go do } \{ \mid (P, \underline{\text{here}}) \Rightarrow \text{go } \llbracket \underline{\text{self}} P \rrbracket \text{ as cbn} \\
&\quad \text{to do } \underline{\text{self}} \Rightarrow \text{go do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_1 \\
&\quad \text{as cbv to } \llbracket \underline{\text{other}} P \rrbracket \\
&\quad \mid \underline{\text{here}} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{\text{here}} \\
\llbracket \text{as } (Q P) \rrbracket_1 &= \llbracket \text{as } Q \text{ as } (_ P) \rrbracket_1 & (\text{otherwise}) \\
\llbracket \text{as } (_ X) \rrbracket_1 &= \text{go do } \{ \mid X[\underline{\text{here}}] \Rightarrow \text{go } \llbracket \underline{\text{self}}.X \rrbracket \text{ as cbn} \\
&\quad \text{to do } \underline{\text{self}} \Rightarrow \text{go do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_1 \\
&\quad \text{as cbv to } \llbracket \underline{\text{other}}.X \rrbracket \\
&\quad \mid \underline{\text{here}} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{\text{here}} \\
\llbracket \text{as } (Q.X) \rrbracket_1 &= \llbracket \text{as } Q \text{ as } (_ X) \rrbracket_1 & (\text{otherwise}) \\
\llbracket \text{as } x \rrbracket_1 &= \text{go } \underline{\text{self}} \text{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1 \\
\llbracket \text{to } P \rrbracket_1 &= \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_0 \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{\text{other}} \\
&\quad \mid \underline{\text{here}} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{\text{here}} \\
\llbracket \Rightarrow J \rrbracket_0 &= \llbracket J \rrbracket & \llbracket \Rightarrow J \rrbracket_1 &= \text{go } (\text{do } \underline{\text{exit}} \Rightarrow \llbracket J \rrbracket) \text{ as cbn to } \underline{\text{here}} \\
\llbracket \Rightarrow M \rrbracket_1 &= \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{here}} & \llbracket \Rightarrow M \rrbracket_0 &= \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{exit}}
\end{aligned}$$

Fig. 13. Translation of source Coe reactions to commands of the Dual Core calculus.

(and special case *if*) statement will automatically step to the following command, which is handled by binding the continuation to the variable *next*, which the empty command $\llbracket \varepsilon \rrbracket$ jumps to.

The translation of reactions is given in fig. 13, which is the most involved because reactions have to juggle the incremental deconstruction of some value with multiple failure paths that retry from the start with another alternative, all while establishing (potentially multiple) recursive self-bindings. To juggle all the tasks at once, each case of translating a reaction assumes some standard variables (not used in the source program) are in scope for standardized purposes: (1) *here* is the context we are matching on, (2) *other* is the alternative to invoke if this reaction can't handle *here*, and (3) *self* is the “recursive” name for itself. Additionally, the translation for reactions $\llbracket R \rrbracket_n$ is indexed by a number $n \in \{0, 1\}$ indicating whether there is still some value remaining to destruct (1) or if it has been completely consumed (0) by a *to*. In the latter case, remove the assumptions about *here* (as it has been matched completely) and *self* being in scope. One repeated detail in the translation is that every time we match on the input in *here*, we have to update *other* and *self* (if it remains in scope) to reflect the current view from the newly rebound *here* denoting the remainder.

Note that the reverse translation is essentially a no-op, since Dual Core is a sub-syntax of the source language. Nevertheless, we explicitly denote the reverse translation as $\llbracket _ \rrbracket^{-1}$ to avoid

ambiguity. The main detail is that the list of alternatives in a (co)pattern-matching **do** denotes a nested sequence of alternative reactions ($R \mid R'$) like so:

$$\begin{aligned} & \llbracket \mathbf{do} \{ \mid \mathbf{as} x_1 \mathbf{to} P_1 \Rightarrow J_1 \mid \mathbf{as} x_2 \mathbf{to} P_2 \Rightarrow J_2 \cdots \mid \mathbf{as} x_n \mathbf{to} P_n \Rightarrow J_n \} \rrbracket^{-1} \\ & = \mathbf{do} \{ \{ \varepsilon \mid \mathbf{as} x_1 \mathbf{to} P_1 \Rightarrow \llbracket J_1 \rrbracket^{-1} \} \mid \mathbf{as} x_2 \mathbf{to} P_2 \Rightarrow \llbracket J_2 \rrbracket^{-1} \dots \} \mid \mathbf{as} x_n \mathbf{to} P_n \Rightarrow \llbracket J_n \rrbracket^{-1} \} \end{aligned}$$

5.1 Translation Correctness

The translation, especially regarding reactions, can be subtle, so we specify the notion of correctness (with proofs in sections G and H) to make sure the meaning of source programs is preserved in the target. The simplest preservation is for polarity, which holds in both directions.

- LEMMA 5.1 (POLARITY SOUNDNESS & COMPLETENESS). (1) *If a source J is well-polarized (respectively, (co)term M) then so is $\llbracket J \rrbracket$ (respectively, $\llbracket M \rrbracket$).*
 (2) *If a target J is well-polarized (respectively, M or V) so is $\llbracket J \rrbracket^{-1}$ (respectively, $\llbracket M \rrbracket^{-1}$ or $\llbracket V \rrbracket^{-1}$).*

The preservation of types is, thankfully, much simpler with a sequent calculus translation than continuation-passing style, because the types themselves don't change. If $M : A$ in the source, then $\llbracket M \rrbracket : A$ in the target. The only real detail we need to consider is how the implicit static variables introduced by the translation correspond to the original typing judgments. These extra implicit features can be translated to the following explicit bindings below to preserve typing:

$$\begin{aligned} Ex \llbracket \Psi \rrbracket &= \underline{exit} : \Psi^\circ & \llbracket \mathbf{whole} \rrbracket &= \bullet & \llbracket \mathbf{part} \rrbracket &= \underline{next} : () : \mathbf{fun} \\ Ex \llbracket \Xi \Psi \rrbracket &= \llbracket \Xi \rrbracket, Ex \llbracket \Psi \rrbracket & x : \bullet &= \bullet & \Gamma_{err} &= \underline{error} : () : \mathbf{fun} \\ Re \llbracket A : \mathbf{fun} \rrbracket &= \underline{other} : A : \mathbf{fun}, \underline{here} : A : \mathbf{val}, \underline{self} : A : \mathbf{fun} & Re \llbracket \bullet \rrbracket &= \underline{other} : () : \mathbf{fun} \end{aligned}$$

THEOREM 5.2 (TYPE SOUNDNESS). *Give any polarity elaborated J , M , or R ,*

- (1) *If $\Gamma \vdash J \mid \Phi$ then $\Gamma, \Gamma_{err}, Ex \llbracket \Phi \rrbracket \vdash \llbracket J \rrbracket$.*
- (2) *If $\Gamma \vdash M : A : K \mid \Psi$ then $\Gamma, \Gamma_{err}, Ex \llbracket \Psi \rrbracket \vdash \llbracket M \rrbracket : A : K$.*
- (3) *If $\Gamma \vdash R : \Psi \mid \Phi$ then $\Gamma, \Gamma_{err}, Ex \llbracket \Phi \rrbracket, Re \llbracket \Psi \rrbracket \vdash \llbracket R \rrbracket_{\llbracket \Psi \rrbracket}$.*

THEOREM 5.3 (TYPE COMPLETENESS). (1) *If $\Gamma \vdash J$ then $\Gamma \vdash \llbracket J \rrbracket^{-1} \mid \mathbf{whole} \bullet$.*

- (2) *If $\Gamma \vdash M : A : K$ then $\Gamma \vdash \llbracket M \rrbracket^{-1} : A : K \mid \bullet$.*
- (3) *If $\Gamma \vdash V : A$; then $\Gamma \vdash \llbracket V \rrbracket^{-1} : A : \mathbf{val} \mid \bullet$.*

Finally, the equational theories of the source and target correspond, too. Every source equality is sound with respect to the Dual Core sequent calculus, and moreover, the source equational theory is complete for guardless (definition 4.5) programs. In fact, we have a stronger notion than the usual soundness: because Dual Core is a sub-calculus of the source language, every guardless source program is *already equal* to some target form within the equational theory.

LEMMA 5.4 (DESUGARING EQUALITY). *In the source equational theory,*

- (1) *If J is guardless, then there is a Dual Core command J' such that $J = \llbracket J' \rrbracket^{-1}$.*
- (2) *If M is guardless, then there is a Dual Core (co)term M' such that $M = \llbracket M' \rrbracket^{-1}$.*

THEOREM 5.5 (EQUATIONAL SOUNDNESS). (1) *If $J = J'$ then $\llbracket J \rrbracket = \llbracket J' \rrbracket$ when defined.*

- (2) *If $M = M'$ then $\llbracket M \rrbracket = \llbracket M' \rrbracket$ when defined.*
- (3) *If $R = R'$ then $\llbracket R \rrbracket_n = \llbracket R' \rrbracket_n$ when defined.*

THEOREM 5.6 (EQUATIONAL COMPLETENESS). (1) *For guardless J, J' , if $\llbracket J \rrbracket = \llbracket J' \rrbracket$ then $J = J'$.*

- (2) *For guardless M, M' , if $\llbracket M \rrbracket = \llbracket M' \rrbracket$ then $M = M'$.*
- (3) *For guardless R, R' , if $\llbracket R \rrbracket_n = \llbracket R' \rrbracket_n$ then $R = R'$.*

6 A Leap of Faith: Implementing Interpretation from the Inside

The reference implementation of Coq provides an interpreter for both the source and intermediate languages described here, including an abstract machine for Dual Core, automatic polarity correction, optional static type checking, and the simplifications in lemma 4.6. Some details of the real implementation were elided for simplification of this specification. For example, in reality, *both* focused and unfocused forms are allowed in the implementation of Dual Core, which strikes a balance for ease of both code generation and execution. For instance, since there is no need to generate names to bind the components of structures, the *only* new local variables introduced during translation are the underlined ones in figs. 12 to 13, which is the same for every source program, making hygiene statically simple. Instead, focusing can be done either before or at runtime.

The interpreter supports both direct execution of program files and an interactive read-eval-print-loop (REPL). But this raises the question: what *is* a REPL for a sequent-based language? Computation in the sequent calculus is about *commands* which don't return (and thus, do not evaluate to "answers"), instead of expressions which do. Rather than assigning an ad-hoc notion of "evaluation" for commands, the interactive interpreter is instead *inside out*: the abstract machine runs *first* and then calls a print-read-process continuation to prompt the user for more commands to execute when it runs out. A pleasant side effect of this inside-out architecture is that the interactive prompt is fully integrated with the control-flow of the program using the exact same semantics as other execution. For example, consider this user interaction at the prompt:

```
0> val x, {prompt0} = do k => go 0, {k} to k
1> x
= 0
2> val y, {prompt2} = prompt0 10
1> x
= 10
2> prompt0(x*x)
3> x
= 0
4> y
= 100
```

Each prompt is marked with its number in sequence from the beginning, starting at 0 and incrementing with each line. On the first line, the user captured the REPL at prompt 0, to use later. Then, when the call to `prompt0 10` is run on line 2, the interpreter jumps *backwards* to line 1. When that happens, `x` has been "updated" to the argument's value 10 and `prompt0` is bound to the calling context `val y, {prompt2} = □`. After that, `prompt0` is called again with `x*x` as the argument, which resets `x` back to its original value 0 and finishes the assignment of `y` to 100. Note that `y` is not yet bound until *after* `prompt0` is called again on the second line 2, resuming the interrupted assignment `val y, {prompt2} = □` that was deferred, captured, and bound to `prompt0`.

7 Related Work

Since its first appearance in the early 1900s [33], the sequent calculus has been used countless times in the theory of logic, proof search, and, eventually, fundamental models of computation; a non-exhaustive account of early works in this last area include [5, 10, 11, 19, 20, 22, 37–39, 46, 60, 61]. In contrast, use of the sequent calculus for *practical* concerns in the design and implementation of programming languages is far more sparse, and mostly about its use in compilers and their intermediate languages for optimization and code generation. Some early examples of this are in the line of work by Ohori *et al.* [41, 49–52]. Another version of this idea is the Sequent Core [28] alternative intermediate language for the Glasgow Haskell Compiler. If we compare Sequent Core to Dual Core, they seem fairly different; that's because they were designed with different goals in mind.

Sequent Core enjoys a one-for-one correspondence with pure functional programs, and so it inherits the asymmetries of those programs. In contrast, Dual Core is designed to allow for perfect dualities between values and continuations, and to express the side effects of unrestricted, non-functional control flow. A more recent use of sequent calculus in compilers is AxCut [56], which is even more low-level, and designed to generate efficient machine code directly. Like Dual Core, AxCut embraces perfect dualities, and has the most similar type system out of all the alternatives above since it reduces the number of overall different types by only considering (positive) data types with producers and consumers, similar to the `val` versus `fun` classification of the same interfaces here.

Turning attention to other uses of sequent calculus in high-level *source* languages, the field is even more sparse [2]. Nevertheless, there are some examples of languages that take similar inspiration. Duo [13, 14] is a notable early example of a usable language that exhibits many similar symmetries between data and codata types. Another is the Polarity language [15, 44], which derives a principled notion of dependent types for object-oriented programs using duality between data and codata types. And while it is more designed as a primarily functional language, the small source language of [16] does inherit some features from its sequent calculus core. For other aspects of the source language, Binder and Ermantraut [12] investigated similar kinds of algebraic properties for patterns more like those found in purely functional languages, rather than control-flow integrated reactions. Our translation of reactions into smaller steps connected by join points is most similar to the translation used for Haskell [55] and untyped Scheme macros for copatterns [26].

8 Future Work

The language design is still just an initial foray into how to tame sequent-calculus fundamentals into a usable programming language. There is still plenty of room to grow to adopt more high-level features, especially regarding the type system. The two most pressing needs unaddressed by the simple type system presented here are a facility for user-defined, recursive (co)data types along with parametric polymorphism, but that is just a start towards incorporating advanced type features from realistic languages. Type inference is also still an open question about how best to accommodate the new sequent-based programs described here, although algorithms based on [14] may prove fruitful. The other clear open direction of extension is to express even more expressive control flow like delimited control [21, 31]. Since there is already an inherent notion of `call/cc`-like control, one possible minimal extension to achieve delimited control is with dynamic binding [4, 8, 23].

While the focus here is on high-level issues, there is also much room for improvement in lower-level ones, too, connecting with the related work on using the sequent calculus for optimization and code generation. For instance, the translation shown here for reactions relies on local bindings like the recursive *self* and alternative *other*; these are essentially just join points [28, 45] since they cannot be passed as arguments unless wrapped in a closure like { *self* }. The current implementation does not attempt to optimize these calls into efficient jumps that re-use the same local binding environment, but this seems like a feasible optimization. In terms of closures, they are actually represented internally only on thunk values following the notion of closure-conversion for call-by-push-value [59], which could be generated and optimized before runtime. The biggest improvement, of course, would come from generating efficient machine code. Since Dual Core is similar to the AxCut [56] representation, connecting them would give a path for direct compilation.

9 Conclusion

Here, we presented a new programming language, *Coo*, for programming directly with the sequent calculus for writing new kinds of programs with an invisible duality and permeable transfer between values and contexts. We can blend multiple ways of interfacing with codata objects: either natural-style applications and “dot” projections, or constructing and sending messages wholesale.

These same affordances are allowed when (co)pattern-matching, and more generally, *reacting* to values or contexts. To make this possible, several usability features are introduced to make it easier to program in directly, including automatic polarity correction and both implicit and explicit control flow. All this translates down to a small, symmetric core based on the classical sequent calculus.

Acknowledgments

References

- [1] Andreas Abel, Brigitte Pientka, David Thibodeau, and Anton Setzer. 2013. Copatterns: Programming Infinite Structures by Observations. In *Proceedings of the 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Rome, Italy) (POPL '13). Association for Computing Machinery, New York, NY, USA, 27–38. doi:10.1145/2429069.2429075
- [2] Michael D. Adams. 2025. Scheme and New Frontiers for Language Design (Keynote). In *Proceedings of the 26th ACM SIGPLAN International Workshop on Scheme and Functional Programming* (Singapore, Singapore) (Scheme '25). Association for Computing Machinery, New York, NY, USA, 1. doi:10.1145/3759537.3771265
- [3] Jean-Marc Andreoli. 1992. Logic Programming with Focusing Proofs in Linear Logic. *Journal of Logic and Computation* 2, 3 (1992), 297–347. doi:10.1093/logcom/2.3.297
- [4] Zena Ariola, Hugo Herbelin, and Amr Sabry. 2007. A Type-Theoretic Foundation of Delimited Continuations. *Higher-Order and Symbolic Computation* 22 (2007), 233–273. Issue 3. doi:10.1007/s10990-007-9006-0
- [5] Zena M. Ariola, Aaron Bohannon, and Amr Sabry. 2009. Sequent calculi and abstract machines. *ACM Transactions on Programming Languages and Systems* 31, 4, Article 13 (May 2009), 48 pages. doi:10.1145/1516507.1516508
- [6] Zena M. Ariola, Paul Downen, and Hugo Herbelin. 2026. Proving and Computing: The Infinite Pigeonhole Principle and Countable Choice. *Electronic Proceedings in Theoretical Computer Science* 441 (March 2026), 1–18. doi:10.4204/eptcs.441.1
- [7] Zena M. Ariola and Hugo Herbelin. 2003. Minimal Classical Logic and Control Operators. In *Automata, Languages and Programming*. Springer Berlin Heidelberg, Berlin, Heidelberg, 871–885. doi:10.1007/3-540-45061-0_68
- [8] Zena M. Ariola, Hugo Herbelin, and Amr Sabry. 2004. A type-theoretic foundation of continuations and prompts. In *Proceedings of the Ninth ACM SIGPLAN International Conference on Functional Programming* (Snow Bird, UT, USA) (ICFP '04). Association for Computing Machinery, New York, NY, USA, 40–53. doi:10.1145/1016850.1016860
- [9] Zena M. Ariola, Hugo Herbelin, and Alexis Saurin. 2011. Classical Call-By-Need and Duality. In *Typed Lambda Calculi and Applications: 10th International Conference* (Novi Sad, Serbia) (TLCA '11). Springer Berlin Heidelberg, Berlin, Heidelberg, 27–44. doi:10.1007/978-3-642-21691-6_6
- [10] Franco Barbanera and Stefano Berardi. 1994. A symmetric lambda calculus for “classical” program extraction. In *Theoretical Aspects of Computer Software*, Masami Hagiya and John C. Mitchell (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 495–515. doi:10.1007/3-540-57887-0_112
- [11] Henk Barendregt and Silvia Ghilezan. 2000. Lambda terms for natural deduction, sequent calculus and cut elimination. *Journal of Functional Programming* 10, 1 (Jan. 2000), 121–134. doi:10.1017/S0956796899003524
- [12] David Binder and Lean Ermantraut. 2025. The Algebra of Patterns. In *39th European Conference on Object-Oriented Programming (ECOOP 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 333)*, Jonathan Aldrich and Alexandra Silva (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2:1–2:28. doi:10.4230/LIPIcs.ECOOP.2025.2
- [13] David Binder, Julian Jabs, Ingo Skupin, and Klaus Ostermann. 2019. Decomposition diversity with symmetric data and codata. *Proc. ACM Program. Lang.* 4, POPL, Article 30 (Dec. 2019), 28 pages. doi:10.1145/3371098
- [14] David Binder, Ingo Skupin, David Läwen, and Klaus Ostermann. 2022. Structural refinement types. In *Proceedings of the 7th ACM SIGPLAN International Workshop on Type-Driven Development* (Ljubljana, Slovenia) (TyDe 2022). Association for Computing Machinery, New York, NY, USA, 15–27. doi:10.1145/3546196.3550163
- [15] David Binder, Ingo Skupin, Tim Süßerkrüb, and Klaus Ostermann. 2024. Deriving Dependently-Typed OOP from First Principles. *Proc. ACM Program. Lang.* 8, OOPSLA1, Article 129 (April 2024), 27 pages. doi:10.1145/3649846
- [16] David Binder, Marco Tzschentke, Marius Müller, and Klaus Ostermann. 2024. Grokking the Sequent Calculus (Functional Pearl). *Proc. ACM Program. Lang.* 8, ICFP, Article 250 (Aug. 2024), 31 pages. doi:10.1145/3674639
- [17] Edwin Brady. 2015. Resource-Dependent Algebraic Effects. In *Trends in Functional Programming*, Jurriaan Hage and Jay McCarthy (Eds.). Springer International Publishing, Cham, 18–33. doi:10.1007/978-3-319-14675-1_2
- [18] Edwin Brady. 2021. Idris 2: Quantitative Type Theory in Practice. In *35th European Conference on Object-Oriented Programming (ECOOP 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 194)*, Anders Möller and Manu Sridharan (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 9:1–9:26. doi:10.4230/LIPIcs.ECOOP.2021.9

- [19] Pierre-Louis Curien and Hugo Herbelin. 2000. The duality of computation. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. Association for Computing Machinery, New York, NY, USA, 233–243. doi:10.1145/351240.351262
- [20] Pierre-Louis Curien and Guillaume Munch-Maccagnoni. 2010. The Duality of Computation under Focus. In *Theoretical Computer Science*, Cristian S. Calude and Vladimiro Sassone (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 165–181. doi:10.1007/978-3-642-15240-5_13
- [21] Olivier Danvy and Andrzej Filinski. 1990. Abstracting control. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming (Nice, France) (LFP '90)*. Association for Computing Machinery, New York, NY, USA, 151–160. doi:10.1145/91556.91622
- [22] Daniel Dougherty, Silvia Ghilezan, Pierre Lescanne, and Silvia Likavec. 2005. Strong Normalization of the Dual Classical Sequent Calculus. In *Logic for Programming, Artificial Intelligence, and Reasoning*, Geoff Sutcliffe and Andrei Voronkov (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 169–183. doi:10.1007/11591191_13
- [23] Paul Downen and Zena M. Ariola. 2014. Delimited control and computational effects. *Journal of Functional Programming* 24, 1 (2014), 1–55. doi:10.1017/S0956796813000312
- [24] Paul Downen and Zena M. Ariola. 2014. The Duality of Construction. In *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*. Springer-Verlag, Berlin, Heidelberg, 249–269. doi:10.1007/978-3-642-54833-8_14
- [25] Paul Downen and Zena M. Ariola. 2020. Compiling With Classical Connectives. *Logical Methods in Computer Science* 16, 3 (Aug. 2020), 13:1–13:57. doi:10.23638/LMCS-16(3:13)2020
- [26] Paul Downen and Adriano Corbelino II. 2025. CoScheme: Compositional Copatterns in Scheme. In *Trends in Functional Programming - 26th International Symposium, TFP 2025, Oxford, UK, January 14-16, 2025, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 15652)*, Jeremy Gibbons (Ed.). Springer, Cham, 205–258. doi:10.1007/978-3-031-99751-8_10
- [27] Paul Downen, Philip Johnson-Freyd, and Zena M. Ariola. 2015. Structures for structural recursion. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming (Vancouver, BC, Canada) (ICFP 2015)*. Association for Computing Machinery, New York, NY, USA, 127–139. doi:10.1145/2784731.2784762
- [28] Paul Downen, Luke Maurer, Zena M. Ariola, and Simon Peyton Jones. 2016. Sequent calculus as a compiler intermediate language. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming (Nara, Japan) (ICFP 2016)*. Association for Computing Machinery, New York, NY, USA, 74–88. doi:10.1145/2951913.2951931
- [29] Paul Downen, Zachary Sullivan, Zena M. Ariola, and Simon Peyton Jones. 2019. Codata in Action. In *Programming Languages and Systems - 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6-11, 2019, Proceedings (Prague, Czech Republic) (Lecture Notes in Computer Science, Vol. 11423)*. Springer International Publishing, Cham, 119–146. doi:10.1007/978-3-030-17184-1_5
- [30] Martin Erwig and Simon Peyton Jones. 2001. Pattern Guards and Transformational Patterns. *Electronic Notes in Theoretical Computer Science* 41, 1 (2001), 3–29. doi:10.1016/S15710661(05)80540-7
- [31] Matthias Felleisen, Mitch Wand, Daniel Friedman, and Bruce Duba. 1988. Abstract continuations: a mathematical semantics for handling full jumps. In *Proceedings of the 1988 ACM Conference on LISP and Functional Programming (Snowbird, Utah, USA) (LFP '88)*. Association for Computing Machinery, New York, NY, USA, 52–62. doi:10.1145/62678.62684
- [32] Andrzej Filinski. 1989. *Declarative Continuations and Categorical Duality*. Master's thesis. Computer Science Department, University of Copenhagen.
- [33] Gerhard Gentzen. 1935. Untersuchungen über das logische Schließen. I. *Mathematische Zeitschrift* 39, 1 (1935), 176–210. doi:10.1007/BF01201353
- [34] Jean-Yves Girard. 1987. Linear logic. *Theoretical Computer Science* 50, 1 (1987), 1–101. doi:10.1016/0304-3975(87)90045-4
- [35] Timothy G. Griffin. 1989. A formulae-as-type notion of control. In *Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (San Francisco, California, USA) (POPL '90)*. Association for Computing Machinery, New York, NY, USA, 47–58. doi:10.1145/96709.96714
- [36] Tatsuya Hagino. 1989. Codatatypes in ML. *Journal of Symbolic Computation* 8, 6 (1989), 629 – 650. doi:10.1016/S0747-7171(89)80065-3
- [37] Hugo Herbelin. 1995. A λ -calculus structure isomorphic to Gentzen-style sequent calculus structure. In *Computer Science Logic, Leszek Pacholski and Jerzy Tiuryn (Eds.)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 61–75. doi:10.1007/BFB0022247
- [38] Hugo Herbelin. 1995. *Séquents qu'on calcule: de l'interprétation du calcul des séquents comme calcul de λ -termes et comme calcul de stratégies gagnantes*. Ph. D. Dissertation. Université Paris 7.
- [39] Hugo Herbelin. 2005. *C'est maintenant qu'on calcule : Au coeur de la dualité*. Habilitation thesis. Université Paris 11.

- [40] Roshan P. James and Amr Sabry. 2011. Yield: Mainstream Delimited Continuations. In *Proceedings of the First International Workshop on the Theory and Practice of Delimited Continuations (TPDC)*. Austin, Texas, 20.
- [41] Shin-ya Katsumata and Atsushi Ohori. 2001. Proof-Directed De-compilation of Low-Level Code. In *Programming Languages and Systems*, David Sands (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 352–366. doi:10.1007/3-540-45309-1_23
- [42] Olivier Laurent. 2002. *Étude de la polarisation en logique*. Ph. D. Dissertation. Université de la Méditerranée - Aix-Marseille II.
- [43] Paul Blain Levy. 2001. *Call-By-Push-Value*. Ph. D. Dissertation. Queen Mary and Westfield College, University of London.
- [44] Bohdan Liesnikov, David Binder, and Tim Süßerkrüb. 2025. Filling the Gaps of Polarity: Implementing Dependent Data and Codata Types with Implicit Arguments. *Art Sci. Eng. Program*. 10, 3 (2025), 41 pages. doi:10.22152/PROGRAMMING-JOURNAL.ORG/2025/10/19
- [45] Luke Maurer, Paul Downen, Zena M. Ariola, and Simon Peyton Jones. 2017. Compiling Without Continuations. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) (PLDI 2017). ACM, New York, NY, USA, 482–494. doi:10.1145/3062341.3062380
- [46] Guillaume Munch-Maccagnoni. 2009. Focalisation and Classical Realisability. In *Computer Science Logic: 23rd international Workshop, CSL 2009, 18th Annual Conference of the EACSL (Coimbra, Portugal) (CSL 2009)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 409–423. doi:10.1007/978-3-642-04027-6_30
- [47] Guillaume Munch-Maccagnoni. 2013. *Syntax and Models of a non-Associative Composition of Programs and Proofs*. Ph. D. Dissertation. Université Paris Diderot.
- [48] Guillaume Munch-Maccagnoni and Gabriel Scherer. 2015. Polarised Intermediate Representation of Lambda Calculus with Sums. In *Proceedings of the 2015 30th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '15)*. IEEE Computer Society, USA, 127–140. doi:10.1109/LICS.2015.22
- [49] Atsushi Ohori. 1999. A Curry-Howard Isomorphism for Compilation and Program Execution. In *Typed Lambda Calculi and Applications*, Jean-Yves Girard (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 280–294. doi:10.5555/645894.671753
- [50] Atsushi Ohori. 1999. The Logical Abstract Machine: A Curry-Howard Isomorphism for Machine Code. In *Functional and Logic Programming*, Aart Middeldorp and Taisuke Sato (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 300–318. doi:10.1007/10705424_20
- [51] Atsushi Ohori. 2004. Register allocation by proof transformation. *Science of Computer Programming* 50, 1 (2004), 161–187. doi:10.1016/j.scico.2004.01.005 12th European Symposium on Programming (ESOP 2003).
- [52] Atsushi Ohori. 2007. A proof theory for machine code. *ACM Trans. Program. Lang. Syst.* 29, 6 (Oct. 2007), 36–es. doi:10.1145/1286821.1286827
- [53] Klaus Ostermann, David Binder, Ingo Skupin, Tim Süßerkrüb, and Paul Downen. 2022. Introduction and elimination, left and right. *Proc. ACM Program. Lang.* 6, ICFP, Article 106 (Aug. 2022), 28 pages. doi:10.1145/3547637
- [54] Michel Parigot. 1992. $\lambda\mu$ -Calculus: An Algorithmic Interpretation of Classical Natural Deduction. In *Logic Programming and Automated Reasoning: International Conference* (St. Petersburg, Russia) (LPAR '92), Andrei Voronkov (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 190–201. doi:10.1007/BFb0013061
- [55] Simon L. Peyton Jones. 1987. *The Implementation of Functional Programming Languages*. Prentice Hall, USA. <https://www.microsoft.com/en-us/research/publication/the-implementation-of-functional-programming-languages-2/>
- [56] Philipp Schuster, Marius Müller, Klaus Ostermann, and Jonathan Immanuel Brachthäuser. 2025. Compiling Classical Sequent Calculus to Stock Hardware: The Duality of Compilation. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 142 (April 2025), 28 pages. doi:10.1145/3720507
- [57] Olin Shivers and David Fisher. 2004. Multi-return function call. In *Proceedings of the Ninth ACM SIGPLAN International Conference on Functional Programming* (Snow Bird, UT, USA) (ICFP '04). Association for Computing Machinery, New York, NY, USA, 79–89. doi:10.1145/1016850.1016864
- [58] Arnaud Spiwack. 2014. A Dissection of L. <https://assert-false.science/arnaud/papers/A%20dissection%20of%20L.pdf>
- [59] Zachary J. Sullivan, Paul Downen, and Zena M. Ariola. 2023. Closure Conversion in Little Pieces. In *Proceedings of the 25th International Symposium on Principles and Practice of Declarative Programming* (Lisboa, Portugal) (PPDP '23). Association for Computing Machinery, New York, NY, USA, Article 10, 13 pages. doi:10.1145/3610612.3610622
- [60] Philip Wadler. 2003. Call-by-value is dual to call-by-name. In *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming* (Uppsala, Sweden) (ICFP '03). Association for Computing Machinery, New York, NY, USA, 189–201. doi:10.1145/944705.944723
- [61] Noam Zeilberger. 2008. On the Unity of Duality. *Annals of Pure and Applied Logic* 153, 1 (2008), 660–96. doi:10.1016/j.apal.2008.01.001
- [62] Noam Zeilberger. 2009. *The Logical Basis of Evaluation Order and Pattern-Matching*. Ph. D. Dissertation. Carnegie Mellon University.

CONTENTS

Abstract	1
1 Introduction	1
2 Programming With and Without the Sequent Calculus	3
2.1 Simple Programs in a Natural Style	3
2.2 Evaluation Order and Code Versus Data	4
2.3 Codata and Copatterns	5
2.4 Sequential and Non-Functional Control	6
2.5 Unusual Filter Functions	7
3 Dual Core: A Sequent Calculus Intermediate Language	8
3.1 Syntax	9
3.2 Operational Semantics	9
3.3 Type System	10
3.4 Equational Theory	12
4 Coo: Combining Natural and Sequent Calculus Programs	12
4.1 Syntax	12
4.2 Type System	14
4.3 Automatic Drive: Static Polarity Inference and Correction	16
4.4 Equational Theory	19
5 Go To Coup: Translation to the Dual Core Sequent Calculus	20
5.1 Translation Correctness	23
6 A Leap of Faith: Implementing Interpretation from the Inside	24
7 Related Work	24
8 Future Work	25
9 Conclusion	25
Acknowledgments	26
References	26
Contents	29
List of Figures	30
List of Theorems	30
A Extended Examples of Coo Programs	32
A.1 Recursively Looping Matches	32
A.2 Handling and Capturing Failure	32
A.3 Yielding Control to Stateful Objects	35
A.4 Infinite Pigeonhole Principle	36
B “Is It Really the Sequent Calculus?”	38
C Additional Definitions and Extensions	41
C.1 Dual Core Polarity	41
C.2 Else Reactions	42
D Example Derivations	42
D.1 Derived Shifts	42
D.2 Unfocused Constructions	43
D.3 Equalities About Recursive Matching	44
E Dual Core Type Safety	45
F Polarity Inference and Correction	49
G Type Soundness and Completeness	60
H Equational Soundness and Completeness	72

LIST OF FIGURES

1	Syntax of the Dual Core calculus.	9
2	Operational semantics of the Dual Core calculus.	10
3	Simple type system for the Dual Core calculus.	11
4	Additional axioms of the Dual Core calculus equational theory.	12
5	Syntax of the Coo source programming language.	13
6	Simple type system for the Coo source language: commands and (co)terms.	14
7	Simple type system for the Coo source language: reactions and copatterns.	15
8	Automatic polarity correction for the source Coo language: (co)term inference and checking.	16
9	Automatic polarity correction for the source Coo language: commands, reactions, and (co)patterns.	17
10	Additional axioms for equational reasoning about Coo commands and terms.	19
11	Additional axioms for equational reasoning about Coo reactions.	20
12	Translation of source Coo (co)terms and commands to Dual Core calculus.	21
13	Translation of source Coo reactions to commands of the Dual Core calculus.	22
14	Untyped polarity system for the Dual Core calculus.	41
15	Extending Coo with failure-abstracting else reactions.	42

LIST OF THEOREMS

2.1	Remark	6
3.1	Theorem (Progress)	11
3.2	Theorem (Preservation)	11
3.3	Lemma (Typed Polarity)	12
3.4	Lemma (Value Reconstruction)	12
4.1	Definition (Polarity Elaboration & Well-Polarized)	18
4.2	Lemma (Typed Polarity)	18
4.3	Lemma (Polarity Correction)	18
4.4	Lemma (Polarity Determinism)	18
4.5	Definition (Guardless)	20
4.6	Lemma (Reaction Enumeration)	20
5.1	Lemma (Polarity Soundness & Completeness)	23
5.2	Theorem (Type Soundness)	23
5.3	Theorem (Type Completeness)	23
5.4	Lemma (Desugaring Equality)	23
5.5	Theorem (Equational Soundness)	23
5.6	Theorem (Equational Completeness)	23
A.1	Remark	33
A.2	Theorem (Infinite Binary Pigeonhole Principle)	36

C.1 Definition (Guardless)	42
E.1 Lemma (Typed Substitution)	45
E.2 Lemma (Focused Value Typing)	45
E.3 Lemma (Pattern Extraction)	46
3.1 Theorem (Progress)	46
3.2 Theorem (Preservation)	47
3.3 Lemma (Typed Polarity)	49
4.2 Lemma (Typed Polarity)	49
F.1 Lemma	56
F.2 Lemma	56
4.3 Lemma (Polarity Correction)	57
4.4 Lemma (Polarity Determinism)	59
G.1 Lemma (Weakening)	60
5.2 Theorem (Type Soundness)	60
5.3 Theorem (Type Completeness)	71
5.1 Lemma (Polarity Soundness & Completeness)	72
H.1 Lemma (Reverse Equational Soundness)	72
3.4 Lemma (Value Reconstruction)	72
H.2 Lemma (Backward-Forward Round Trip)	73
H.3 Lemma	75
H.4 Lemma	76
5.5 Theorem (Equational Soundness)	76
4.6 Lemma (Reaction Enumeration)	83
5.4 Lemma (Desugaring Equality)	84
H.5 Lemma (Forward-Backward Round Trip)	85
5.6 Theorem (Equational Completeness)	85

A Extended Examples of Coo Programs

A.1 Recursively Looping Matches

Due to the affordances of nesting different kinds of reactions arbitrarily, we get surprising interactions with the control flow possibilities of the sequent calculus. For example, **match** is another form of statement, similar to pattern-matching expressions in functional languages. This match statement prints 40, because the **with** signals we will match on the “whole” list value:

```
match collecting{range(20,30)}.Collect{fn x => x+x} with
| Nil      => { print "empty" }
| Cons[x,xs] => { pprint x };
```

But we could choose to match in a different way. For example, this is a form of in-place while loop written as a *recursive match* (both **as** and **where** are equivalent and introduce a recursive definition, but **as** reads more naturally here) that prints the even numbers between 0 and 100 (exclusive).

```
match 0 as continue with
| 100 => { }
| n   => { pprint n;
          go n+2 to continue; };
```

What is going on here? The recursive name *continue* acts as a label for context **match** □ **as** ... waiting for a value to check. If that value is 100, then the response is an empty command, and we move to the next statement. Otherwise, print the number *n* and refill that context with *n*+2 via **go** *n*+2 **to** *continue*. A **go** thing **to** place is a more general form of goto statement that specifies a target location (place) in addition to the thing to send, compared to an implicit **go** thing sending thing out of the nearest unlabeled **do** => ... Additionally, we can add a label to a **do**-expression to use as an explicit **go to** target. Here is a short-cutting definition of the product function for multiplying the numbers in a list, ending immediately if a 0 is encountered:

```
fun product xs = do end => {
  val n = do => match (1, xs) as continue with
    | acc, Nil      => go acc
    | acc, Cons[0,xs'] => go 0 to end
    | acc, Cons[x,xs'] => go acc*x, xs' to continue;
  go n to end;
};
```

Note that the return pointer of the function is given an explicit label *end* that we can jump ahead to later. We then start a loop with the initial accumulator 1 and starting list *xs*, and calculate the product *n* with three cases. If the list is empty, set *acc* as the value of *n*. If the list starts with 0, immediately jump to the end with 0 as the answer. Otherwise, continue the loop again with an updated accumulator, and the rest of the list. If the whole loop is finished, return *n* as the answer.

A.2 Handling and Capturing Failure

It’s convenient to be able to directly match on the value of an expressing while binding it in a **val** *P* = ... assignment. However, unless there is only one possible shape of output the expression might evaluate to, that pattern match is at risk of failing. Often, we should try to recover from these failure conditions, instead of stopping the whole program. That’s why Coo adopts Idris’ pattern-bind convention [17, 18], which gives the programmer an easy way to handle these scenarios gracefully, and is also a good match for the underlying control flow semantics.

For example, consider this classic purely functional solution to avoiding divide-by-zero errors by returning an optional result that is *Nothing* when the divisor is 0.

```
fun | safeDiv x 0 = Nothing
    | safeDiv x y = Just[x/y];
```

The downside of `safeDiv` is that chaining numerical operations is cumbersome without something like monadic `do`-notation to focus on the “happy path” and leave the tedious error handling out of the way. The addition of an `else` clause accomplishes a similar kind of purpose, but allows the programmer to add arbitrary code to the handler. The only stipulation is that the `else` handler is a *whole* command that spells out exactly where to go next, since the following line of code is not an option (after all, the next line of code usually depends on variables in the assignments, which never happened due to match failure). The choice of where to go may be as simple as the lexically nearest implicit `do => ...` expression, which is enough to determine the control flow. For example, here is a function which divides two numbers twice in such a way that they operations undo each other, printing out each result to see how far it gets:

```
fun undoDivide x y = do => {
  val Just[a] = safeDiv x y
    else Nothing => go Nothing;
  pprint a;
  val Just[b] = safeDiv x a
    else Nothing => go Nothing;
  pprint b;
  go Just[b];
};
```

In the happy path, the result of `undoDivide x y` will always be `Just[x]`. However, there are two possibilities for failure. First, the caller might pass in a `0` for the second argument, which immediately fails at the first division. Second, the first integer division might underflow to `0`, in which case the second operation fails. We can test out all three scenarios of a full success or two different failures, which prints out different incremental progress along the way:

```
1> undoDivide 20 10
2
10
= Just[10]
2> undoDivide 10 20
0
= Nothing
3> undoDivide 1 0
= Nothing
```

Remark A.1. Of course, we would be remiss to point out that such a simple example has a more direct solution, using the non-functional control flow of the sequent calculus to manipulate multiple continuations. In particular, rather than return an optional result to a single continuation, the division operator can take *two* continuations: the main one expects the integer when division is defined, and the other is invoked on an error.

```
fun go parDiv x y to {err}, {main} if
  | y == 0 = go to err
  |       = go x/y to main;
```

Then, we can use this parallel division operator to simulate the *exact* same results as `undoDivide` from before, but this time avoiding the intermediate allocation and checking of `Just` versus `Nothing` like so:

```
fun undoDivide' x y = do => {
  fun err = do () => go Nothing;
  val a = parDiv x y {err};
  pprint a;
  val b = parDiv x a {err};
  pprint b;
```

```
    go Just[b];
};
```

Notice how we create an `err` continuation in the first line of the function: when invoked with any empty tuple input, it skips everything and **goes** out of the function call with `Nothing` as the answer. As long as `err` is never invoked, then we reach the end of the function, which produces `Just[b]`. Each of the applications as before has the same response:

```
1> undoDivide ' 20 10
2
10
= Just[10]
2> undoDivide ' 10 20
0
= Nothing
3> undoDivide ' 1 0
= Nothing
```

In addition to *handling* failure, *Coo* makes it possible to *capture* it as well. Just like most other mainstream languages with pattern-matching definitions, the possible options are processed top down: if the first option doesn't work, it fails and the next one is tried instead, and so on. But besides just immediately failing and moving on completely, there is usually no way to succeed for now but still talk about what would happen *if hypothetically* we failed instead. We can get a handle on this “what if” scenario in *Coo* by adding an **else** `f` clause in the middle of a reaction. In the reaction **else** `f R`, the (negative **fun**) variable `f` gets bound to a representation of the *rest* of the alternative reactions from this point, which `R` can invoke as an ordinary function as many times as it wants.

For example, a cliché but commonly-known programming problem is “Fizz Buzz”, where one must print each number in some range, but with the following replacement:

- multiples of three are printed as “fizz”,
- multiples of five are printed as “buzz”, and
- multiples of *both* three and five (*i.e.*, 15) are printed as “fizzbuzz”.

While not the most challenging problem, it is that last combined replacement that takes some care: we have to make sure the special “fizzbuzz” case supersedes the other two. Usually, in the most straightforward encoding, this involves an initial check for divisibility by both, and then the other two cases, followed by a default case where we just print the number. There are many ways to express this logic more cleanly, but one new way in *Coo* is with an **else** clause like so:

```
fun fizzbuzz n if
| n%3 == 0 else other = concat "fizz" other
| n%5 == 0           = "buzz"
|                   = "";
```

In the first `n%3 == 0` case, the `other` function will return the string result given as if we failed and jumped to the second `n%5 == 0` case instead. Therefore, multiples of neither three or five fail the first two cases and become the empty string. Multiples of just five fail the first case anyway, and become “buzz”. Then for multiples of three, the first case succeeds and we have two different outcomes:

- If the number *is also* a multiple of 5, then the `n%5 == 0` case in `other` succeeds and returns “buzz”, which combines together to form “fizzbuzz”.
- If the number *is not* a multiple of 5, then the `n%5 == 0` case in `other` fails, so it returns the empty string which combines to just “fizz”.

Using this function to print the numbers from 1 to 40 in a recursive **match** loop (as above), then looks like:

```

match 1 as continue with
  | 40 => { }
  | n => {
    match fizzbuzz n with
      | "" => { pprint n }
      | s  => { print s };
    go n+1 to continue;
  };

```

This barely scratches the surface of what can be done with an **else** reaction, though. When placed at the end of the reaction (right before the right-hand side), it just denotes the next alternative result as if every input remains the same. However, an **else** may be placed *anywhere* within a reaction: everything to the left is held constant, and everything that remains to be matched is are variable parameters to the captured failure function.

For example, consider a function that takes three numbers, a, b, and c, and returns the pair of a/b and a/c. Now, this is fine if neither b nor c are 0. But if either one is 0, we can help the caller by replacing their bad input with some default values (say 10 for b and 2 for c, just to pick some) which won't cause an error. The main logic of returning (a/b, a/c) doesn't change in any case—this is the main purpose of the function. However, it would be nice to add some special cases in front (“pre-handlers” if you will) that fix the input before the main logic runs. Moreover, the two special cases (b is 0 or c is 0), might both overlap, but the way to handle them is independent from one another (analogous to fizz buzz above). One way to insert the two different overlapping pre-handlers to replace some of the parameters to the functions can be achieved with strategically-placed **else** binders like so:

```

fun | divide2 a b else f of 0 => f 10
    | divide2 a else f of 0 c => f 2 c
    | divide2 a b c = (a/b, a/c);

```

With the **else**-powered pre-handlers in place, every possible call to divide2 successfully returns some pair of numbers, using any combination of zero, one, or two defaults as needed.

```

1> divide2 100 5 20
= 20, 5
2> divide2 100 5 0
= 20, 10
3> divide2 100 0 20
= 50, 5
4> divide2 100 0 0
= 50, 10

```

A.3 Yielding Control to Stateful Objects

The family of range objects from section 2 are all stateless: they are pure objects that return the same results on the same inputs no matter the rest of the context or how many times they are called. This may be quite disappointing to object-oriented programmers who want to be able to update some state to build up an answer during the process of iteration. While Coq does not have any built-in notion of mutable state (and there are no plans to add any), it already is possible to simulate stateful objects in a fairly direct style using a certain pattern of classical control-flow.

Recall this definition from remark 2.1 of the subtraction-based yield operator as a function:

```

fun go yield x {cxt} to k = go (x, k) to cxt;

```

We can use **yield** to define another modifier on the range interface which adds a **Compose{f}** method yielding each number in the range to f. If there are no numbers in the range, we finish with f. Done.

Otherwise, we yield the next number (`self.Lower`) using `f.Next` as the extra continuation, which returns back a new continuation `g` with the same interface to continue composing with.

```
fun yielding{obj} where
| self.Compose{f} = do => {
    if self.Done => go f.Done;
    val {g} = yield (self.Lower) {f.Next};
    go self.Step{yielding}.Compose{g};
  }
| self = obj;
```

The key, then, is to define and pass a family of continuations for `Compose` that are indexed by the current state. To make things look a little nicer, we use this helper function

```
fun state x {obj} = obj(x);
```

which simply constructs the instance of `obj` at the given state `x`. With this and `yielding`, we can define a stateful composition to iterate over a range like so:

```
yielding{range(1,10)}.Compose{
  state 0 {fn as
    | self(acc).Done = acc
    | self(acc).Next(x) = do => {
        pprint x;
        val acc = acc+x;
        go {self(acc)}
      }
  }
}
```

The above code adds up all the numbers in the range 1 to 10, printing each number along the way. We start with a Ruby-style chain of method calls `yielding{...}.Compose{...}` parameterized by blocks of code. The main block of code then begins with the line `state 0 {...}` which initializes and constructs the following in-place object to begin with the state 0. Then, in each of the “methods” of that object, we can refer to `self(acc)` to name both the object itself (`self`) as well as the argument of the constructor (`acc`). From this starting point, there are two methods that might be called:

- `Done` returns the `acc` value passed by the constructor,
- `Next(x)` prints the number `x`, adds it to the accumulator `acc`, and then returns the same object but with the updated accumulator state `{self(acc)}`.

A.4 Infinite Pigeonhole Principle

The prescient filter function is the core engine powering the algorithmic solution to the *infinite pigeonhole principle* [6]. This is a variant of the well-known pigeonhole principle: if there are more pigeons than holes, then at least one hole contains more than one pigeon. If we have infinite pigeons, we can go further to say that at least one hole contains an infinite number of pigeons. Spelling it out more formally, we can specify the special binary case of just two holes like so:

THEOREM A.2 (INFINITE BINARY PIGEONHOLE PRINCIPLE). *Given any infinite stream of boolean values (True or False), it must contain either infinite instances of True or infinite instances of False. These instances can be distinguished by their natural number index into the stream (that is, distance from the Head element).*

Of course, mathematically this is true. And of course, algorithmically this is impossible to compute: there is no way to know in advance which of the two cases we are in without inspecting an infinite number of boolean values. And yet we can simulate it to unbounded, infinite precision so that no observer can catch us making it up as we go along.

To derive the infinite pigeonhole principle from the prescient filter function, we need a few standard operations on streams first. Both maps and zips are the stream codata type analogues of the map and zip functions on lists, and we can define the stream that counts all the natural numbers from 0.

```
fun maps {f} where
  | stream{xs}.Head = f(xs.Head)
  | stream{xs}.Tail = {stream(xs.Tail)};

fun | zips {xs} {ys} .Head = (xs.Head, ys.Head)
  | zips {xs} {ys} .Tail = {zips (xs.Tail) (ys.Tail)};

fun | count(n).Head = n
  | count(n).Tail = {count(n+1)};

val nats = {count 0};
```

The algorithm for locating the infinite pigeons is then defined as:

```
fun infinitePigeons bits = do => {
  val decided = prescientFilter {snd} {zips nats bits};
  go {maps {fst} decided};
};
```

To run this code, we need an example boolean stream that we are trying to sort into holes. We'll make an example that starts with some given finite list, appended onto a constant stream that is always the same value after that.

```
fun | always(x).Head = x
  | always(x).Tail = {always(x)};

fun | appends Nil {ys} = ys
  | appends Cons[x,xs] ys .Head = x
  | appends Cons[x,xs] ys .Tail = {appends xs ys};
```

So that test is a stream starting with a list of five particular booleans (that are mostly True) and then is always False after those. The object pigeons is then an infinite list of indexes, which either points to all True values or to all False values in test.

```
fun test =
  appends
  Cons[True, Cons[True, Cons[False, Cons[True, Cons[True, Nil]]]]]
  {always False};

val pigeons = infinitePigeons {test};
```

To investigate the result, we can use this takes function that takes a finite prefix of a stream and converts it into a list:

```
fun | takes 0 {xs} = Nil
  | takes n {xs} = Cons[xs.Head, takes (n-1) (xs.Tail)];
```

Now, consider what happens if we start looking at some prefixes of pigeons. We might look at the first 1, 3, or 4 indexes

```
val pigeons1 = takes 1 pigeons;
val pigeons3 = takes 3 pigeons;
val pigeons4 = takes 4 pigeons;
```

and doing so will produce the following lists:

```

pigeons1 = Cons[0, Nil];
pigeons3 = Cons[0, Cons[1, Cons[3, Nil]]];
pigeons4 = Cons[0, Cons[1, Cons[3, Cons[4, Nil]]]];

```

Huh, we seem to be pointing at `True` values (notably skipping index 2, which points to `False`), even though we know that, at the end, there will only be infinite `False` to come. Why is that? We haven't looked deep enough to hit always `False`. At the beginning of the stream, `True` is much more plentiful. The `prescientFilter` function is essentially a greedy algorithm that tries to meet its demand with the smallest prefix of the stream possible. If you only want to find four of the same boolean values, then these will do.

However, if we instead look a little further ahead, past the point where the stream is always `False`

```

val pigeons1 = takes 1 pigeons;
val pigeons3 = takes 3 pigeons;
val pigeons4 = takes 4 pigeons;
val pigeons5 = takes 5 pigeons;
val pigeons9 = takes 9 pigeons;

```

we instead get entirely different answers throughout, retroactively changing the previous answers so they are consistent with the latter ones:

```

pigeons1 = Cons[2, Nil];
pigeons3 = Cons[2, Cons[5, Cons[6, Nil]]];
pigeons4 = Cons[2, Cons[5, Cons[6, Cons[7, Nil]]]];
pigeons5 = Cons[2, Cons[5, Cons[6, Cons[7, Cons[8, Nil]]]]];
pigeons9 = Cons[2,
  Cons[5,
    Cons[6,
      Cons[7,
        Cons[8,
          Cons[9,
            Cons[10,
              Cons[11,
                Cons[12, Nil]]]]]]]]];

```

Note that the first element is 2 (that lone `False` interspersed among the initial `Trues`) before counting dutifully from 5. This is the consistency criteria that the `prescientFilter` function maintains. Even though it might have been easier to point to `True` for the first four examples, once we need the fifth example, it is easier (and in fact, only possible) to point at `False` instead, so *every* observer must find the same pigeons no matter how many they want. That consistency across all observers is what maintains the illusion that we have somehow computed the infinite in finite time.

B “Is It Really the Sequent Calculus?”

A reader who is already quite familiar with the sequent calculus might wonder if the Dual Core calculus presented here really “counts” as a sequent calculus.³ After all, the Dual Core calculus can look a bit unusual compared to the other computational interpretations of the sequent calculus in the literature. But more to the point, there are many different interpretations as to what the sequent calculus is “really about”—*i.e.*, what is its most salient feature(s) that we can use to classify a logical system as a “sequent calculus” or not—so we should explain the view taken here. What

³Note that the full Coq source language is not meant to be exclusively a sequent calculus. It intentionally incorporates features from natural deduction and functional-imperative languages to make it easier to write practical programs. So for the purposes of our claim that the language represents a high-level sequent calculus programming language, we mean that (1) it has within it an entire sequent calculus (Dual Core) as a sub-language, and (2) the full source language faithfully translates to that sequent calculus sub-language. This is a secondary interpretation of section 5, outside of its immediate use as an implementation strategy.

does it mean that Dual Core is a sequent calculus after we scrub away all the artifacts of programs (variable names, (co)terms, commands) from the inference rules of its type system?

The first criteria—the one which is likely the most universally agreed upon and certainly the easiest to check—is about the properties of its rules. A sequent calculus should include *introduction rules* for its logical connectives (a rule that introduces a new connective *below* the horizontal line of inference) but should *never ever* include any *elimination rules* (a rule that introduces a new connective *above* the line). For this, we can check each inference rule of Dual Core, and confirm that none of them are elimination rules. Some rules count as neither—such as the *Cut*, *Id*, *Act*, *Dat*, or *Cod*—but the rules which do introduce a specific new connective all do so below the line. In this sense, Dual Core easily qualifies as a form of sequent calculus.

A second potential criteria—not as common or universal as the first, but still common enough—is that a sequent calculus ought to feature *sequents*: the environments $\Gamma = A\dots$ to the left (and sometimes to the right, too) of a turnstile $\vdash$ denoting logical entailment. Dual Core satisfies this property, but it is somewhat anachronistic. Originally, natural deduction was defined without the use of sequents (Γ), but they have proven to be so useful that modern presentations of natural deduction almost always include them; to the point that it is hard to find new modern work on natural deduction in that original style [33]. So while true, this is not a particularly surprising nor an impactful distinction in today’s landscape.

Instead, people sometimes refine this criterion to something more strict: a sequent calculus ought to have *explicit structural rules*: the rules of weakening (adding an unused formula to the sequent below the line), contraction (merging duplicates of the same formula above the line into one below the line) and exchange (reordering the formulas in a sequent). Dual Core clearly does not meet this more refined criterion, nor does it aim to. Rather, it follows from the tradition of Curien and Herbelin [19] who have managed to design a version of the sequent calculus that foregoes the need for explicit structural rules and instead embedded them as admissible properties of the system as a whole. This is a conscious design choice to make the presentation simpler (fewer rules, shorter derivations, and more freedom to rewrite judgments into an equivalent form) and to more closely match an efficient algorithmic implementation. However, it would be possible to redesign the rules such that weakening, contraction, and exchange become explicit again like in [46]. For our present purposes so far, this does not add anything at the moment to justify the extra complexity. However, this is a potential avenue for future versions investigating sub-structural (e.g., linear) logics like [46], or low-level optimization and code generation involving register assignment and movement instructions [51, 56].

What remains is then the elephant in the room: the structure of types in Dual Core, and the way they are connected to programs, is just different from the other related sequent calculi in the literature. Somehow, it gets away with fewer distinctions, and fewer duplications, than other presentations of the sequent calculus, which makes it harder to compare directly. For instance, some sequent calculi [19] rely on a hard distinction between *producers* (terms or expressions) versus *consumers* (coterminals or continuations) as part of their logical interpretation, but Dual Core intentionally equivocates between them. Other sequent calculi [46] instead rely on a hard distinction between *positive* and *negative* types—sometimes to the point where connectives are restricted to what kind of types they can apply to—but Dual Core arranges so every type can be interpreted simultaneously in *both* a positive (i.e., **val**) and negative (i.e., **fun**) way, without change, and all combinations of connectives have a well-formed meaning. Yet more sequent calculi [61] make both binary distinctions at the same time.

So we should spell out more precisely how the structure of types in Dual Core is connected to these other sequent calculi. First, let’s consider how the types in polarized sequent calculus similar to [46]—which only has producer terms connected via an orthogonality relationship between

types—relate to Dual Core types. Recall that in Dual Core, a type on its own is not enough to classify a (co)term; it must be paired with the *kind* (**val** or **fun**) as well. In this sense, every positive type can be associated with a **val** mode of use for some type, and every negative type with a **fun** mode of use.

If we focus on the traditional linear logic library of connectives [34], their erasure translation to an unkinded Dual Core type looks something like this (using the numeric indexes 1 and 2 for the projections/injections of the additive product and sum types):

$$\begin{array}{ll}
 \llbracket 1 \rrbracket = () & \llbracket \perp \rrbracket = () \\
 \llbracket A \otimes B \rrbracket = (\llbracket A \rrbracket, \llbracket B \rrbracket) & \llbracket A \wp B \rrbracket = (\llbracket A \rrbracket, \llbracket B \rrbracket) \\
 \llbracket 0 \rrbracket = [] & \llbracket \top \rrbracket = [] \\
 \llbracket A \oplus B \rrbracket = [1 : \llbracket A \rrbracket \mid 2 : \llbracket B \rrbracket] & \llbracket A \& B \rrbracket = [1 : \llbracket A \rrbracket \mid 2 : \llbracket B \rrbracket] \\
 \llbracket \downarrow A \rrbracket = \{ \llbracket A \rrbracket \} & \llbracket \uparrow A \rrbracket = \{ \llbracket A \rrbracket \}
 \end{array}$$

We’ve completely lost the polarity distinction! Instead, it has to be added back in at the level of the sequent, to restore the meaning of constructing a positive value versus negatively destructing it. For this, we need to recover the erased piece of information from a type, its polarity *a.k.a* kind,

$$\begin{array}{ll}
 \text{kind}(A) = \mathbf{val} & (\text{if } A \text{ is positive}) \\
 \text{kind}(A) = \mathbf{fun} & (\text{if } A \text{ is negative})
 \end{array}$$

Then, we can interpret a polarized sequent into an explicitly kinded one of Dual Core like so:

$$\llbracket x_i : A_i.^{\text{.}} \vdash M : B \rrbracket = x_i : \llbracket A_i \rrbracket : \text{kind}(A_i).^{\text{.}} \vdash M : \llbracket B \rrbracket : \text{kind}(B)$$

Even though they don’t make explicit use of polarity in their type systems, we can do the same trick to embed calculi similar to [19] as well, which makes a distinction between producers and consumers on the left and right of the turnstile. Even though Dual Core forces all bound identifiers to the left and all (co)terms to the right, we can freely exchange between them by logical duality via flipping between the **val** or **fun** kinds. So understanding that types like $A \rightarrow B$, $A \& B$, and $A \wp B$ are about function terms (on the right), and that $A \oplus B$ and $A \otimes B$ are about value-producing terms (on the right), we can translate Curien and Herbelin’s sequents through the unstated polarity of their formulas like so:

$$\begin{array}{l}
 \llbracket x_i : B_i.^{\text{.}} \vdash v : A \mid \alpha_j : C_j.^{\text{.}} \rrbracket = x_i : \llbracket B_i \rrbracket : \text{kind}(B_i).^{\text{.}}, \alpha_j : \llbracket C_j \rrbracket : \text{kind}(C_j)^{\circ} \vdash v : \llbracket A \rrbracket : \text{kind}(A) \\
 \llbracket x_i : B_i.^{\text{.}} \mid e : A \vdash \alpha_j : C_j.^{\text{.}} \rrbracket = x_i : \llbracket B_i \rrbracket : \text{kind}(B_i).^{\text{.}}, \alpha_j : \llbracket C_j \rrbracket : \text{kind}(C_j)^{\circ} \vdash e : \llbracket A \rrbracket : \text{kind}(A)^{\circ} \\
 \llbracket c : (x_i : B_i.^{\text{.}} \vdash \alpha_j : C_j.^{\text{.}}) \rrbracket = x_i : \llbracket B_i \rrbracket : \text{kind}(B_i).^{\text{.}}, \alpha_j : \llbracket C_j \rrbracket : \text{kind}(C_j)^{\circ} \vdash c
 \end{array}$$

Thus, we can work through the multiple levels of inversions. A term of (the implicitly) positive type $A \oplus B$ is going to correspond to a (co)term of type $[1 : \llbracket A \rrbracket \mid 2 : \llbracket B \rrbracket] : \mathbf{val}$ since $\text{kind}(A \oplus B) = \mathbf{val}$, which constructs a tagged value. A coterm of $A \oplus B$ is going to correspond to the inverse of that: a (co)term of type $[1 : \llbracket A \rrbracket \mid 2 : \llbracket B \rrbracket] : \mathbf{fun}$ that matches on those tagged values since $\text{kind}(A \oplus B)^{\circ} = \mathbf{val}^{\circ} = \mathbf{fun}$. The dual negative type $A \& B$ is exactly the opposite. A term of (the implicitly) negative type $A \& B$ is going to correspond to a (co)term of type $[1 : \llbracket A \rrbracket \mid 2 : \llbracket B \rrbracket] : \mathbf{fun}$ since $\text{kind}(A \& B) = \mathbf{fun}$, which matches on a tagged value. A coterm of $A \& B$ corresponds to $[1 : \llbracket A \rrbracket \mid 2 : \llbracket B \rrbracket] : \mathbf{val}$ since $\text{kind}(A \& B)^{\circ} = \mathbf{fun}^{\circ} = \mathbf{val}$. The many binary duality switches (left vs right, positive vs negative, term vs coterm) all add up to the same collapsed form in Dual Core.

Going the other way from Dual Core types to another system is a bit more ambiguously open-ended, since we’re now responsible for restoring additional distinctions that were erased. One potential simple interpretation is similar to the view of [56]: just assume there are only ever positive data types with producers and consumers. That way, the meaning of each type is constant, and

$$\begin{array}{c}
\frac{\Gamma \vdash M :: \mathbf{go}(I) \quad \Gamma \vdash N :: \mathbf{to}(I)}{\Gamma \vdash \mathbf{go } M \mathbf{ as } I \mathbf{ to } N} \text{ Cut} \quad \frac{}{\Gamma, x :: K \vdash x :: K} \text{ Id} \quad \frac{\Gamma, x :: K^\circ \vdash J}{\Gamma \vdash \mathbf{do } x \Rightarrow J :: K} \text{ Act} \\
\\
\frac{\Gamma \vdash V ;}{\Gamma \vdash V :: \mathbf{val}} \text{ Dat} \quad \frac{\Delta_i \vdash P_i ; \quad \Gamma, x_i :: \mathbf{fun}, \Delta_i \vdash J_i \quad .i.}{\Gamma \vdash \mathbf{do } \{ \mid \mathbf{as } x_i \mathbf{ to } P_i \Rightarrow J_i.i. \} :: \mathbf{fun}} \text{ Cod} \\
\\
\frac{}{\Gamma, x :: \mathbf{val} \vdash x ;} \text{ IdV}; \quad \frac{\Gamma \vdash M :: \mathbf{fun}}{\Gamma \vdash \{ M \} ;} \{ \} V; \\
\\
\frac{}{\Gamma \vdash () ;} ()V; \quad \frac{\Gamma \vdash V ; \quad \Gamma \vdash W ;}{\Gamma \vdash (V, W) ;} (,)V; \quad \frac{\Gamma \vdash V ;}{\Gamma \vdash X_j[V] ;} [[]]V; \\
\\
\frac{}{x :: \mathbf{val} \vdash x ;} \text{ IdV};; \quad \frac{}{x :: \mathbf{fun} \vdash \{ x \} ;} \{ \} V;; \\
\\
\frac{}{\bullet \vdash () ;} ()V;; \quad \frac{\Delta \vdash P ; \quad \Delta' \vdash P' ;}{\Delta, \Delta' \vdash (P, P') ;} (,)V;; \quad \frac{\Delta \vdash P ;}{\Delta \vdash X_j[P] ;} [[]]V;;
\end{array}$$

Fig. 14. Untyped polarity system for the Dual Core calculus.

the extra kind distinction of **val** versus **fun** only tells you which side of the producer or consumer divide it is on. This way, we could interpret types in the [56] style as their straightforward reading as simple positive data types:

$$\begin{aligned}
\llbracket () \rrbracket &= 1 \\
\llbracket (A, B) \rrbracket &= \llbracket A \rrbracket \otimes \llbracket B \rrbracket \\
\llbracket [X : A...] \rrbracket &= \bigoplus [X : A...] \\
\llbracket \{ A \} \rrbracket &= \downarrow \llbracket A \rrbracket
\end{aligned}$$

Then we only need to assign the producer (**prd**) or consumer (**cns**) role based on the kind:

$$\llbracket \mathbf{val} \rrbracket = \mathbf{prd} \qquad \llbracket \mathbf{fun} \rrbracket = \mathbf{cns}$$

This embedding is particularly nice, since very little changes—just our point of view:

$$\llbracket x_i : A_i : K_i.i. \vdash M : B : K \rrbracket = x_i : \llbracket A_i \rrbracket : \llbracket K_i \rrbracket.i. \vdash M : \llbracket B \rrbracket : \llbracket K \rrbracket$$

Or in other words, the notion of values and functions in Dual Core corresponds closely to the distinction between producers and consumers in AxCut [56]. What's interesting is that the two seem to have converged on a surprisingly similar design for very different reasons: one concerned with low-level machine code generation details that high-level programmers do not want to be bothered with, and the other for designing a language that makes it easy to read and write programs without much consideration for how it will be compiled efficiently.

C Additional Definitions and Extensions

C.1 Dual Core Polarity

The full definition of the inference rules for the Dual Core polarity system is given in fig. 14. Note that the only difference from the type system in fig. 3 is that each judgement has the specific types A, B removed:

- A typing environment $\Gamma = x_i : A_i : K_i.i.$ becomes the polarity environment $\Gamma = x_i :: K_i.i.$
- A polarized command $\Gamma \vdash J$ is only different in the environment Γ

Extended reaction syntax:

$$\text{Reaction} \ni R ::= \dots \mid \mathbf{else} \ x \ R$$

Typing rules for **else** reactions:

$$\frac{\Gamma, x : A : \mathbf{fun} \vdash R : A : \mathbf{fun} \mid \Phi}{\Gamma \vdash \mathbf{else} \ x \ R : A : \mathbf{fun} \mid \Phi} \text{Other} \quad \frac{\Gamma, x : () : \mathbf{fun} \vdash R : \bullet \mid \Phi}{\Gamma \vdash \mathbf{else} \ x \ R : \bullet \mid \Phi} \text{Else}$$

Polarity correction of **else** reactions:

$$\frac{\Gamma, x :: \mathbf{fun} \vdash R :: \Psi' \rightsquigarrow R' \mid \Psi}{\Gamma \vdash \mathbf{else} \ x \ R :: \Psi' \rightsquigarrow \mathbf{else} \ x \ R' \mid \Psi} \text{Else}$$

Translation of source **else** reactions to the Dual Core calculus:

$$\llbracket \mathbf{else} \ x \ R \rrbracket_n = \mathbf{go} \ \underline{\text{other}} \ \mathbf{as} \ \mathbf{cbn} \ \mathbf{to} \ \mathbf{do} \ x \Rightarrow \llbracket R \rrbracket_n$$

Fig. 15. Extending Coo with failure-abstracting **else** reactions.

- A typed (co)term $\Gamma \vdash M : A : K$ becomes $\Gamma \vdash M :: K$
- A typed (co)value $\Gamma \vdash V : A$; becomes $\Gamma \vdash V$;
- A typed (co)pattern $\Delta \vdash P : A$;; becomes $\Gamma \vdash P$;;

C.2 Else Reactions

The **else** reactions introduced in section A are surprisingly easy to formalize and implement, due to the underlying sequent calculus representation and the use of continuations to guide the flow between the chain of alternative reactions. The entire extension of the Coo source language with **else** is shown fig. 15, which updates the syntax, typing rules, polarity correction rules, and translation.

Each of the theorems in the paper also hold with **else** extensions, too, and the following proofs already account for them. For the purpose of classifying *guardless* programs, both **if** reactions and **else** reactions are considered as the same family and excluded together.

Definition C.1 (Guardless). A source reaction, command, or (co)term, is *guardless* if it does not contain any **if** $P \Leftarrow M \ R$ or **else** $x \ R$ reactions.

So any theorem that only covers guardless programs will also be free from **else** $x \ R$ as well.

D Example Derivations

D.1 Derived Shifts

The typing rules of the internally defined return and force functions

$$\mathbf{force} = \mathbf{do} \{ \mid (\{ x \}, k) \Rightarrow \mathbf{go} \ x \ \mathbf{as} \ \mathbf{cbn} \ \mathbf{to} \ k \} \quad \mathbf{return} = \mathbf{do} \{ \mid (x, \{ k \}) \Rightarrow \mathbf{go} \ x \ \mathbf{as} \ \mathbf{cbv} \ \mathbf{to} \ k \}$$

can be derived for any Γ containing $\mathbf{return} : A \rightarrow \{ A \} : \mathbf{fun}$ and $\mathbf{force} : \{ A \} \rightarrow A : \mathbf{fun}$ like so:

$$\frac{\Gamma \vdash \mathbf{return} : A \rightarrow \{ A \} : \mathbf{fun} \mid \Psi}{\Gamma \vdash \mathbf{return} \ M : \{ A \} : \mathbf{fun} \mid \Psi} \text{Id} \quad \frac{\Gamma \vdash M : A : \mathbf{val} \mid \Psi}{\Gamma \vdash \mathbf{return} \ M : \{ A \} : \mathbf{fun} \mid \Psi} \rightarrow FE$$

$$\frac{\Gamma \vdash \mathbf{force} : \{ A \} \rightarrow A : \mathbf{fun} \mid \Psi}{\Gamma \vdash \mathbf{force} \ M : A : \mathbf{fun} \mid \Psi} \text{Id} \quad \frac{\Gamma \vdash M : \{ A \} : \mathbf{val} \mid \Psi}{\Gamma \vdash \mathbf{force} \ M : A : \mathbf{fun} \mid \Psi} \rightarrow FE$$

Moreover, the function definitions themselves are well-typed schematically for any A like so, first with $\Delta = x : A : \text{fun}, k : A : \text{val}$ for force

$$\frac{\frac{x : A : \text{fun} \vdash \{x\} : \{A\} ;; \quad \{\}V;; \quad \frac{}{k : A : \text{val} \vdash k : A ;;} IdV;; \quad \frac{}{\Delta \vdash x : A : \text{fun}} Id \quad \frac{}{\Delta \vdash k : A : \text{val}} Id}{\frac{\Delta \vdash (\{x\}, k) : \{A\} \rightarrow A ;; \quad \frac{}{\Delta \vdash \text{go } x \text{ as cbn to } k} Id}{\bullet \vdash \text{do } \{ \mid (\{x\}, k) \Rightarrow \text{go } x \text{ as cbn to } k \} : \{A\} \rightarrow A : \text{fun}} Cod$$

and second with $\Delta = x : A : \text{val}, k : A : \text{fun}$ for return

$$\frac{\frac{x : A : \text{val} \vdash x : A ;; \quad IdV;; \quad \frac{}{k : A : \text{fun} \vdash \{k\} : \{A\} ;;} \{\}V;; \quad \frac{}{\Delta \vdash x : A : \text{val}} Id \quad \frac{}{\Delta \vdash k : A : \text{fun}} Id}{\frac{\Delta \vdash (x, \{k\}) : A \rightarrow \{A\} ;; \quad \frac{}{\Delta \vdash \text{go } x \text{ as cbv to } k} Id}{\bullet \vdash \text{do } \{ \mid (x, \{k\}) \Rightarrow \text{go } x \text{ as cbv to } k \} : A \rightarrow \{A\} : \text{fun}} Cod$$

The equation

$$\text{go return } x \text{ as cbn to } y = \text{go } x \text{ as cbv to force } y$$

about the internally-defined polarity shifts is derived in the equational theory like so:

$$\begin{aligned} \text{go return } x \text{ as cbn to } y &= \text{go do } \{ \mid \{z\} \Rightarrow \text{go return } x \text{ as cbn to } \{z\} \} \text{ as cbn to } y & (\eta) \\ &= \text{go do } \{ \mid \{z\} \Rightarrow \text{go } x \text{ as cbv to } z \} \text{ as cbn to } y & (\text{return}) \\ \text{go } x \text{ as cbn to force } y &= \text{go (do } y' \Rightarrow \text{go } x \text{ as cbv to force } y') \text{ as cbn to } y & (\beta) \\ &= \text{go do } \{ \mid \{z\} \Rightarrow \text{go } x \text{ as cbv to force } \{z\} \} \text{ as cbn to } y & (\eta\beta) \\ &= \text{go do } \{ \mid \{z\} \Rightarrow \text{go } x \text{ as cbv to } z \} \text{ as cbn to } y & (\text{force}) \end{aligned}$$

And by applying duality, we have the reversed versions of the commands:

$$\begin{aligned} \text{go } \{M\} \text{ as cbv to } N &= \text{go } M \text{ as cbn to } !N \\ \text{go force } x \text{ as cbn to } y &= \text{go } x \text{ as cbv to return } y \end{aligned}$$

D.2 Unfocused Constructions

We have the following derived typing rules for unfocused constructions:

$$\frac{\Gamma \vdash M : A : \text{val} \quad \Gamma \vdash N : A : \text{val}}{\Gamma \vdash \langle M, N \rangle : (A, B) : \text{val}} (,)V \quad \frac{\Gamma \vdash M : A_j : \text{val}}{\Gamma \vdash \langle X_j[M] \rangle : [X_i : A_i.!.] : \text{val}} [[]]V$$

To derive tagged union construction, let the intermediate environments

$$\begin{aligned} \Gamma_1 &= \Gamma, x : [X_i : A_i.!.] : \text{fun} \\ \Gamma_2 &= \Gamma, x : [X_i : A_i.!.] : \text{fun}, y : A_j : \text{val} \end{aligned}$$

$$\frac{\begin{array}{c} \Gamma \vdash M : A_j : \text{val} \\ \vdots \\ \Gamma_1 \vdash M : A_j : \text{val} \end{array} \quad \frac{\frac{\frac{}{\Gamma_2 \vdash y : A_j ;} IdV; \quad \frac{}{\Gamma_2 \vdash X_j[y] : [X_i : A_i.!.] ;} [[]]V; \quad \frac{}{\Gamma_2 \vdash X_j[y] : [X_i : A_i.!.] : \text{val}} Data \quad \frac{}{\Gamma_2 \vdash x : [X_i : A_i.!.] : \text{fun}} Id}{\frac{\Gamma_1, y : A_j : \text{val} \vdash \text{go } X_j[y] \text{ as cbv to } x}{\Gamma_1 \vdash \text{do } y \Rightarrow \text{go } X_j[y] \text{ as cbv to } x : A_j : \text{fun}} Act} Cut$$

$$\frac{\Gamma, x : [X_i : A_i.!.] : \text{fun} \vdash \text{go } M \text{ as cbv} \quad \text{to do } y \Rightarrow \text{go } X_j[y] \text{ as cbv to } x}{\Gamma \vdash \text{do } x \Rightarrow \text{go } M \text{ as cbv} : [X_i : A_i.!.] : \text{val}} Act$$

$$\text{to do } y \Rightarrow \text{go } X_j[y] \text{ as cbv to } x$$

$$\begin{aligned}\Gamma_1 &= \Gamma, x : (A, B) : \mathbf{fun} \\ \Gamma_2 &= \Gamma, x : (A, B) : \mathbf{fun}, y : A : \mathbf{val} \\ \Gamma_3 &= \Gamma, x : (A, B) : \mathbf{fun}, y : A : \mathbf{val}, z : B : \mathbf{val}\end{aligned}$$

$$\begin{aligned}
& \text{do } \{ | \text{ as } x \text{ to } P_i \Rightarrow \text{go } M \text{ as cbn to } P_{i..} \} \\
&= \text{do } \{ | P_i \Rightarrow \text{go do } \{ | \text{ as } x \text{ to } P_i \Rightarrow \text{go } M \text{ as cbn to } P_{i..} \} \text{ as cbn to } P_{i..} \} & (\eta_{\text{react}}) \\
&= \text{do } \{ | P_i \Rightarrow \text{go } P_i \text{ as cbv to do } \{ | \text{ as } x \text{ to } P_i \Rightarrow \text{go } M \text{ as cbn to } P_{i..} \} \text{ ..} \} & (\beta^\circ) \\
&= \text{do } \{ | P_i \Rightarrow \text{go } M[\text{do } \{ | \text{ as } x \text{ to } P_i \Rightarrow \text{go } M \text{ as cbn to } P_{i..} \} / x :: \text{fun}] \text{ as cbn to } P_{i..} \} & (\beta_{DC}) \\
&= M[\text{do } \{ | \text{ as } x \text{ to } P_i \Rightarrow \text{go } M \text{ as cbn to } P_{i..} \} / x] & (\eta_{\text{react}})
\end{aligned}$$

And similarly, unrolling a loop once by internally inlining the recursive reference is:

$$\begin{aligned}
& \text{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \} \\
&= \text{do } \{ \mid P_i \Rightarrow \text{go do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \} \text{ as cbn to } P_{i..} \} & (\eta_{\text{react}}) \\
&= \text{do } \{ \mid P_i \Rightarrow \text{go } P_i \text{ as cbv to do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \} \} & (\beta^\circ) \\
&= \text{do } \{ \mid P_i \Rightarrow J_i[\text{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \} / x_i] \} & (\beta_{DC})
\end{aligned}$$

However, the η_{rec} rule does something different than both these derived equalities. It allows us to pull one step of matching out of the recursive loop. If we inline the recursion on the left-hand side, calling x_i will match on P_i and then again on P'_j . In contrast, the matching P_i s are out outside the recursion on the right-hand side, so calls to x_i will immediately match on P'_j , skipping the first step.

One way to informally sketch why η_{rec} equality is consistent with observational equivalence is to reason coinductively: suppose the two sides are equal, and then try to come up with an observation separating them to contradict the equality. The only possible distinction—that doesn't get stuck on the match or ignore them entirely—is to pass a value V which matches *both* a pattern in $P_{i..}$ and then subsequently one in $P'_{ij..}$ to reach a particular response J_{ij} . That means there's some choice for i and j where $V = P_i[\sigma]$ and $V = P'_{ij}[\sigma']$. Then, the two sides will both reduce down as follows:

$$\begin{aligned}
& \text{go do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow \text{go do } \{ \mid P'_{ij} \Rightarrow J_{ij..} \} \text{ as cbn to } P_{i..} \} \text{ as cbn to } V \\
& \mapsto^* J_{ij} [\sigma, \text{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow \text{go do } \{ \mid P'_{ij} \Rightarrow J_{ij..} \} \text{ as cbn to } P_{i..} \} / x_i, \sigma',] & (\beta_{DC}) \\
& \text{go do } \{ \mid P_i \Rightarrow \text{go do } \{ \mid \text{as } x_i \text{ to } P'_{ij} \Rightarrow J_{ij..} \} \text{ as cbn to } P_{i..} \} \text{ as cbn to } V \\
& \mapsto^* J_{ij} [\sigma, \sigma', \text{do } \{ \mid \text{as } x_i \text{ to } P'_{ij} \Rightarrow J_{ij..} \} / x_i] & (\beta_{DC}) \\
&= J_{ij} [\sigma, \text{do } \{ \mid P_i \Rightarrow \text{go do } \{ \mid \text{as } x_i \text{ to } P'_{ij} \Rightarrow J_{ij..} \} \text{ as cbn to } P_{i..} \} / x_i, \sigma'] & (\eta_{\text{react}})
\end{aligned}$$

which is equal by the coinductive hypothesis, since the two sides could not be separated.

E Dual Core Type Safety

LEMMA E.1 (TYPED SUBSTITUTION). *Given $\Gamma \vdash N : A : \text{fun}$ then*

- (1) *If $\Gamma, x : A : \text{fun} \vdash J$ then $\Gamma \vdash J[N/x]$.*
- (2) *If $\Gamma, x : A : \text{fun} \vdash M : B : K$ then $\Gamma \vdash M[N/x] : B : K$.*
- (3) *If $\Gamma, x : A : \text{fun} \vdash V : B$; then $\Gamma \vdash V[N/x] : B$;*

Likewise, given $\Gamma \vdash W : A$; then

- (1) *If $\Gamma, x : A : \text{val} \vdash J$ then $\Gamma \vdash J[W/x]$.*
- (2) *If $\Gamma, x : A : \text{val} \vdash M : B : K$ then $\Gamma \vdash M[W/x] : B : K$.*
- (3) *If $\Gamma, x : A : \text{val} \vdash V : B$; then $\Gamma \vdash V[W/x] : B$;*

PROOF. By mutual induction on the given type derivation with $x : A$ in the environment. Note that for the second set of substitutions for a value variable $x : A : \text{val}$, then the term being substituted in *must* be value, and not a general term to build a correct derivation. In particular, the IdV ; rule allows for $x : A : \text{val}$ to be used during a focused derivation of a value (denoted by the ; in the typing judgement), and there is no way to build a focused derivation for a non-value term. Moreover, trying to replace the x in (x, y) with a non-value $\text{do } z \Rightarrow J$ would result in $((\text{do } z \Rightarrow J), y)$, which is not even syntactically valid. $\square$

LEMMA E.2 (FOCUSED VALUE TYPING). *If $\Gamma \vdash V : A : \text{val}$ is derivable then so is $\Gamma \vdash V : A$;*

PROOF. Derivations concluding $\Gamma \vdash V : A : \text{val}$ can only end in one of two rules:

- *Dat*, in which case a derivation of $\Gamma \vdash V : A$; is already given as its premise, or
- *Id* concluding $\Gamma', x : A : \mathbf{val} \vdash x : A : \mathbf{val}$, which can be replaced by the *IdV*; concluding $\Gamma', x : A : \mathbf{val} \vdash x : A$;. $\square$

LEMMA E.3 (PATTERN EXTRACTION). *If $\text{Dom}(\sigma) = FV(P)$, $V = P[\sigma]$, and both $\Gamma \vdash V : A$; and $\Delta \vdash P : A$;; are derivable, then the following are all derivable, too:*

- $\Gamma \vdash x[\sigma] : A$; for every $x : B : \mathbf{val} \in \Delta$, and
- $\Gamma \vdash x[\sigma] : A : \mathbf{fun}$ for every $x : B : \mathbf{fun} \in \Delta$.

PROOF. By induction on the given derivation of $\Delta \vdash P : A$;; and inversion on the derivation of $\Gamma \vdash V : A$; using the assumption $V = P[\sigma]$, where the assured derivations of each $x[\sigma]$ are found and extracted from V 's typing derivation at each of the corresponding *IdV*;; and $\{\}V$;; axioms in P 's typing derivation. For the *IdV*;; case, the derivation needed for that value variable is exactly the derivation $\Gamma \vdash V : A$; as given. For the $\{\}V$;; case, the derivation of $\Gamma \vdash V : A$; must end with a matching $\{\}V$; due to $V = P[\sigma]$, and then the derivation needed for that function variable is the premise to $\{\}V$;; $\square$

THEOREM 3.1 (PROGRESS). *If $x : A : \mathbf{fun}, \dots \vdash J$ then either $J \mapsto J'$ for some J' or J is a final command of the form $\mathbf{go } V \text{ as } \mathbf{cbv} \text{ to } x$ for some $x : A : \mathbf{fun} \in \Gamma$.*

PROOF. By inversion on the given typing derivation. Since the reduction rule for the next operational step always applies exactly to the top-level command, we only need inspect the final rule—which is *always* a *Cut*—at the bottom of the derivation,

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{go}(I) \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : A : \mathbf{to}(I) \end{array}}{\Gamma \vdash \mathbf{go } M \text{ as } I \text{ to } N} \text{Cut}$$

and proceed by cases on I and the two sub-derivations $\mathcal{D}$ and/or $\mathcal{E}$.

- (β°) If $I = \mathbf{cbn}$, then the β° rule applies for any M and N with

$$\mathbf{go } M \text{ as } \mathbf{cbn} \text{ to } N \mapsto \mathbf{go } N \text{ as } \mathbf{cbv} \text{ to } M \quad (\beta^\circ)$$

Otherwise, it must be that $I = \mathbf{cbv}$ so the final *Cut* of the command's typing derivation is

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : A : \mathbf{fun} \end{array}}{\Gamma \vdash \mathbf{go } M \text{ as } \mathbf{cbv} \text{ to } N} \text{Cut}$$

in all remaining cases below.

- (β_{fun}) If $M = \mathbf{do } x \Rightarrow J$ because $\mathcal{D}$ ends in

$$\frac{\begin{array}{c} \vdots \mathcal{D}' \\ \Gamma, x : A : \mathbf{fun} \vdash J \end{array}}{\Gamma \vdash \mathbf{do } x \Rightarrow J : A : \mathbf{val}} \text{Act}$$

then the β_{fun} rule applies with

$$\mathbf{go } \mathbf{do } x \Rightarrow J \text{ as } \mathbf{cbv} \text{ to } N \mapsto J[N/x] \quad (\beta_{\text{fun}})$$

Otherwise, $\mathcal{D}$ must end in the *Dat* rule because all others contradict the assumptions:

- *Cod* is impossible, because that would require $\mathcal{D}$'s conclusion be $\Gamma \vdash M : A : \mathbf{fun}$, but that contradicts the conclusion $\Gamma \vdash M : A : \mathbf{val}$ from the assumption that $I = \mathbf{cbv}$.
- *Id* is impossible, because that would require some value variable $y : B : \mathbf{val}$ in the assumed environment $\Gamma = x : A : \mathbf{fun}, \dots$ containing only function variables.

Therefore, if β° and β_{fun} reductions don't apply, it must be that $M = V$ so the final *Cut* of the command's typing derivation is

$$\frac{\frac{\frac{\vdots \mathcal{D}'}{\Gamma \vdash V : A ;}}{\Gamma \vdash V : A : \mathbf{val}} \text{Dat} \quad \frac{\frac{\vdots \mathcal{E}}{\Gamma \vdash N : A : \mathbf{fun}}}{\Gamma \vdash \mathbf{go } V \text{ as cbv to } N} \text{Cut}$$

in all remaining cases below.

- (β_{val}) If $N = \mathbf{do } x \Rightarrow J$ because $\mathcal{E}$ ends in

$$\frac{\frac{\frac{\vdots \mathcal{E}'}{\Gamma, x : A : \mathbf{val} \vdash J}}{\Gamma \vdash \mathbf{do } x \Rightarrow J : A : \mathbf{fun}} \text{Act}$$

then the β_{val} rule applies with

$$\mathbf{go } V \text{ as cbv do } x \Rightarrow J \mapsto J[V/x] \quad (\beta_{val})$$

- (β_{DC}) If $N = \mathbf{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \}$ because $\mathcal{E}$ ends in

$$\frac{\frac{\frac{\vdots \mathcal{E}'_i}{\Delta_i \vdash P_i : A ;} \quad \frac{\vdots \mathcal{E}''_i}{\Gamma, x_i : A : \mathbf{fun}, \Delta_i \vdash J_i \dots i..} \quad \{P_{i..}\} \text{ covers } A}{\Gamma \vdash \mathbf{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \} : A : \mathbf{fun}} \text{Cod}$$

then the β_{DC} rule applies. In particular, note that the assumption $\{P_{i..} \text{ covers } A\}$ implies that there is some $P_j \in \{P_{i..}\}$ and some substitution σ such that $P_j[\sigma] = V$ because $\Gamma \vdash V : A ;$ is derivable for an environment Γ containing only function variables $y : B : \mathbf{fun}$. Therefore, we get the reduction

$$\mathbf{go } V \text{ as cbv to do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_{i..} \} \mapsto J_j[\sigma, N/x_j] \quad (\beta_{DC})$$

- (*final command*) If $N = x$ because $\mathcal{E}$ is

$$\overline{\Gamma \vdash x : A : \mathbf{fun}} \text{Id}$$

then the command $\mathbf{go } V \text{ as cbv to } x$ has the form of a final command.

- The last remaining case is if $\mathcal{E}$ ends in the *Dat* rule, but this is impossible, because it contradicts the conclusion $\Gamma N : A : \mathbf{fun}$ from the assumption that $I = \mathbf{cbv}$. $\square$

THEOREM 3.2 (PRESERVATION). *If $J \mapsto J'$ then $\Gamma \vdash J$ implies $\Gamma \vdash J'$.*

PROOF. By cases on the reduction rule and inversion on the derivation given for the left-hand side, using lemma E.1 to rebuild a new derivation for the right-hand side after substitution.

- (β°) Given the reduction step

$$\mathbf{go } M \text{ as cbn to } N \mapsto \mathbf{go } N \text{ as cbv to } M \quad (\beta^\circ)$$

the typing derivation of the left-hand side must end with

$$\frac{\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M : A : \mathbf{fun}} \quad \frac{\vdots \mathcal{E}}{\Gamma \vdash N : A : \mathbf{val}}}{\Gamma \vdash \mathbf{go } M \text{ as cbn to } N} \text{Cut}$$

- (β_{fun}) Given the reduction step

$$\text{go do } x \Rightarrow J \text{ as cbv to } N \mapsto J[N/x] \quad (\beta_{fun})$$

the typing derivation of the left-hand side must end with

$$\frac{\frac{\frac{\vdots \mathcal{D}}{\Gamma, x : A : \mathbf{fun} \vdash J} \quad \Gamma \vdash \text{do } x \Rightarrow J : A : \mathbf{val}}{\Gamma \vdash \text{go do } x \Rightarrow J \text{ as cbv to } N} \quad \frac{\frac{\vdots \mathcal{E}}{\Gamma \vdash N : A : \mathbf{fun}} \quad \text{Act}}{\Gamma \vdash \text{go do } x \Rightarrow J \text{ as cbv to } N} \quad \text{Cut}$$

so from lemma E.1, we have the substituted derivation

$$\frac{\vdots \mathcal{D}[\mathcal{E}/x : A : \mathbf{fun}]}{\Gamma \vdash J[N/x]}$$

- (β_{val}) Given the reduction step

$$\text{go } V \text{ as cbv to do } x \Rightarrow J \text{ as cbv} \mapsto J[V/x] \quad (\beta_{val})$$

the typing derivation of the left-hand side must end with

$$\frac{\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash V : A : \mathbf{val}} \quad \frac{\frac{\vdots \mathcal{E}}{\Gamma, x : A : \mathbf{val} \vdash J} \quad \Gamma \vdash \text{do } x \Rightarrow J : A : \mathbf{fun}}{\Gamma \vdash \text{go } V \text{ as cbv to do } x \Rightarrow J} \quad \text{Act}}{\Gamma \vdash \text{go } V \text{ as cbv to do } x \Rightarrow J} \quad \text{Cut}$$

We can extract a derivation $\mathcal{D}'$ of $\Gamma \vdash V : A$; from $\mathcal{D}$ using lemma E.2, so from lemma E.1, we have the substituted derivation

$$\frac{\vdots \mathcal{E}[\mathcal{D}/x : A : \mathbf{val}, \mathcal{D}'/x : A ;]}{\Gamma \vdash J[V/x]}$$

- (β_{DC}) Given the step (where $N = \text{do } \{ | \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \}$ and $\text{Dom}(\sigma) = \text{FV}(P_j)$)

$$\text{go } P_j[\sigma] \text{ as cbv to do } \{ | \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \} \mapsto J_j[\sigma, N/x_j] \quad (\beta_{DC})$$

the typing derivation of the left-hand side must end with

$$\frac{\frac{\frac{\vdots \mathcal{D}_j}{\Gamma \vdash P_j[\sigma] : A ;} \quad \Gamma \vdash P_j[\sigma] : A : \mathbf{val}}{\Gamma \vdash \text{go } P_j[\sigma] \text{ as cbv to do } \{ | \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \}} \quad \text{Dat} \quad \frac{\frac{\frac{\vdots \mathcal{E}_i}{\Delta_i \vdash P_i : A ;} \quad \Gamma, x_i : A : \mathbf{fun}, \Delta_i \vdash J_i \quad .i. \quad \{ P_i.i. \} \text{ covers } A}{\Gamma \vdash \text{do } \{ | \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \} : A : \mathbf{fun}} \quad \text{Cod}}{\Gamma \vdash \text{go } P_j[\sigma] \text{ as cbv to do } \{ | \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \}} \quad \text{Cut}$$

From the derivations $\mathcal{D}_j$ of the value $P_j[\sigma]$ and $\mathcal{E}_j$ for the matching pattern P_j , we can extract derivations for each variable in Δ_j via lemma E.3; call each one $\mathcal{D}_y$ for each $y \in \Delta_j$. We then get a substituted derivation from lemma E.1, where $\mathcal{E}$ denotes the entire second premise to the final *Cut* rule ending in *Cod*,

$$\frac{\vdots \mathcal{E}'_j[\mathcal{E}/x_j, \mathcal{D}_y/y, {}^{y \in \Delta_j} \dots]}{\Gamma \vdash J[N/x_j, \sigma]}$$

□

F Polarity Inference and Correction

LEMMA 3.3 (TYPED POLARITY). *All well-typed commands, terms, values, patterns are well-polarized.*

PROOF. First, define the erasure of the typing environments Γ and Δ as follows:

$$\text{erase}(x : A : K \dots) = x :: K \dots \qquad \text{erase}(\bullet) = \bullet$$

Then, we the following statements hold mutually by induction on the given typing derivation:

- (1) If $\Gamma \vdash J$ is derivable then so is $\text{erase}(\Gamma) \vdash J$
- (2) If $\Gamma \vdash M : A : K$ is derivable then so is $\text{erase}(\Gamma) \vdash M :: K$
- (3) If $\Gamma \vdash V : A ;$ is derivable then so is $\text{erase}(\Gamma) \vdash V ;$
- (4) If $\Delta \vdash P : A ; ;$ is derivable then so is $\text{erase}(\Delta) \vdash P ; ;$

The case of each typing inference rule follows directly from the inductive hypotheses by applying the polarity inference rule of the same name. $\square$

LEMMA 4.2 (TYPED POLARITY). *If a command, (co)term, or reaction is well-typed and polarity elaborated, then it is well-polarized.*

PROOF. In addition to erasing full environments Γ , we also need to erase other typing information from Ψ and Φ like so:

$$\text{erase}(A : K) = K \qquad \text{erase}(\Xi \Psi) = \text{erase}(\Psi)$$

Then, we can prove the following statements mutually by induction on the given typing derivation:

- (1) If $\Gamma \vdash J \mid \Phi$ is derivable then so is $\text{erase}(\Gamma) \vdash J \rightsquigarrow J \mid \text{erase}(\Phi)$.
- (2) If $\Gamma \vdash M : A : \mathbf{go}(I) \mid \Psi$ is derivable then so is $\text{erase}(\Gamma) \vdash M :: \mathbf{go} I \rightsquigarrow M \mid \text{erase}(\Psi)$.
Equivalently, if $\Gamma \vdash M : A : \mathbf{to}(I) \mid \Psi$ then $\text{erase}(\Gamma) \vdash M :: \mathbf{to} I \rightsquigarrow M \mid \text{erase}(\Psi)$.
- (3) If $\Gamma \vdash R : A : \Psi \mid \Phi$ is derivable then so is $\text{erase}(\Gamma) \vdash R :: \text{erase}(\Psi) \rightsquigarrow R \mid \text{erase}(\Phi)$.
- (4) If $\Delta ; ; A \vdash Q : B$ is derivable then so is $\text{erase}(\Delta) ; ; \vdash Q$.

The first key point is that, for a polarity-inferable M , its kind $\mathbf{go}(I)$ in the typing derivation of $\Gamma \vdash M : A : \mathbf{go}(I) \mid \Psi$ ensures that we will infer the same kind during polarity inference $\text{erase}(\Gamma) \vdash M \rightsquigarrow M :: \mathbf{go}(I) \mid \text{erase}(\Psi)$. That in turn ensures that the automatic shift insertion $\text{shift}_I(\mathbf{go} M :: \mathbf{go}(I)) = M$ (or equivalently $\text{shift}_I(\mathbf{to} M :: \mathbf{to}(I)) = M$) is the identity function.

The second key point is that at the cut rules GoTo and Go , the assumption that the command is polarity-elaborated rules out the cases where the annotation is ε or A , leaving only the $\text{GoTo}_{\rightleftharpoons}$ and $\text{Go}_{\rightleftharpoons}$ rules which read the **cbv** or **cbn** tag off the annotation instead of changing the annotation.

In detail, the cases for transforming a typing derivation to a polarity derivation are:

- (GoTo) Given $\text{strat}(O) = I$ (from the assumption of polarity elaboration) and

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : K \mid \Psi \end{array} \quad \langle A : K \mid O \mid A' : K' \rangle \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : A' : K' \mid \Psi \end{array}}{\Gamma \vdash \mathbf{go} M \text{ as } O \text{ to } N \mid \Xi \Psi} \text{GoTo}$$

we know $K = \mathbf{go}(I)$ and $K' = \mathbf{to}(I)$ and from the premise $\langle A : K \mid O \mid A' : K' \rangle$ and assumption $\text{strat}(O) = I$, and have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go} I \rightsquigarrow M \mid \text{erase}(\Psi) \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \text{erase}(\Gamma) \vdash N :: \mathbf{to} I \rightsquigarrow N \mid \text{erase}(\Psi) \end{array}}{\text{erase}(\Gamma) \vdash \mathbf{go} M \text{ as } O \text{ to } N \rightsquigarrow \mathbf{go} M \text{ as } O \text{ to } N \mid \text{erase}(\Psi)} \text{GoTo}_{\rightleftharpoons}$$

- (Go) Given $\text{strat}(O) = I$ (from the assumption of polarity elaboration) and

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A' : K' \mid A : K \quad \langle A' : K' \mid O \mid A : K^\circ \rangle \end{array}}{\Gamma \vdash \mathbf{go} M \mathbf{as} O \mid \Xi A : K} \text{Go}$$

we know $K' = \mathbf{go}(I)$ and $K = \mathbf{to}(I)$ and from the premise $\langle A' : K' \mid O \mid A : K \rangle$ and assumption $\text{strat}(O) = I$, and have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go} I \rightsquigarrow M \mid K \end{array}}{\text{erase}(\Gamma) \vdash \mathbf{go} M \mathbf{as} O \rightsquigarrow \mathbf{go} M \mathbf{as} O \mid K} \text{Go} \Leftrightarrow$$

- (End) Given

$$\overline{\Gamma \vdash \varepsilon \mid \mathbf{part} \Psi} \text{End}$$

we have the polarity derivation

$$\overline{\text{erase}(\Gamma) \vdash \varepsilon \rightsquigarrow \varepsilon \mid \text{erase}(\Psi)} \text{End}$$

- (Fun) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma, \text{name}(R) : A : \mathbf{fun} \vdash J \mid \Xi \Psi \end{array}}{\Gamma \vdash \mathbf{fun} R; J \mid \Xi \Psi} \text{Fun}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash R :: \mathbf{fun} \rightsquigarrow R \mid \text{erase}(\Psi) \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \text{erase}(\Gamma), \text{name}(R) :: \mathbf{fun} \vdash J \rightsquigarrow J \mid \text{erase}(\Psi) \end{array}}{\text{erase}(\Gamma) \vdash \mathbf{fun} R; J \rightsquigarrow \mathbf{fun} R; J \mid \text{erase}(\Psi)} \text{Fun}$$

- (Val) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Delta \vdash P : A; \end{array} \quad \begin{array}{c} \vdots \mathcal{F} \\ \Gamma, \Delta \vdash J \mid \Xi \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{G} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi \end{array}}{\Gamma \vdash \mathbf{val} P \Leftarrow M \mathbf{else} R; J \mid \Xi \Psi} \text{Val}$$

we have the polarity derivation (where $\Gamma' = \text{erase}(\Gamma)$, $\Delta' = \text{erase}(\Delta)$, and $\Psi' = \text{erase}(\Psi)$) by erasing $\mathcal{E}$ from lemma 3.3 to get $\mathcal{E}'$

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma' \vdash M :: \mathbf{go} \mathbf{cbv} \rightsquigarrow M \mid \Psi' \end{array} \quad \begin{array}{c} \vdots \mathcal{E}' \\ \Delta' \vdash P; \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{F}) \\ \Gamma', \Delta' \vdash J \rightsquigarrow J \mid \Psi' \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{G}) \\ \Gamma' \vdash R :: \mathbf{fun} \rightsquigarrow R \mid \Psi' \end{array}}{\Gamma' \vdash \mathbf{val} P \Leftarrow M \mathbf{else} R; J \rightsquigarrow \mathbf{val} P \Leftarrow M \mathbf{else} R; J \mid \Psi'} \text{Val}$$

- (Match) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{part} \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{F} \\ \Gamma \vdash J \mid \Xi \Psi \end{array}}{\Gamma \vdash \mathbf{match} M R; J \mid \Xi \Psi} \text{Match}$$

we have the polarity derivation (where $\Gamma' = \text{erase}(\Gamma)$ and $\Psi' = \text{erase}(\Psi)$)

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma' \vdash M :: \mathbf{go\ cbv} \rightsquigarrow M \mid \Psi' \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \Gamma' \vdash R :: \mathbf{fun} \rightsquigarrow R \mid \Psi' \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{F}) \\ \Gamma' \vdash J \rightsquigarrow J \mid \Psi' \end{array}}{\Gamma' \vdash \mathbf{match\ } M\ R; J \rightsquigarrow \mathbf{match\ } M\ R; J \mid \Psi'} \text{ Match}$$

- (*Id*) Given

$$\overline{\Gamma, x : A : \mathbf{go}(I) \vdash x : A : \mathbf{go}(I) \mid \Psi} \text{ Id}$$

we have the polarity derivation

$$\frac{\overline{\text{erase}(\Gamma), x :: \mathbf{go}(I) \vdash x \rightsquigarrow x :: \mathbf{go\ } I \mid \text{erase}(\Psi)}}{\text{erase}(\Gamma), x :: \mathbf{go}(I) \vdash x :: \mathbf{go\ } I \rightsquigarrow x \mid \text{erase}(\Psi)} \begin{array}{l} \text{Id} \\ \text{Shift} \end{array}$$

- (*React*) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole\ } \Psi \end{array}}{\Gamma \vdash \mathbf{do\ } R : A : \mathbf{fun} \mid \Psi} \text{ React}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash R :: \mathbf{fun} \rightsquigarrow R \mid \text{erase}(\Psi) \end{array}}{\text{erase}(\Gamma) \vdash \mathbf{do\ } R \rightsquigarrow \mathbf{do\ } R :: \mathbf{fun} \mid \text{erase}(\Psi)} \text{ React}$$

$$\frac{\text{erase}(\Gamma) \vdash \mathbf{do\ } R \rightsquigarrow \mathbf{do\ } R :: \mathbf{fun} \mid \text{erase}(\Psi)}{\text{erase}(\Gamma) \vdash \mathbf{do\ } R :: \mathbf{go\ cbn} \rightsquigarrow \mathbf{do\ } R \mid \text{erase}(\Psi)} \text{ Shift}$$

- (*Act*) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma, x : A : \mathbf{fun} \vdash J \mid \mathbf{whole\ } \Psi \end{array}}{\Gamma \vdash \mathbf{do\ } x \Rightarrow J : A : \mathbf{val} \mid \Psi} \text{ Act}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma), x :: \mathbf{fun} \vdash J \rightsquigarrow J \mid \text{erase}(\Psi) \end{array}}{\text{erase}(\Gamma) \vdash \mathbf{do\ } x \Rightarrow J :: \mathbf{go\ cbv} \rightsquigarrow \mathbf{do\ } x \Rightarrow J \mid \text{erase}(\Psi)} \text{ Act}$$

- (*Act'*) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash J \mid \mathbf{whole\ } A : \mathbf{val} \end{array}}{\Gamma \vdash \mathbf{do\ } \Rightarrow J : A : \mathbf{val} \mid \Psi} \text{ Act'}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash J \rightsquigarrow J \mid \mathbf{val} \end{array}}{\text{erase}(\Gamma) \vdash \mathbf{do\ } \Rightarrow J :: \mathbf{go\ cbv} \rightsquigarrow \mathbf{do\ } \Rightarrow J \mid \text{erase}(\Psi)} \text{ Act'}$$

- $(\cdot, \cdot)VI$ Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : B : \mathbf{val} \mid \Psi \end{array}}{\Gamma \vdash (M, N) : (A, B) : \mathbf{val} \mid \Psi} (\cdot, \cdot)VI$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbv} \rightsquigarrow M \mid \text{erase}(\Psi) \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \text{erase}(\Gamma) \vdash N :: \mathbf{go\ cbv} \rightsquigarrow N \mid \text{erase}(\Psi) \end{array}}{\frac{\text{erase}(\Gamma) \vdash (M, N) \rightsquigarrow (M, N) :: \mathbf{val} \mid \text{erase}(\Psi)}{\text{erase}(\Gamma) \vdash (M, N) :: \mathbf{go\ cbv} \rightsquigarrow (M, N) \mid \text{erase}(\Psi)} \text{Shift}} (\cdot, \cdot)VI$$

- $(\cdot, \cdot)FE$ Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : (A, B) : \mathbf{fun} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : A : \mathbf{val} \mid \Psi \end{array}}{\Gamma \vdash M N : B : \mathbf{fun} \mid \Psi} (\cdot, \cdot)FE$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbn} \rightsquigarrow M \mid \text{erase}(\Psi) \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \text{erase}(\Gamma) \vdash N :: \mathbf{go\ cbv} \rightsquigarrow N \mid \text{erase}(\Psi) \end{array}}{\frac{\text{erase}(\Gamma) \vdash M N \rightsquigarrow M N :: \mathbf{fun} \mid \text{erase}(\Psi)}{\text{erase}(\Gamma) \vdash M N :: \mathbf{go\ cbn} \rightsquigarrow M N \mid \text{erase}(\Psi)} \text{Shift}} (\cdot, \cdot)FE$$

- $([\cdot])VI$ Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A_j : \mathbf{val} \mid \Psi \end{array}}{\Gamma \vdash X_j[M] : [X_i : A_i.\dot{!}] : \mathbf{val} \mid \Psi} [\cdot]VI$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbv} \rightsquigarrow M \mid \text{erase}(\Psi) \end{array}}{\frac{\text{erase}(\Gamma) \vdash X_j[M] \rightsquigarrow X_j[M] :: \mathbf{val} \mid \text{erase}(\Psi)}{\text{erase}(\Gamma) \vdash X_j[M] :: \mathbf{go\ cbv} \rightsquigarrow X_j[M] \mid \text{erase}(\Psi)} \text{Shift}} [\cdot]VI$$

- $([\cdot])FE$ Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : [X_i : A_i.\dot{!}] : \mathbf{fun} \mid \Psi \end{array}}{\Gamma \vdash M.X_j : A_j : \mathbf{fun} \mid \Psi} [\cdot]FE$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbn} \rightsquigarrow M \mid \text{erase}(\Psi) \end{array}}{\frac{\text{erase}(\Gamma) \vdash M.X_j \rightsquigarrow M.X_j :: \mathbf{fun} \mid \text{erase}(\Psi)}{\text{erase}(\Gamma) \vdash M.X_j :: \mathbf{go\ cbn} \rightsquigarrow M.X_j \mid \text{erase}(\Psi)} \text{Shift}} [\cdot]FE$$

- ($()VI$) Given

$$\overline{\Gamma \vdash () : () : \mathbf{val} \mid \Psi} \quad ()VI$$

we have the polarity derivation

$$\frac{\overline{\text{erase}(\Gamma) \vdash () \rightsquigarrow () :: \mathbf{val} \mid \text{erase}(\Psi)} \quad ()VI}{\text{erase}(\Gamma) \vdash () :: \mathbf{go\ cbv} \rightsquigarrow () \mid \text{erase}(\Psi)} \text{Shift}$$

- ($\{\}VI$) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{fun} \mid \Psi \end{array}}{\Gamma \vdash \{M\} : \{A\} : \mathbf{val} \mid \Psi} \quad \{\}VI$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbn} \rightsquigarrow M \mid \text{erase}(\Psi) \end{array}}{\text{erase}(\Gamma) \vdash \{M\} \rightsquigarrow \{M\} :: \mathbf{val} \mid \text{erase}(\Psi)} \quad \{\}VI$$

$$\frac{\text{erase}(\Gamma) \vdash \{M\} \rightsquigarrow \{M\} :: \mathbf{val} \mid \text{erase}(\Psi)}{\text{erase}(\Gamma) \vdash \{M\} :: \mathbf{go\ cbv} \rightsquigarrow \{M\} \mid \text{erase}(\Psi)} \text{Shift}$$

- ($\{\}FE$) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : \{A\} : \mathbf{fun} \mid \Psi \end{array}}{\Gamma \vdash !M : A : \mathbf{val} \mid \Psi} \quad \{\}FE$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbn} \rightsquigarrow M \mid \text{erase}(\Psi) \end{array}}{\text{erase}(\Gamma) \vdash !M \rightsquigarrow !M :: \mathbf{val} \mid \text{erase}(\Psi)} \quad \{\}FE$$

$$\frac{\text{erase}(\Gamma) \vdash !M \rightsquigarrow !M :: \mathbf{val} \mid \text{erase}(\Psi)}{\text{erase}(\Gamma) \vdash !M :: \mathbf{go\ cbv} \rightsquigarrow !M \mid \text{erase}(\Psi)} \text{Shift}$$

- ($Cede$) Given

$$\overline{\Gamma \vdash \varepsilon : \Psi \mid \Phi} \quad Cede$$

we have the polarity derivation

$$\overline{\text{erase}(\Gamma) \vdash \varepsilon :: \text{erase}(\Psi) \rightsquigarrow \varepsilon \mid \text{erase}(\Phi)} \quad Cede$$

- (Alt) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash R_1 : \Psi \mid \Phi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash R_2 : \Psi \mid \Phi \end{array}}{\Gamma \vdash \{R_1 \mid R_2\} : \Psi \mid \Phi} \quad Alt$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash R_1 :: \text{erase}(\Psi) \rightsquigarrow R_1 \mid \text{erase}(\Phi) \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \text{erase}(\Gamma) \vdash R_2 :: \text{erase}(\Psi) \rightsquigarrow R_2 \mid \text{erase}(\Phi) \end{array}}{\text{erase}(\Gamma) \vdash \{R_1 \mid R_2\} :: \text{erase}(\Psi) \rightsquigarrow \{R_1 \mid R_2\} \mid \text{erase}(\Phi)} \quad Alt$$

- (To) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Delta \vdash P : A \end{array} ; ; \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma, \Delta \vdash R : \bullet \mid \Phi \end{array}}{\Gamma \vdash \text{to } P R : A : \text{fun} \mid \Phi} \text{To}$$

we have the polarity derivation, by erasing $\mathcal{D}$ by lemma 3.3 to get $\mathcal{D}'$

$$\frac{\begin{array}{c} \vdots \mathcal{D}' \\ \text{erase}(\Delta) \vdash P \end{array} ; ; \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \text{erase}(\Gamma, \Delta) \vdash R :: \bullet \rightsquigarrow R \mid \text{erase}(\Phi) \end{array}}{\text{erase}(\Gamma) \vdash \text{to } P R :: \text{fun} \rightsquigarrow \text{to } P R \mid \text{erase}(\Phi)} \text{To}$$

- (As) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Delta ; ; A \vdash Q : B \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma, \Delta \vdash R : B : \text{fun} \mid \Phi \end{array}}{\Gamma \vdash \text{as } Q R : A : \text{fun} \mid \Phi} \text{As}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Delta) ; ; \vdash Q \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \text{erase}(\Gamma, \Delta) \vdash R :: \text{fun} \rightsquigarrow R \mid \text{erase}(\Phi) \end{array}}{\text{erase}(\Gamma) \vdash \text{as } Q R :: \text{fun} \rightsquigarrow \text{as } Q R \mid \text{erase}(\Phi)} \text{As}$$

- (If) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \text{val} \mid \Psi' \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Delta \vdash P : A \end{array} ; ; \quad \begin{array}{c} \vdots \mathcal{F} \\ \Gamma, \Delta \vdash R : \Psi \mid \Xi \Psi' \end{array}}{\Gamma \vdash \text{if } P \Leftarrow M R : \Psi \mid \Xi \Psi'} \text{If}$$

we have the polarity derivation (where $\Gamma' = \text{erase}(\Gamma)$ and $\Delta' = \text{erase}(\Delta)$), by erasing $\mathcal{E}$ by lemma 3.3 to get $\mathcal{E}'$

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma' \vdash M :: \text{go cbv} \rightsquigarrow M \mid \text{erase}(\Psi') \end{array} \quad \begin{array}{c} \vdots \mathcal{E}' \\ \Delta' \vdash P \end{array} ; ; \quad \begin{array}{c} \vdots IH(\mathcal{F}) \\ \Gamma', \Delta \vdash R :: \text{erase}(\Psi) \rightsquigarrow R \mid \text{erase}(\Psi') \end{array}}{\Gamma' \vdash \text{if } P \Leftarrow M R :: \text{erase}(\Psi) \rightsquigarrow \text{if } P \Leftarrow M R \mid \text{erase}(\Psi')} \text{If}$$

- (Other and Else) Given either

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma, x : A : \text{fun} \vdash R : A : \text{fun} \mid \Phi \end{array}}{\Gamma \vdash \text{else } x R : A : \text{fun} \mid \Phi} \text{Other} \quad \text{or} \quad \frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma, x : () : \text{fun} \vdash R : \bullet \mid \Phi \end{array}}{\Gamma \vdash \text{else } x R : \bullet \mid \Phi} \text{Else}$$

we have the same polarity derivation (where $\Psi' = \text{fun}$ in the *Other* case and $\Psi' = \bullet$ in the *Else* case)

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \text{erase}(\Gamma), x :: \text{fun} \vdash R :: \Psi' \rightsquigarrow R \mid \text{erase}(\Phi) \end{array}}{\text{erase}(\Gamma) \vdash \text{else } x R :: \Psi' \rightsquigarrow \text{else } x R \mid \text{erase}(\Phi)} \text{Else}$$

- (Exec) Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash J \mid \Phi \end{array}}{\Gamma \vdash \Rightarrow J : \bullet \mid \Phi} \text{Exec}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots \\ IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash J \rightsquigarrow J \mid \text{erase}(\Phi) \end{array}}{\text{erase}(\Gamma) \vdash \{\Rightarrow J\} :: \bullet \rightsquigarrow \{\Rightarrow J\} \mid \text{erase}(\Phi)} \text{Exec}$$

- (Exec') Given

$$\frac{\begin{array}{c} \vdots \\ \mathcal{D} \\ \Gamma \vdash J \mid \Xi A : \mathbf{fun} \end{array}}{\Gamma \vdash \{\Rightarrow J\} : A : \mathbf{fun} \mid \Xi \Psi} \text{Exec'}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots \\ IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash J \rightsquigarrow J \mid \mathbf{fun} \end{array}}{\text{erase}(\Gamma) \vdash \{\Rightarrow J\} :: \mathbf{fun} \rightsquigarrow \{\Rightarrow J\} \mid \text{erase}(\Psi)} \text{Eval'}$$

- (Eval) Given

$$\frac{\begin{array}{c} \vdots \\ \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{fun} \mid \Psi \end{array}}{\Gamma \vdash \{\Rightarrow M\} : A : \mathbf{fun} \mid \Xi \Psi} \text{Eval}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots \\ IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbn} \rightsquigarrow M \mid \text{erase}(\Psi) \end{array}}{\text{erase}(\Gamma) \vdash \{\Rightarrow M\} :: \mathbf{fun} \rightsquigarrow \{\Rightarrow M\} \mid \text{erase}(\Psi)} \text{Eval}$$

- (Eval') Given

$$\frac{\begin{array}{c} \vdots \\ \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{fun} \mid A : \mathbf{fun} \end{array}}{\Gamma \vdash \{\Rightarrow M\} : \bullet \mid \Xi A : \mathbf{fun}} \text{Eval'}$$

we have the polarity derivation

$$\frac{\begin{array}{c} \vdots \\ IH(\mathcal{D}) \\ \text{erase}(\Gamma) \vdash M :: \mathbf{go\ cbn} \rightsquigarrow M \mid \mathbf{fun} \end{array}}{\text{erase}(\Gamma) \vdash \{\Rightarrow M\} :: \bullet \rightsquigarrow \{\Rightarrow M\} \mid \mathbf{fun}} \text{Eval'}$$

- (IdF;;) Given

$$\frac{}{x : A : \mathbf{fun} ;; A \vdash x : A} \text{IdF;;}$$

we have the polarity derivation

$$\frac{}{x :: \mathbf{fun} ;; \vdash x} \text{IdF;;}$$

- ($\rightarrow F;;$) Given

$$\frac{\begin{array}{c} \vdots \\ \mathcal{D} \\ \Delta ;; A \vdash Q : A' \rightarrow B' \end{array} \quad \begin{array}{c} \vdots \\ \mathcal{E} \\ \Delta' \vdash P : A' ;; \end{array}}{\text{erase}(\Delta), \Delta' ;; A \vdash Q P : B'} \rightarrow F;;$$

we have the polarity derivation, by erasing $\mathcal{E}$ by lemma 3.3 to get $\mathcal{E}'$

$$\frac{\frac{\vdots IH(\mathcal{D})}{\text{erase}(\Delta) ;; \vdash Q} \quad \frac{\vdots \mathcal{E}'}{\text{erase}(\Delta') \vdash P ;;}}{\text{erase}(\Delta, \Delta') ;; \vdash Q P} \rightarrow F ;;$$

- ($[[]F ;;$) Given

$$\frac{\frac{\vdots \mathcal{D}}{\Delta ;; A \vdash Q : [X_i : B_i..]} \quad [[]F ;;}{\Delta ;; A \vdash Q.X_j : B_j} [[]F ;;$$

we have the polarity derivation

$$\frac{\frac{\vdots IH(\mathcal{D})}{\text{erase}(\Delta) ;; \vdash Q} \quad [[]F ;;}{\text{erase}(\Delta) ;; \vdash Q.X_j} [[]F ;;$$

□

LEMMA F.1. $\text{strat}(I : O) = I$

PROOF. By cases on O :

$$\begin{array}{ll} O = \varepsilon & \text{strat}(I : \varepsilon) = \text{strat}(I) = I \\ O = A & \text{strat}(I : (A)) = \text{strat}(I : A) = I \end{array}$$

In all other cases of O , $I : O$ is undefined. □

LEMMA F.2.

If $\Gamma \vdash M_1 \rightsquigarrow M_2 :: K \mid \Psi$ is derivable, so is $\Gamma \vdash \text{shift}_I(D M_1 :: K) \rightsquigarrow \text{shift}_I(D M_2 :: K) :: D(I) \mid \Psi$.

PROOF. Suppose $\mathcal{D}$ is a derivation of $\Gamma \vdash M_1 \rightsquigarrow M_2 :: K \mid \Psi$. The derivation is built by cases on D , I , and K .

- If $D(I) = K$, then $\text{shift}_I(D M_i :: K) = M_i$, so the derivation $\mathcal{D}$ already suffices as-is.
- If $D = \text{go}$, $I = \text{cbv}$, and $K = \text{fun}$, then $\text{shift}_{\text{cbv}}(\text{go } M_i :: \text{fun}) = !M_i$, so the new derivation is (where $\text{shift}_{\text{cbn}}(\text{go } M_2 :: \text{fun}) = M_2$)

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M_1 \rightsquigarrow M_2 :: \text{fun} \mid \Psi} \quad \text{Shift}}{\frac{\Gamma \vdash M_1 :: \text{go cbn} \rightsquigarrow M_2 \mid \Psi}{\Gamma \vdash !M_1 \rightsquigarrow !M_2 :: \text{val} \mid \Psi} \{ \} FE}$$

- If $D = \text{to}$, $I = \text{cbn}$, and $K = \text{fun}$, then $\text{shift}_{\text{cbn}}(\text{to } M_i :: \text{fun}) = \{ M_i \}$, so the new derivation is (where $\text{shift}_{\text{cbn}}(\text{go } M_2 :: \text{fun}) = M_2$)

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M_1 \rightsquigarrow M_2 :: \text{fun} \mid \Psi} \quad \text{Shift}}{\frac{\Gamma \vdash M_1 :: \text{go cbn} \rightsquigarrow M_2 \mid \Psi}{\Gamma \vdash \{ M_1 \} \rightsquigarrow \{ M_2 \} :: \text{val} \mid \Psi} \{ \} FI}$$

- If $D = \text{go}$, $I = \text{cbn}$, and $K = \text{val}$, then $\text{shift}_{\text{cbn}}(\text{go } M_i :: \text{val}) = \text{return } M_i$, so so the new derivation is (where $\text{shift}_{\text{cbv}}(\text{go } M_2 :: \text{val}) = M_2$)

$$\frac{\frac{\vdots \mathcal{E} \quad \frac{\Gamma \vdash M_1 \rightsquigarrow M_2 :: \text{val} \mid \Psi}{\Gamma \vdash M_1 :: \text{go cbv} \rightsquigarrow M_2 \mid \Psi} \text{Shift}}{\Gamma \vdash \text{return } M_1 \rightsquigarrow \text{return } M_2 :: \text{fun} \mid \Psi} \rightarrow FE$$

with the sub-derivation $\mathcal{E}$ checking the polarity of the return function defined as

$$\text{return} = \text{do} \{ (x, \{k\}) \Rightarrow \text{go } x \text{ as cbv to } k \}$$

built like so (where $\Gamma' = \Gamma, x :: \text{val}, k :: \text{fun}$):

$$\frac{\frac{\frac{x :: \text{val} \vdash x :: \text{val} \mid \Psi}{x :: \text{val}, k :: \text{fun} \vdash (x, \{k\}) :: \text{fun} \mid \Psi} \text{IdV};; \frac{\frac{\frac{\Gamma' \vdash x \rightsquigarrow x :: \text{val} \mid \Psi}{\Gamma' \vdash x :: \text{go cbv} \rightsquigarrow x \mid \Psi} \text{Id}}{\Gamma' \vdash \text{go } x \text{ as cbv to } k \rightsquigarrow \text{go } x \text{ as cbv to } k \mid \Psi} \text{Shift}}{\Gamma' \vdash \{ \Rightarrow \text{go } x \text{ as cbv to } k \} :: \bullet \rightsquigarrow \{ \Rightarrow \text{go } x \text{ as cbv to } k \} \mid \Psi} \text{GoTo} \Leftrightarrow}{\frac{\frac{\frac{\Gamma' \vdash x \rightsquigarrow x :: \text{val} \mid \Psi}{\Gamma' \vdash x :: \text{go cbv} \rightsquigarrow x \mid \Psi} \text{Id}}{\Gamma' \vdash \text{go } x \text{ as cbv to } k \rightsquigarrow \text{go } x \text{ as cbv to } k \mid \Psi} \text{Shift}}{\Gamma' \vdash \{ \Rightarrow \text{go } x \text{ as cbv to } k \} :: \bullet \rightsquigarrow \{ \Rightarrow \text{go } x \text{ as cbv to } k \} \mid \Psi} \text{GoTo} \Leftrightarrow}{\frac{\frac{\Gamma \vdash (x, \{k\}) \Rightarrow \text{go } x \text{ as cbv to } k :: \text{fun} \rightsquigarrow \{ (x, \{k\}) \Rightarrow \text{go } x \text{ as cbv to } k \} \mid \Psi}{\Gamma \vdash \text{return} \rightsquigarrow \text{return} :: \text{fun} \mid \Psi} \text{React}}{\Gamma \vdash \text{return} :: \text{go cbn} \rightsquigarrow \text{return} \mid \Psi} \text{Shift}$$

- If $D = \text{to}$, $I = \text{cbv}$, and $K = \text{val}$, then $\text{shift}_{\text{cbv}}(\text{to } M_i :: \text{val}) = \text{force } M_i$, so so the new derivation is (where $\text{shift}_{\text{cbv}}(\text{go } M_2 :: \text{val}) = M_2$)

$$\frac{\frac{\vdots \mathcal{E} \quad \frac{\Gamma \vdash M_1 \rightsquigarrow M_2 :: \text{val} \mid \Psi}{\Gamma \vdash M_1 :: \text{go cbv} \rightsquigarrow M_2 \mid \Psi} \text{Shift}}{\Gamma \vdash \text{return } M_1 \rightsquigarrow \text{force } M_2 :: \text{fun} \mid \Psi} \rightarrow FE$$

with the sub-derivation $\mathcal{E}$ checking the polarity of the force function defined as

$$\text{force} = \text{do} \{ (\{x\}, k) \Rightarrow \text{go } x \text{ as cbn to } k \}$$

built like so (where $\Gamma' = \Gamma, x :: \text{fun}, k :: \text{val}$):

$$\frac{\frac{\frac{x :: \text{fun} \vdash \{x\} :: \text{fun} \mid \Psi}{x :: \text{fun}, k :: \text{val} \vdash (\{x\}, k) :: \text{fun} \mid \Psi} \text{IdV};; \frac{\frac{\frac{\Gamma' \vdash x \rightsquigarrow x :: \text{fun} \mid \Psi}{\Gamma' \vdash x :: \text{go cbn} \rightsquigarrow x \mid \Psi} \text{Id}}{\Gamma' \vdash \text{go } x \text{ as cbn to } k \rightsquigarrow \text{go } x \text{ as cbn to } k \mid \Psi} \text{Shift}}{\Gamma' \vdash \{ \Rightarrow \text{go } x \text{ as cbn to } k \} :: \bullet \rightsquigarrow \{ \Rightarrow \text{go } x \text{ as cbn to } k \} \mid \Psi} \text{GoTo} \Leftrightarrow}{\frac{\frac{\frac{\Gamma' \vdash x \rightsquigarrow x :: \text{fun} \mid \Psi}{\Gamma' \vdash x :: \text{go cbn} \rightsquigarrow x \mid \Psi} \text{Id}}{\Gamma' \vdash \text{go } x \text{ as cbn to } k \rightsquigarrow \text{go } x \text{ as cbn to } k \mid \Psi} \text{Shift}}{\Gamma' \vdash \{ \Rightarrow \text{go } x \text{ as cbn to } k \} :: \bullet \rightsquigarrow \{ \Rightarrow \text{go } x \text{ as cbn to } k \} \mid \Psi} \text{GoTo} \Leftrightarrow}{\frac{\frac{\Gamma \vdash (\{x\}, k) \Rightarrow \text{go } x \text{ as cbn to } k :: \text{fun} \rightsquigarrow \{ (\{x\}, k) \Rightarrow \text{go } x \text{ as cbn to } k \} \mid \Psi}{\Gamma \vdash \text{force} \rightsquigarrow \text{force} :: \text{fun} \mid \Psi} \text{React}}{\Gamma \vdash \text{force} :: \text{go cbn} \rightsquigarrow \text{force} \mid \Psi} \text{Shift}$$

□

LEMMA 4.3 (POLARITY CORRECTION).

- (1) J is well-polarized iff $\Gamma \vdash J' \rightsquigarrow J \mid \Psi$ for some Γ and Ψ .
- (2) M is well-polarized iff $\Gamma \vdash M' :: D \mid \Psi$ for some Γ, D, I , and Ψ .
- (3) R is well-polarized iff $\Gamma \vdash R' :: \Psi \rightsquigarrow R \mid \Psi'$ for some Γ, Ψ , and Ψ' .

PROOF. The assumption that J, M , or R is well-polarized immediately proves that they are on the right-hand side of a correction $\rightsquigarrow$ by setting $J' = J, M' = M$, or $R' = R$, respectively.

The reverse implication holds by mutual induction on the given derivation, specifically:

- (1) If $\Gamma \vdash J' \rightsquigarrow J \mid \Psi$ is derivable then so is $\Gamma \vdash J \rightsquigarrow J \mid \Psi$.
- (2) If $\Gamma \vdash M' :: D \mid I \rightsquigarrow M \mid \Psi$ is derivable then so is $\Gamma \vdash M :: D \mid I \rightsquigarrow M \mid \Psi$.
- (3) If $\Gamma \vdash M' \rightsquigarrow M :: K \mid \Psi$ is derivable then so is $\Gamma \vdash M \rightsquigarrow M :: K \mid \Psi$.
- (4) If $\Gamma \vdash R' :: \Psi \rightsquigarrow R \mid \Psi'$ is derivable then so is $\Gamma \vdash R :: \Psi \rightsquigarrow R \mid \Psi'$.

The new, reflexive, derivation is almost exactly the same as the original one in every step, except

- the cut cases $GoTo_{\Leftarrow}$ and $GoTo_{\Rightarrow}$ become $GoTo_{\Leftrightarrow}$ due to the updated annotation $I : O$ after polarity correction,
- likewise the cut case $Go_{\Leftarrow}$ becomes $Go_{\Leftrightarrow}$, and
- the $shift_I(D \mid M' :: K)$ operation in the *Shift* rule becomes the identity function, due to the fact that $D(I) = K$ is ensured in the right-hand side of the correction.

The cases involving these rules—*GoTo*, *Go*, and *Shift*—where the second derivation can be different (due to added polarity annotations or shifts on the right-hand side of $\rightsquigarrow$) are transformed from the first derivation as follows (using lemma F.1 where $strat(I : O) = I$ is needed):

- ($Go_{\Leftrightarrow}$) Given the first derivation

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M' :: \mathbf{go} \, strat(O) \rightsquigarrow M \mid \mathbf{go}(strat(O)) \end{array}}{\Gamma \vdash \mathbf{go} \, M' \text{ as } O \rightsquigarrow \mathbf{go} \, M \text{ as } O \mid \mathbf{go}(strat(O))} Go_{\Leftrightarrow}$$

we have the second reflexive derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma \vdash M :: \mathbf{go} \, strat(O) \rightsquigarrow M \mid \mathbf{go}(strat(O)) \end{array}}{\Gamma \vdash \mathbf{go} \, M \text{ as } O \rightsquigarrow \mathbf{go} \, M \text{ as } O \mid \mathbf{go}(strat(O))} Go_{\Leftrightarrow}$$

- ($Go_{\Leftarrow}$) Given the first derivation

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M' :: \mathbf{go} \, I \rightsquigarrow M \mid \mathbf{go}(I) \end{array}}{\Gamma \vdash \mathbf{go} \, M' \text{ as } O \rightsquigarrow \mathbf{go} \, M \text{ as } I : O \mid \mathbf{go}(I)} Go_{\Leftarrow}$$

we have the second derivation, where $strat(I : O) = I$

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma \vdash M :: \mathbf{go} \, I \rightsquigarrow M \mid \mathbf{go}(I) \end{array}}{\Gamma \vdash \mathbf{go} \, M \text{ as } I : O \rightsquigarrow \mathbf{go} \, M \text{ as } I : O \mid \mathbf{go}(I)} Go_{\Leftrightarrow}$$

- ($GoTo_{\Leftrightarrow}$) Given the first derivation

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M' :: \mathbf{go} \, strat(O) \rightsquigarrow M \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N' :: \mathbf{to} \, strat(O) \rightsquigarrow N \mid \Psi \end{array}}{\Gamma \vdash \mathbf{go} \, M' \text{ as } O \text{ to } N' \rightsquigarrow \mathbf{go} \, M \text{ as } O \text{ to } N \mid \Psi} GoTo_{\Leftrightarrow}$$

we have the second reflexive derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma \vdash M :: \mathbf{go} \, strat(O) \rightsquigarrow M \mid \Psi \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \Gamma \vdash N :: \mathbf{to} \, strat(O) \rightsquigarrow N \mid \Psi \end{array}}{\Gamma \vdash \mathbf{go} \, M \text{ as } O \text{ to } N \rightsquigarrow \mathbf{go} \, M \text{ as } O \text{ to } N \mid \Psi} GoTo_{\Leftrightarrow}$$

- ($GoTo_{\Leftarrow}$) Given the first derivation

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash N' \rightsquigarrow N :: \mathbf{to}(I) \mid \Psi} \quad \frac{\vdots \mathcal{E}}{\Gamma \vdash M' :: \mathbf{go} I \rightsquigarrow M \mid \Psi}}{\Gamma \vdash \mathbf{go} M' \text{ as } O \text{ to } N' \rightsquigarrow \mathbf{go} M \text{ as } I : O \text{ to } N \mid \Psi} GoTo_{\Leftarrow}$$

we have the second derivation, where $strat(I : O) = I$ and $shift_I(\mathbf{to} N :: \mathbf{to}(I)) = N$,

$$\frac{\frac{\vdots IH(\mathcal{E})}{\Gamma \vdash M :: \mathbf{go} I \rightsquigarrow M \mid \Psi} \quad \frac{\frac{\vdots IH(\mathcal{D})}{\Gamma \vdash N \rightsquigarrow N :: \mathbf{to}(I) \mid \Psi} Shift}{\Gamma \vdash N :: \mathbf{to} I \rightsquigarrow N \mid \Psi} GoTo_{\Leftarrow}}{\Gamma \vdash \mathbf{go} M \text{ as } I : O \text{ to } N \rightsquigarrow \mathbf{go} M \text{ as } I : O \text{ to } N \mid \Psi} GoTo_{\Leftarrow}$$

- ($GoTo_{\Rightarrow}$) Given the first derivation

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash N' \rightsquigarrow \bullet \mid \Psi} \quad \frac{\vdots \mathcal{E}}{\Gamma \vdash M' \rightsquigarrow M :: \mathbf{go}(I) \mid \Psi} \quad \frac{\vdots \mathcal{F}}{\Gamma \vdash N' :: \mathbf{to} I \rightsquigarrow N \mid \Psi}}{\Gamma \vdash \mathbf{go} M' \text{ as } O \text{ to } N' \rightsquigarrow \mathbf{go} M \text{ as } I : O \text{ to } N \mid \Psi} GoTo_{\Rightarrow}$$

we have the second derivation, where $strat(I : O) = I$ and $shift_I(\mathbf{go} M :: \mathbf{go}(I)) = M$,

$$\frac{\frac{\vdots IH(\mathcal{E})}{\Gamma \vdash M \rightsquigarrow M :: \mathbf{go}(I) \mid \Psi} Shift \quad \frac{\vdots IH(\mathcal{F})}{\Gamma \vdash N :: \mathbf{to} I \rightsquigarrow N \mid \Psi} GoTo_{\Leftarrow}}{\Gamma \vdash \mathbf{go} M \text{ as } I : O \text{ to } N \rightsquigarrow \mathbf{go} M \text{ as } I : O \text{ to } N \mid \Psi} GoTo_{\Leftarrow}$$

- ($Shift$) Given the first derivation

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M' \rightsquigarrow M :: K \mid \Psi}}{\Gamma \vdash M' :: D I \rightsquigarrow shift_I(D M :: K) \mid \Psi} Shift$$

we have a second derivation from applying lemma F.2 to the inductive hypothesis $IH(\mathcal{D})$

$$\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma \vdash M \rightsquigarrow M :: K \mid \Psi} \quad \vdots \text{lemma F.2}}{\frac{\Gamma \vdash shift_I(D M :: K) \rightsquigarrow shift_I(D M :: K) :: D(I) \mid \Psi}{\Gamma \vdash shift_I(D M :: K) :: D I \rightsquigarrow shift_I(D M :: K) \mid \Psi} Shift} Shift$$

All other inference rules remain unchanged in the second derivation, which is then just built from the inductive hypotheses of its premises. $\square$

LEMMA 4.4 (POLARITY DETERMINISM).

- (1) If $\Gamma \vdash J \rightsquigarrow J_1 \mid \Psi$ and $\Gamma \vdash J \rightsquigarrow J_2 \mid \Psi$ then $J_1 = J_2$.
- (2) If $\Gamma \vdash R :: \Psi \rightsquigarrow R_1 \mid \Psi'$ and $\Gamma \vdash R :: \Psi \rightsquigarrow R_2 \mid \Psi'$ then $R_1 = R_2$.
- (3) If $\Gamma \vdash M :: D I \rightsquigarrow M_1 \mid \Psi$ and $\Gamma \vdash M :: D I \rightsquigarrow M_2 \mid \Psi$ then $M_1 = M_2$.
- (4) If $\Gamma \vdash M \rightsquigarrow M_1 :: K_1 \mid \Psi$ and $\Gamma \vdash M \rightsquigarrow M_2 :: K_2 \mid \Psi$ then $M_1 = M_2$ and $K_1 = K_2$.

PROOF. By induction on the given derivation. $\square$

PROOF. By induction on the first given typing derivation and inverson on the second, using the fact that when the shared parameters (the environments Γ, Ψ , the code J, R, M , and the tags D and I) are forced to be the same, the only remaining parameters must be equal in any inference rule. This is due to the syntax-directed nature of the polarity correction rules, such that the syntax of the right-hand side must match the syntax of the left-hand side, with the possible addition of updated annotations ($I : O$) or shifting operations ($\text{shift}(I)(D M :: K)$) based on information that is already forced to be equivalent in the two derivations by assumption, or by the inductive hypothesis (in the case of a shift).

Note the following facts about the inference rules that keep them syntax-directed on the left-hand side of $\leadsto$, to avoid non-determinism:

- For every term M , the kind K (or the lack thereof) inferred from $\Gamma \vdash M \leadsto M' :: K \mid \Phi$ or $\Gamma \vdash M \leadsto \bullet \mid \Phi$ is fully determined by the syntax of M and the environments Γ and Φ .
 - In most cases, K *only* depends on the syntax of M itself.
 - When M is a variable x , the kind K is additionally determined by Γ .
 - When M has the form $\text{do } R$, then the kind is always either **fun** when $R \notin \text{Action}$, or there is no inferred kind when $R \in \text{Action}$. Note that this means there are some times when the kind of $\text{do } R$ should be **fun**, but it is not inferred because the syntax of R is not specific enough to make the inference obvious; in those cases, the polarity needs to be checked instead of inferred.
- When checking the polarity of a term M , the *Shift* rule is mutually exclusive with *Act* and *Act'* because *Shift* requires that a direct inference $M \leadsto M' :: K$ gives a specific kind, whereas the other two rules only apply to terms of the form $\text{do } R_{\text{act}}$ whose syntax such that direct inference $M \leadsto \bullet$ does not give a specific kind.
- Given any command $\text{go } M \text{ as } O \text{ to } N$, at most one inference rule among $\text{GoTo}_{\Leftarrow}$, $\text{GoTo}_{\Leftarrow\Leftarrow}$, or $\text{GoTo}_{\Rightarrow}$ are applicable with it as the left-hand side:
 - The $\text{GoTo}_{\Leftarrow}$ rule is mutually exclusive with the other two because it requires that $\text{strat}(O)$ is defined (forcing $O = I$ or $O = I : A$) but the other rules add an evaluation order annotation $I : O$ (forcing $O = \varepsilon$ or $O = A$). There is no possible O such that both $\text{GoTo}_{\Leftarrow}$ and another one of the other two are applicable.
 - $\text{GoTo}_{\Leftarrow\Leftarrow}$ and $\text{GoTo}_{\Rightarrow}$ are mutually exclusive with one another, because $\text{GoTo}_{\Leftarrow\Leftarrow}$ requires that a kind is directly inferrable from some $N \leadsto N' :: K$, whereas $\text{GoTo}_{\Rightarrow}$ requires that the same inference failed as $N \leadsto \bullet$.
- Given any command $\text{go } M \text{ as } O$, at most one rule is applicable, because $\text{Go}_{\Leftarrow}$ and $\text{Go}_{\Leftarrow\Leftarrow}$ are mutually exclusive for the same reason about the annotation O that forces $\text{GoTo}_{\Leftarrow}$ to be exclusive with the others. $\square$

G Type Soundness and Completeness

LEMMA G.1 (WEAKENING). (1) If $\Gamma \vdash J \mid \Phi$ is derivable then so is $\Gamma, x : A : K \vdash J \mid \Phi$.
 (2) If $\Gamma \vdash M \mid \Psi$ is derivable then so is $\Gamma, x : A : K \vdash M \mid \Psi$.
 (3) If $\Gamma \vdash R : \Psi \mid \Phi$ is derivable then so is $\Gamma, x : A : K \vdash R : \Psi \mid \Phi$.

PROOF. By mutual induction on the given derivation. $\square$

THEOREM 5.2 (TYPE SOUNDNESS). Give any polarity elaborated J, M , or R ,

- (1) If $\Gamma \vdash J \mid \Phi$ then $\Gamma, \Gamma_{\text{err}}, \text{Ex}[\![\Phi]\!] \vdash \llbracket J \rrbracket$.
- (2) If $\Gamma \vdash M : A : K \mid \Psi$ then $\Gamma, \Gamma_{\text{err}}, \text{Ex}[\![\Psi]\!] \vdash \llbracket M \rrbracket : A : K$.
- (3) If $\Gamma \vdash R : \Psi \mid \Phi$ then $\Gamma, \Gamma_{\text{err}}, \text{Ex}[\![\Phi]\!], \text{Re}[\![\Psi]\!] \vdash \llbracket R \rrbracket_{|\Psi|}$.

PROOF. By mutual induction on the given typing derivation.

First, the typing rules for (co)terms:

- *Id* Given

$$\frac{}{\Gamma, x : A : K \vdash x : A : K \mid \Psi} Id$$

the translation is $\llbracket x \rrbracket = x$ with the derivation

$$\frac{}{\Gamma, x : A : K, \Gamma_{err}, Ex[\Psi] \vdash x : A : K} Id$$

- *()I* Given

$$\frac{}{\Gamma \vdash () : () : \mathbf{val} \mid \Psi} ()I$$

the translation is $\llbracket () \rrbracket = ()$ with the derivation

$$\frac{}{\Gamma, \Gamma_{err}, Ex[\Psi] \vdash () : () : \mathbf{val}} ()R$$

- *(,)VI* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : B : \mathbf{val} \mid \Psi \end{array}}{\Gamma \vdash (M, N) : (A, B) : \mathbf{val} \mid \Psi} (,)VI$$

the translation is $\llbracket (M, N) \rrbracket = \langle \llbracket M \rrbracket, \llbracket N \rrbracket \rangle$ with this derivation thanks to the derived rule for pair construction:

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket M \rrbracket : A : \mathbf{val} \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket N \rrbracket : B : \mathbf{val} \end{array}}{\Gamma, \Gamma_{err}, Ex[\Psi] \vdash \langle \llbracket M \rrbracket, \llbracket N \rrbracket \rangle : (A, B) : \mathbf{val}} (,)V$$

- *[]V* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A_j : \mathbf{val} \mid \Psi \end{array}}{\Gamma \vdash X_j[M] : [X_i : A_{i..t.}] : \mathbf{val} \mid \Psi} []V$$

the translation is $\llbracket X_j[M] \rrbracket = \langle X_j[\llbracket M \rrbracket] \rangle$ with this derivation thanks to the derived rule for tagged union construction:

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket M \rrbracket : A_j : \mathbf{val} \end{array}}{\Gamma, \Gamma_{err}, Ex[\Psi] \vdash \langle X_j[\llbracket M \rrbracket] \rangle : [X_i : A_{i..t.}] : \mathbf{val}} []V$$

- *{ }VI* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{fun} \mid \Psi \end{array}}{\Gamma \vdash \{ M \} : \{ A \} : \mathbf{val} \mid \Psi} \{ \}VI$$

the translation is $\llbracket \{ M \} \rrbracket = \{ \llbracket M \rrbracket \}$ with the following derivation

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket M \rrbracket : A : \mathbf{fun} \end{array}}{\Gamma, \Gamma_{err}, Ex[\Psi] \vdash \{ \llbracket M \rrbracket \} : \{ A \} : \mathbf{val}} \downarrow R$$

- *Act* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma, x : A : \mathbf{fun} \vdash J \mid \mathbf{whole} \Psi \end{array}}{\Gamma \vdash \mathbf{do} x \Rightarrow J : A : \mathbf{val} \mid \Psi} \text{Act}$$

the translation is $\llbracket \mathbf{do} x \Rightarrow J \rrbracket = \mathbf{do} x \Rightarrow \llbracket J \rrbracket$ with the derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma, \Gamma_{err}, Ex[\llbracket \mathbf{whole} \Psi \rrbracket], x : A : \mathbf{fun} \vdash \llbracket J \rrbracket \end{array}}{\Gamma, \Gamma_{err}, Ex[\llbracket \Psi \rrbracket] \vdash \mathbf{do} x \Rightarrow \llbracket J \rrbracket : A : \mathbf{val}} \text{Act}$$

- *Act'* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash J \mid \mathbf{whole} A : \mathbf{val} \end{array}}{\Gamma \vdash \mathbf{do} \Rightarrow J : A : \mathbf{val} \mid \Psi} \text{Act}$$

the translation is $\llbracket \mathbf{do} \Rightarrow J \rrbracket = \mathbf{do} \underline{exit} \Rightarrow \llbracket J \rrbracket$ and $Ex[\llbracket \mathbf{whole} A : \mathbf{val} \rrbracket] = \underline{exit} : A : \mathbf{fun}$ with the derivation

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma, \Gamma_{err}, \underline{exit} : A : \mathbf{fun} \vdash \llbracket J \rrbracket \end{array}}{\Gamma, \Gamma_{err} \vdash \mathbf{do} \underline{exit} \Rightarrow \llbracket J \rrbracket : A : \mathbf{val}} \text{Act}$$

$$\frac{\vdots \text{Weak}}{\Gamma, \Gamma_{err}, Ex[\llbracket \Psi \rrbracket] \vdash \mathbf{do} \underline{exit} \Rightarrow \llbracket J \rrbracket : A : \mathbf{val}}$$

- *React* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi \end{array}}{\Gamma \vdash \mathbf{do} R : A : \mathbf{fun} \mid \Psi} \text{React}$$

the translation depends on the syntax of R

- If $R = \mathbf{to} x \Rightarrow J$ or $R = \Rightarrow J$, then the derivation is the same as that for *Act* or *Act'* above, except that the kind of A is opposite.
- Otherwise, the translation is

$$\llbracket \mathbf{do} R \rrbracket = \mathbf{do} \{ \mid \text{as } \underline{self} \text{ to } \underline{here} \Rightarrow \mathbf{go} \mathbf{do} \{ \} \text{ as } \mathbf{cbn} \text{ to } \mathbf{do} \underline{other} \Rightarrow \llbracket R \rrbracket_{\mathbf{fun}} \}$$

with the following derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex[\llbracket \Psi \rrbracket], \underline{self} : A : \mathbf{fun}, \underline{here} : A : \mathbf{val}$)

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma', \underline{other} : A : \mathbf{fun} \vdash \llbracket R \rrbracket_{\mathbf{fun}} : A : \mathbf{val} \end{array}}{\Gamma' \vdash \mathbf{do} \underline{other} \Rightarrow \llbracket R \rrbracket_{\mathbf{fun}} : A : \mathbf{val}} \text{Act}$$

$$\frac{\Gamma' \vdash \mathbf{do} \{ \} : A : \mathbf{fun} \quad \text{Cod} \quad \Gamma' \vdash \mathbf{do} \underline{other} \Rightarrow \llbracket R \rrbracket_{\mathbf{fun}} : A : \mathbf{val}}{\Gamma' \vdash \mathbf{go} \mathbf{do} \{ \} \text{ as } \mathbf{cbn} \text{ to } \mathbf{do} \underline{other} \Rightarrow \llbracket R \rrbracket_{\mathbf{fun}} : A : \mathbf{fun}} \text{Cod}$$

$$\frac{\underline{here} : A : \mathbf{val} \vdash \underline{here} : A ; \quad IdV; \quad \Gamma' \vdash \mathbf{go} \mathbf{do} \{ \} \text{ as } \mathbf{cbn} \text{ to } \mathbf{do} \underline{other} \Rightarrow \llbracket R \rrbracket_{\mathbf{fun}} : A : \mathbf{fun}}{\Gamma, \Gamma_{err}, Ex[\llbracket \Psi \rrbracket] \vdash \mathbf{do} \{ \mid \text{as } \underline{self} \text{ to } \underline{here} \Rightarrow \mathbf{go} \mathbf{do} \{ \} \text{ as } \mathbf{cbn} \text{ to } \mathbf{do} \underline{other} \Rightarrow \llbracket R \rrbracket_{\mathbf{fun}} \} : A : \mathbf{fun}} \text{Cut}$$

- $\rightarrow FE$ Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A \rightarrow B : \mathbf{fun} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : A : \mathbf{val} \mid \Psi \end{array}}{\Gamma \vdash M N : B : \mathbf{fun} \mid \Psi} \rightarrow FE$$

the translation is

$$\llbracket M \ N \rrbracket = \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \langle \llbracket N \rrbracket, \underline{\text{here}} \rangle$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex[\Psi], \underline{\text{here}} : B : \text{val}$)

$$\frac{\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma' \vdash \llbracket M \rrbracket : A \rightarrow B : \text{fun}} \quad \frac{\frac{\vdots IH(\mathcal{E})}{\Gamma' \vdash \llbracket N \rrbracket : A : \text{val}} \quad \overline{\Gamma' \vdash \underline{\text{here}} : B : \text{val}} Id}{\Gamma' \vdash \langle \llbracket N \rrbracket, \underline{\text{here}} \rangle : (A, B) : \text{val}} (.)V}{\frac{\Gamma, \Gamma_{err}, Ex[\Psi], \underline{\text{here}} : B : \text{val} \vdash \text{go } \llbracket M \rrbracket \text{ as cbn to } \langle \llbracket N \rrbracket, \underline{\text{here}} \rangle Cut}{\Gamma, \Gamma_{err}, Ex[\Psi] \vdash \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \langle \llbracket N \rrbracket, \underline{\text{here}} \rangle : B : \text{fun} Act} Act$$

Note that $A \rightarrow B = (A, B)$, so that the *Cut* type matches.

- $[[]]$ FE Given

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M : [X_i : A_i.i.] : \text{fun} \mid \Psi}}{\Gamma \vdash M.X_j : A_j : \text{fun} \mid \Psi} [[]]FE$$

the translation is

$$\llbracket M.X \rrbracket = \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } X[\underline{\text{here}}]$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex[\Psi], \underline{\text{here}} : A_j : \text{val}$)

$$\frac{\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma' \vdash \llbracket M \rrbracket : [X_i : A_i.i.] : \text{fun}} \quad \frac{\overline{\Gamma' \vdash \underline{\text{here}} : A_j : \text{val}} Id}{\Gamma' \vdash X_j[\underline{\text{here}}] : [X_i : A_i.i.] : \text{val}} (.)V}{\frac{\Gamma, \Gamma_{err}, Ex[\Psi], \underline{\text{here}} : A_j : \text{val} \vdash \text{go } \llbracket M \rrbracket \text{ as cbn to } X_j[\underline{\text{here}}] Cut}{\Gamma, \Gamma_{err}, Ex[\Psi] \vdash \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } X_j[\underline{\text{here}}] : A_j : \text{fun} Act} Act$$

- $\{ \}$ FE Given

$$\frac{\Gamma \vdash M : \{ A \} : \text{fun} \mid \Psi}{\Gamma \vdash !M : A : \text{val} \mid \Psi} \{ \}FE$$

the translation is

$$\llbracket !M \rrbracket = \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \{ \underline{\text{here}} \}$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex[\Psi], \underline{\text{here}} : A : \text{fun}$)

$$\frac{\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma' \vdash \llbracket M \rrbracket : \{ A \} : \text{fun}} \quad \frac{\overline{\Gamma' \vdash \underline{\text{here}} : A : \text{fun}} Id}{\Gamma' \vdash \{ \underline{\text{here}} \} : \{ A \} : \text{val}} (.)V}{\frac{\Gamma, \Gamma_{err}, Ex[\Psi], \underline{\text{here}} : A : \text{fun} \vdash \text{go } \llbracket M \rrbracket \text{ as cbn to } \{ \underline{\text{here}} \} Cut}{\Gamma, \Gamma_{err}, Ex[\Psi] \vdash \text{do } \underline{\text{here}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \{ \underline{\text{here}} \} : A : \text{val} Act} Act$$

Second, the typing rules for reactions:

- *Cede* Given

$$\overline{\Gamma \vdash \varepsilon : \Psi \mid \Phi} Cede$$

the translation depends on the mode $|\Psi| \in \{0, 1\}$:

(0) In command mode, where $\Psi = \bullet$ the translation is

$$\begin{aligned} Re\llbracket\Psi\rrbracket &= \underline{other} : () : \mathbf{fun} \\ \llbracket\varepsilon\rrbracket_0 &= \mathbf{go} () \mathbf{as\ cbv\ to\ } \underline{other} \end{aligned}$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex\llbracket\Phi\rrbracket$)

$$\frac{\frac{\Gamma', \underline{other} : () : \mathbf{fun} \vdash () : () ;}{\Gamma', \underline{other} : () : \mathbf{fun} \vdash () : () : \mathbf{cbv}} ()V; \quad \frac{\Gamma', \underline{other} : () : \mathbf{fun} \vdash \underline{other} : () : \mathbf{fun}}{\Gamma', \underline{other} : () : \mathbf{fun} \vdash \mathbf{go} () \mathbf{as\ cbv\ to\ } \underline{other}} \text{Id} \quad \text{Cut}$$

(1) In (co)term mode, where $\Psi = A : \mathbf{fun}$, the translation is

$$\begin{aligned} Re\llbracket\Psi\rrbracket &= \underline{other} : A : \mathbf{fun}, \underline{self} : A : \mathbf{fun}, \underline{here} : A : \mathbf{val} \\ \llbracket\varepsilon\rrbracket_1 &= \mathbf{go\ here\ as\ cbv\ to\ } \underline{other} \end{aligned}$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex\llbracket\Phi\rrbracket, \underline{other}:A:\mathbf{fun}, \underline{here}:A:\mathbf{val}, \underline{self}:A:\mathbf{fun}$)

$$\frac{\frac{\Gamma' \vdash \underline{here} : A : \mathbf{val}}{\Gamma' \vdash \mathbf{go\ here\ as\ cbv\ to\ } \underline{other}} \text{Id} \quad \frac{\Gamma' \vdash \underline{other} : A : \mathbf{fun}}{\Gamma' \vdash \mathbf{go\ here\ as\ cbv\ to\ } \underline{other}} \text{Id}}{\Gamma' \vdash \mathbf{go\ here\ as\ cbv\ to\ } \underline{other}} \text{Cut}$$

• *Alt Given*

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash R_1 : \Psi \mid \Phi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash R_2 : \Psi \mid \Phi \end{array}}{\Gamma \vdash \{ R_1 \mid R_2 \} : \Psi \mid \Phi} \text{Alt}$$

the translation is

$$\llbracket R_1 \mid R_2 \rrbracket_n = \mathbf{go} (\mathbf{do\ } \underline{other} \Rightarrow \llbracket R_1 \rrbracket) \mathbf{as\ cbv\ to\ } (\mathbf{do\ } \underline{here} \Rightarrow \llbracket R_2 \rrbracket)$$

whose derivation depends on the mode $n = |\Psi|$:

(0) $\Psi = \bullet$ and $Re\llbracket\Psi\rrbracket = \underline{other} : () : \mathbf{fun}$. So the derivation is (where $\Gamma' = \Gamma, \Gamma_{err}, Ex\llbracket\Phi\rrbracket$)

$$\frac{\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma', Re\llbracket\Psi\rrbracket \vdash \llbracket R_1 \rrbracket} \quad \frac{\Gamma', Re\llbracket\Psi\rrbracket \vdash \llbracket R_1 \rrbracket}{\Gamma' \vdash \mathbf{do\ } \underline{other} \Rightarrow \llbracket R_1 \rrbracket : () : \mathbf{val}} \text{Act} \quad \frac{\frac{\vdots IH(\mathcal{E})}{\Gamma', Re\llbracket\Psi\rrbracket \vdash \llbracket R_2 \rrbracket} \quad \frac{\Gamma', Re\llbracket\Psi\rrbracket \vdash \llbracket R_2 \rrbracket}{\Gamma', Re\llbracket\Psi\rrbracket, \underline{here} : () : \mathbf{val} \vdash \llbracket R_2 \rrbracket} \text{Weak}}{\Gamma', Re\llbracket\Psi\rrbracket \vdash \mathbf{do\ } \underline{other} \Rightarrow \llbracket R_1 \rrbracket : () : \mathbf{val} \quad \Gamma', Re\llbracket\Psi\rrbracket \vdash \mathbf{do\ } \underline{here} \Rightarrow \llbracket R_2 \rrbracket : () : \mathbf{fun}} \text{Act} \quad \text{Cut}$$

$$\Gamma', Re\llbracket\Psi\rrbracket \vdash \mathbf{go} (\mathbf{do\ } \underline{other} \Rightarrow \llbracket R_1 \rrbracket) \mathbf{as\ cbv\ to\ } (\mathbf{do\ } \underline{here} \Rightarrow \llbracket R_2 \rrbracket)$$

Of note we must weaken both branches of the *Cut* to make the environments match. On the **go** side, the other variable is bound internally by a **do**, so it must be added back to the context afterward. On the **to** side, the body of the **do** never refers to the bound variable here, so the proof from the inductive hypothesis must be weakened with an extra unused free variable for **do** to bind.

(1) $\Psi = A : \mathbf{fun}$ and $Re\llbracket\Psi\rrbracket = \underline{other} : A : \mathbf{fun}, \underline{here} : A : \mathbf{val}, \underline{self} : A : \mathbf{fun}$. Let

$$\begin{aligned} \Gamma' &= \Gamma, \Gamma_{err}, Ex\llbracket\Phi\rrbracket, Re\llbracket\Psi\rrbracket \\ \Gamma' - \underline{other} &= \Gamma, \Gamma_{err}, Ex\llbracket\Phi\rrbracket, \underline{here} : A : \mathbf{val}, \underline{self} : A : \mathbf{fun} \\ \Gamma' - \underline{here} &= \Gamma, \Gamma_{err}, Ex\llbracket\Phi\rrbracket, \underline{other} : A : \mathbf{fun}, \underline{self} : A : \mathbf{fun} \end{aligned}$$

in the following derivation

$$\begin{array}{c}
 \begin{array}{c} \vdots \\ IH(\mathcal{D}) \\ \hline \Gamma' \vdash \llbracket R_1 \rrbracket \end{array} \quad \text{Act} \quad \begin{array}{c} \vdots \\ IH(\mathcal{E}) \\ \hline \Gamma' \vdash \llbracket R_2 \rrbracket \end{array} \quad \text{Act} \\
 \begin{array}{c} \vdots \\ \text{Weak} \\ \hline \Gamma' \vdash \text{do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket : A : \text{val} \end{array} \quad \begin{array}{c} \vdots \\ \text{Weak} \\ \hline \Gamma' \vdash \text{do } \underline{here} \Rightarrow \llbracket R_2 \rrbracket : A : \text{fun} \end{array} \\
 \hline \Gamma' \vdash \text{go } (\text{do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket) \text{ as cbv to } (\text{do } \underline{here} \Rightarrow \llbracket R_2 \rrbracket) \quad \text{Cut}
 \end{array}$$

Here, the weakenings on both sides of the *Cut* are the same, because both **do**-expressions bind a free variable from the inductive hypothesis, which must be re-added to the environment during *Cut*.

- To Given

$$\frac{\begin{array}{c} \vdots \\ \mathcal{D} \\ \hline \Delta \vdash P : A ; \end{array} \quad \begin{array}{c} \vdots \\ \mathcal{E} \\ \hline \Gamma, \Delta \vdash R : \bullet \mid \Phi \end{array}}{\Gamma \vdash \text{to } P R : A : \text{fun} \mid \Phi} \text{To}$$

the inductive hypothesis $IH(\mathcal{E})$ of $\Gamma, \Gamma_{err}, Ex\llbracket \Phi \rrbracket, \underline{other} : () : \text{fun} \vdash \llbracket R \rrbracket_0$ gives us a derivation matching the type-annotated translation

$$\begin{aligned}
 \llbracket \text{to } P R \rrbracket_1 = & \text{go do } \{ \mid P[\Delta] : A \Rightarrow \text{go do } (\underline{other} : () : \text{val}) \Rightarrow \llbracket R \rrbracket_0 \\
 & \text{as cbv : } () \\
 & \text{to do } (_ : () : \text{fun}) \Rightarrow \text{go } P[\Delta] \\
 & \text{as cbv : } A \\
 & \text{to } (\underline{other} : A : \text{fun}) \\
 & \mid \underline{here} : A : \text{val} \Rightarrow \text{go } (\underline{here} : A : \text{val}) \\
 & \text{as cbv : } A \\
 & \text{to } (\underline{other} : A : \text{fun}) \} \\
 & \text{as cbn : } A \\
 & \text{to } (\underline{here} : A : \text{val})
 \end{aligned}$$

in the original environment where $\underline{other} : A : \text{fun}$, $\underline{here} : A : \text{val}$, and $\underline{self} : A : \text{fun}$.

Of note, the free variable $\underline{other} : A : \text{fun}$ is wrapped in a functional coterm $\text{do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other}$ of type $() : \text{fun}$ that captures the pattern P from the match: given an empty input tuple $()$, this new coterm will send the previous $P : A : \text{val}$ to the original $\underline{other} : A : \text{fun}$. This new coterm is used to re-bind $\underline{other} : () : \text{fun}$ at a new type for $\llbracket R \rrbracket_0$ who will only ever send an empty tuple to $\underline{other}$ if it fails to respond.

- As Given

$$\frac{\begin{array}{c} \vdots \\ \mathcal{D} \\ \hline \Delta ;; A \vdash Q : B \end{array} \quad \begin{array}{c} \vdots \\ \mathcal{E} \\ \hline \Gamma, \Delta \vdash R : B : \text{fun} \mid \Phi \end{array}}{\Gamma \vdash \text{as } Q R : A : \text{fun} \mid \Phi} \text{As}$$

the inductive hypothesis on $IH(\mathcal{E})$ is a derivation of

$$\Gamma, \Gamma_{err}, Ex\llbracket \Phi \rrbracket, \underline{other} : A \rightarrow B : \text{fun}, \underline{here} : A \rightarrow B : \text{val}, \underline{self} : A \rightarrow B : \text{fun} \vdash \llbracket R \rrbracket_1$$

From here, the translation $\llbracket \text{as } Q R \rrbracket$ and its derivation follows by induction on the derivation $\mathcal{D}$ of the copattern Q 's type

- $IdF;;$ then, $Q = x$, $B = A$ and the entire derivation $\mathcal{D}$ is just

$$\frac{}{x : A ;; A \vdash x : A} IdF;;$$

The translation and its derivation are (where $\Gamma' = \Gamma, \underline{exit} : \Psi^\circ, \llbracket \Xi \rrbracket, \llbracket A : \mathbf{fun} \rrbracket$)

$$\llbracket \mathbf{as } x R \rrbracket_1 = \mathbf{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1$$

$$\frac{\frac{}{\Gamma' \vdash \underline{self} : A : \mathbf{fun}} Id \quad \frac{\vdots IH(\mathcal{E})}{\Gamma' \vdash \mathbf{go } \llbracket R \rrbracket_1} Cut}{\Gamma' \vdash \mathbf{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1}$$

- $\rightarrow F;;$ then $Q = (Q' P')$, $\Delta = \Delta_1, \Delta_2$ in the derivation $\mathcal{D}$:

$$\frac{\frac{\vdots \mathcal{D}_1}{\Delta_1 ;; A \vdash Q' : A' \rightarrow B} \quad \frac{\vdots \mathcal{D}_2}{\Delta_2 \vdash P' : A' ;;}}{\Delta_1, \Delta_2 ;; A \vdash Q' P' : B} \rightarrow F;;$$

If Q' is just a variable x (so that $\mathcal{D}_1$ is just a $IdF;;$ axiom with $\Delta_1 = x : A' \rightarrow B : \mathbf{fun}$) not used in R (so that $\mathcal{E}$ can be weakened to $\Gamma, \Delta_2 \vdash R : B : \mathbf{fun} \mid \Phi$), that gives us a derivation matching the type-annotated translation

$$\begin{aligned} & \llbracket \mathbf{as } (x P) R \rrbracket_1 \\ &= \mathbf{go do } \{ \mid (P[\Delta] : A, \underline{here} : B : \mathbf{val}) : A \rightarrow B \Rightarrow \\ & \quad \mathbf{go do } (\underline{here} : B : \mathbf{val}) \Rightarrow \mathbf{go } (\underline{self} : A \rightarrow B : \mathbf{fun}) \\ & \quad \quad \mathbf{as cbn} : A \rightarrow B \\ & \quad \quad \mathbf{to } \langle P[\Delta] : A, \underline{here} : B : \mathbf{val} \rangle \\ & \quad \mathbf{as cbn} : B \\ & \quad \mathbf{to do } (\underline{self} : B : \mathbf{fun}) \Rightarrow \mathbf{go do } (\underline{other} : B : \mathbf{fun}) \Rightarrow \llbracket R \rrbracket_1 \\ & \quad \quad \mathbf{as cbv} : B \\ & \quad \quad \mathbf{to do } (\underline{here} : B : \mathbf{val}) \Rightarrow \mathbf{go } (\underline{other} : A \rightarrow B : \mathbf{fun}) \\ & \quad \quad \quad \mathbf{as cbn} : A \rightarrow B \\ & \quad \quad \quad \mathbf{to } \langle P[\Delta] : A, \underline{here} : B : \mathbf{val} \rangle \\ & \quad \mid \underline{here} : A \rightarrow B : \mathbf{val} \Rightarrow \mathbf{go } (\underline{here} : A \rightarrow B : \mathbf{val}) \\ & \quad \quad \mathbf{as cbv} : A \rightarrow B \\ & \quad \quad \mathbf{to } (\underline{other} : A \rightarrow B : \mathbf{fun}) \} \\ & \mathbf{as cbn} : A \text{ to } B \\ & \mathbf{to } (\underline{here} : A \rightarrow B : \mathbf{cbv}) \end{aligned}$$

in the original environment where $\underline{other} : A \rightarrow B : \mathbf{fun}$, $\underline{here} : A \rightarrow B : \mathbf{val}$, and $\underline{self} : A \rightarrow B : \mathbf{fun}$. Similar to the previous case for To , we have to rebind $\underline{other}$ as well as now $\underline{self}$ at new types; they were originally functions expecting $A \rightarrow B$, and in the context of the inductive hypothesis for $\llbracket R \rrbracket_1$, must instead be functions expecting only B . The rebinding is done for both of these free variables by forming a new (co)term which takes a new remainder for the call-stack $\underline{here} : B : \mathbf{val}$ and pushes the captured $P[\Delta] A$ onto it. Thus, in both cases, the locally-bound redefinitions for $\underline{other}$ and $\underline{self}$ surrounding $\llbracket R \rrbracket_1$ have type B .

Otherwise, in the general case the translation is

$$\llbracket \mathbf{as } (Q' P') R \rrbracket_1 = \llbracket \mathbf{as } Q' \text{ of } P' R \rrbracket_1$$

where the right-hand side has the new source derivation

$$\frac{\Delta_1 ;; A \vdash Q' : A' \rightarrow B \quad \frac{\frac{\vdots \mathcal{D}_1}{\Delta_1 ;; A \vdash Q' : A' \rightarrow B} \quad \frac{\frac{\vdots \mathcal{D}_2}{\Delta_2 \vdash P' : A'} \quad \frac{\vdots \mathcal{E}}{\Gamma, \Delta_1, \Delta_2 \vdash R : B : \mathbf{fun} \mid \Phi}}{\Gamma, \Delta_1 \vdash \mathbf{of} P' R : A' \rightarrow B : \mathbf{fun} \mid \Phi} \text{Pop}}{\Gamma \vdash \mathbf{as} Q' \mathbf{of} P' R : A : \mathbf{fun} \mid \Phi} \text{As}$$

and the final translated command and derivation comes from the inductive hypothesis on $\mathcal{D}_1$.

- $\llbracket []F \rrbracket$; then $Q = Q'.X_j$ and $B = B_j$ in the derivation $\mathcal{D}$:

$$\frac{\frac{\vdots \mathcal{D}'}{\Delta ;; A \vdash Q' : [X_i : B_i.i.]} \quad \llbracket []F \rrbracket}{\Delta ;; A \vdash Q'.X : B_j}$$

If Q' is just a variable x (so that $\mathcal{D}_1$ is just a IdF ; axiom with $\Delta_1 = x : A' \rightarrow B : \mathbf{fun}$) not used in R (so that $\mathcal{E}$ can be weakened to $\Gamma, \Delta_2 \vdash R : B : \mathbf{fun} \mid \Phi$), that gives us a derivation matching the type-annotated translation

$$\begin{aligned} & \llbracket \mathbf{as} x.X_j R \rrbracket_1 \\ &= \mathbf{go} \mathbf{do} \{ \mid X_j[\underline{here} : A_j : \mathbf{val}] : [X_i : A_i.i.] \Rightarrow \\ & \quad \mathbf{go} \mathbf{do} (\underline{here} : A_j : \mathbf{val}) \Rightarrow \mathbf{go} (\underline{self} : [X_i : A_i.i.] : \mathbf{fun}) \\ & \quad \quad \mathbf{as} \mathbf{cbn} : [X_i : A_i.i.] \\ & \quad \quad \mathbf{to} X_j[\underline{here} : A_j : \mathbf{val}] \\ & \quad \mathbf{as} \mathbf{cbn} : A_j \\ & \quad \mathbf{to} \mathbf{do} (\underline{self} : A_j : \mathbf{fun}) \Rightarrow \mathbf{go} \mathbf{do} (\underline{other} : A_j : \mathbf{fun}) \Rightarrow \llbracket R \rrbracket_1 \\ & \quad \quad \mathbf{as} \mathbf{cbv} : A_j \\ & \quad \quad \mathbf{to} \mathbf{do} (\underline{here} : A_j : \mathbf{val}) \Rightarrow \mathbf{go} (\underline{other} : [X_i : A_i.i.] : \mathbf{fun}) \\ & \quad \quad \quad \mathbf{as} \mathbf{cbn} : [X_i : A_i.i.] \\ & \quad \quad \quad \mathbf{to} X_j[\underline{here} : A_j : \mathbf{val}] \\ & \quad \mid \underline{here} : [X_i : A_i.i.] : \mathbf{val} \Rightarrow \mathbf{go} (\underline{here} : [X_i : A_i.i.] : \mathbf{val}) \\ & \quad \quad \mathbf{as} \mathbf{cbv} : [X_i : A_i.i.] \\ & \quad \quad \mathbf{to} (\underline{other} : [X_i : A_i.i.] : \mathbf{fun}) \} \\ & \mathbf{as} \mathbf{cbn} : [X_i : A_i.i.] \\ & \mathbf{to} (\underline{here} : [X_i : A_i.i.] : \mathbf{val}) \end{aligned}$$

analogously to the previous special case for $Q' = _ P$, except instead the original types of $\underline{other}$, $\underline{here}$, and $\underline{self}$ in the context are $[X_i : A_i.i.]$ instead of $A \rightarrow B$, and locally rebound to A_j in the scope of $\llbracket R \rrbracket_1$.

Otherwise, in the general case the translation is

$$\llbracket \mathbf{as} Q'.X_j R \rrbracket_1 = \llbracket \mathbf{as} Q' \mathbf{of} .X_j R \rrbracket_1$$

where the right-hand side has the new source derivation

$$\frac{\Delta ;; A \vdash Q' : [X_i : B_i.i.] \quad \frac{\frac{\vdots \mathcal{D}'}{\Delta ;; A \vdash Q' : [X_i : B_i.i.]} \quad \frac{\frac{\vdots \mathcal{E}}{\Gamma, \Delta \vdash R : B_j : \mathbf{fun} \mid \Phi}}{\Gamma, \Delta \vdash \mathbf{of} .X_j R : [X_i : B_i.i.] : \mathbf{fun} \mid \Phi} \text{Proj}}{\Gamma \vdash \mathbf{as} Q' \mathbf{of} .X_j R : A : \mathbf{fun} \mid \Phi} \text{As}$$

and the final translated command and derivation comes from the inductive hypothesis on $\mathcal{D}'$.

- *If Given*

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \mid \Psi' \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Delta \vdash P : A ; \end{array} \quad \begin{array}{c} \vdots \mathcal{F} \\ \Gamma, \Delta \vdash R : \Psi \mid \Xi \Psi' \end{array}}{\Gamma \vdash \mathbf{if } P \Leftarrow M R : \Psi \mid \Xi \Psi'} \text{ If}$$

the inductive hypotheses

$$IH(\mathcal{D}) \text{ of } \Gamma, \Gamma_{err}, Ex[\llbracket \Psi' \rrbracket] \vdash \llbracket M \rrbracket : A : \mathbf{val}$$

$$IH(\mathcal{E}) \text{ of } \Gamma, \Delta, \Gamma_{err}, Ex[\llbracket \Xi \Psi' \rrbracket], Re[\llbracket \Psi \rrbracket] \vdash \llbracket R \rrbracket_{|\Psi|}$$

give a derivation matching the type-annotated translation

$$\begin{aligned} \llbracket \mathbf{if } P \Leftarrow M R \rrbracket_n &= \mathbf{go } \llbracket M \rrbracket \\ &\quad \mathbf{as cbv} : A \\ &\quad \mathbf{to do } \{ \mid P[\Delta] : A \Rightarrow \llbracket R \rrbracket_n \\ &\quad \quad \mid _ : A \Rightarrow \llbracket \varepsilon \rrbracket_n \} \end{aligned}$$

where the sub-derivation of $\Gamma, \Gamma_{err}, Ex[\llbracket \Xi \Psi' \rrbracket], Re[\llbracket \Psi \rrbracket] \vdash \llbracket \varepsilon \rrbracket_n$ is as above for the *Cede* case.

- *Other Given*

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma, x : A : K \vdash R : A : \mathbf{fun} \mid \Phi \end{array}}{\Gamma \vdash \mathbf{else } x R : A : \mathbf{fun} \mid \Phi} \text{ Other}$$

the translation is

$$\llbracket \mathbf{else } x R \rrbracket_1 = \mathbf{go } \underline{\text{other}} \mathbf{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1$$

with the typing derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex[\llbracket \Phi \rrbracket], Re[\llbracket A : K \rrbracket]$)

$$\frac{\frac{\Gamma' \vdash \underline{\text{other}} : A : \mathbf{fun}}{\Gamma' \vdash \underline{\text{other}} : A : \mathbf{fun}} \text{ Id} \quad \frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma', x : A : \mathbf{fun} \vdash \llbracket R \rrbracket_1 \end{array}}{\Gamma' \vdash \mathbf{do } x \Rightarrow \llbracket R \rrbracket_1 : A : \mathbf{val}} \text{ Act}}{\Gamma \vdash \mathbf{go } \underline{\text{other}} \mathbf{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1} \text{ Cut}$$

- *Else* follows the same translation and derivation steps as the *Other* case, except that we have $\underline{\text{other}} : () : \mathbf{fun}$ instead of $\underline{\text{other}} : A : \mathbf{fun}$.
- *Exec Given*

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash J \mid \Phi \end{array}}{\Gamma \vdash \Rightarrow J : \bullet \mid \Phi} \text{ Exec}$$

the translation is

$$\llbracket \Rightarrow J \rrbracket_0 = \llbracket J \rrbracket$$

such that the inductive hypothesis $IH(\mathcal{D})$ is already a derivation of $\Gamma, \Gamma_{err}, Ex[\llbracket \Phi \rrbracket] \vdash \llbracket J \rrbracket$.

- *Exec' Given*

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash J \mid \Xi A : \mathbf{fun} \end{array}}{\Gamma \vdash \Rightarrow J : A : \mathbf{fun} \mid \Xi \Psi} \text{ Exec'}$$

the translation is

$$\llbracket \Rightarrow J \rrbracket_1 = \text{go do } \underline{\text{exit}} \Rightarrow \llbracket J \rrbracket \text{ as cbn to } \underline{\text{here}}$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, \llbracket \Xi \rrbracket$)

$$\frac{\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma', \underline{\text{exit}} : A : \text{val} \vdash \llbracket J \rrbracket} \quad \Gamma' \vdash \text{do } \underline{\text{exit}} \Rightarrow \llbracket J \rrbracket : A : \text{fun}}{Act} \quad \vdots Weak}{\Gamma', \underline{\text{exit}} : \Psi^\circ, Re \llbracket A : \text{fun} \rrbracket \vdash \text{do } \underline{\text{exit}} \Rightarrow \llbracket J \rrbracket : A : \text{fun}} \quad \frac{\Gamma', \underline{\text{exit}} : \Psi^\circ, Re \llbracket A : \text{fun} \rrbracket \vdash \underline{\text{here}} : A : \text{val}}{Id} \quad \frac{}{Cut} \quad \Gamma', \underline{\text{exit}} : \Psi^\circ, Re \llbracket A : \text{fun} \rrbracket \vdash \text{go do } \underline{\text{exit}} \Rightarrow \llbracket J \rrbracket \text{ as cbn to } \underline{\text{here}}$$

- *Eval* Given

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M : A : \text{fun} \mid \Psi}}{\Gamma \vdash \Rightarrow M : A : \text{fun} \mid \Xi \Psi} Eval$$

the translation is

$$\llbracket \Rightarrow M \rrbracket_1 = \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{here}}$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, \llbracket \Xi \rrbracket$)

$$\frac{\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma, \Gamma_{err}, \underline{\text{exit}} : \Psi^\circ \vdash \llbracket M \rrbracket : A : \text{fun}} \quad \vdots Weak}{\Gamma', \underline{\text{exit}} : \Psi^\circ, Re \llbracket A : \text{fun} \rrbracket \vdash \llbracket M \rrbracket : A : \text{fun}} \quad \frac{\Gamma', \underline{\text{exit}} : \Psi^\circ, Re \llbracket A : \text{fun} \rrbracket \vdash \underline{\text{here}} : A : \text{val}}{Id}}{\Gamma', \underline{\text{exit}} : \Psi^\circ, Re \llbracket A : \text{fun} \rrbracket \vdash \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{here}}} Cut$$

- *Eval'* Given

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M : A : \text{fun} \mid A : K}}{\Gamma \vdash \Rightarrow M : \bullet \mid \Xi A : \text{fun}} Eval$$

the translation is

$$\llbracket \Rightarrow M \rrbracket_1 = \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{exit}}$$

with the derivation (where $\Gamma' = \Gamma, \Gamma_{err}, \llbracket \Xi \rrbracket$)

$$\frac{\frac{\frac{\vdots IH(\mathcal{D})}{\Gamma, \Gamma_{err}, \underline{\text{exit}} : A : \text{val} \vdash \llbracket M \rrbracket : A : \text{fun}} \quad \vdots Weak}{\Gamma', \underline{\text{exit}} : A : \text{val}, Re \llbracket A : \text{fun} \rrbracket \vdash \llbracket M \rrbracket : A : \text{fun}} \quad \frac{\Gamma', \underline{\text{exit}} : A : \text{val}, Re \llbracket A : \text{fun} \rrbracket \vdash \underline{\text{exit}} : A : \text{val}}{Id}}{\Gamma', \underline{\text{exit}} : A : \text{val}, Re \llbracket A : \text{fun} \rrbracket \vdash \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{exit}}} Cut$$

Third, the typing rules for commands:

- *GoTo*: Given

$$\frac{\frac{\vdots \mathcal{D}}{\Gamma \vdash M : A : K \mid \Psi} \quad \langle A : K \mid O \mid A' : K' \rangle \quad \frac{\vdots \mathcal{E}}{\Gamma \vdash N : A' : K' \mid \Psi}}{\Gamma \vdash \text{go } M \text{ as } O \text{ to } N; J \mid \Xi \Psi} GoTo$$

Because the command is polarity-elaborated, either $O = I$ or $O = I : A$, such that $K = \mathbf{go}(I)$, $K' = \mathbf{to}(I)$, and $A = A'$ from $\langle A : K \mid O \mid A' : K' \rangle$. The translated derivation is

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket M \rrbracket : A : \mathbf{go}(I) \\ \vdots Weak \\ \Gamma, \Gamma_{err}, Ex[\Xi \Psi] \vdash \llbracket M \rrbracket : A : \mathbf{go}(I) \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket N \rrbracket : A : \mathbf{to}(I) \\ \vdots Weak \\ \Gamma, \Gamma_{err}, Ex[\Xi \Psi] \vdash \llbracket N \rrbracket : A : \mathbf{to}(I) \end{array}}{\Gamma, \Gamma_{err}, Ex[\Xi \Psi] \vdash \mathbf{go} \llbracket M \rrbracket \text{ as } I \text{ to } \llbracket N \rrbracket} \text{Cut}$$

- *Go Given*

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A' : K' \mid A : K \quad \langle A' : K' \mid O \mid A : K^\circ \rangle \end{array}}{\Gamma \vdash \mathbf{go} M \text{ as } O; J \mid \Xi A : K} \text{Go}$$

Because the command is polarity-elaborated, either $O = I$ or $O = I : A$, such that $K' = \mathbf{go}(I)$, $K^\circ = \mathbf{to}(I)$, and $A = A'$ from $\langle A : K \mid O \mid A' : K' \rangle$. The translated derivation is (where $\Gamma' = \Gamma, \Gamma_{err}, \underline{exit} : A : \mathbf{to}(I)$)

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma' \vdash \llbracket M \rrbracket : A : \mathbf{go}(I) \\ \vdots Weak \\ \Gamma', Ex[\Xi] \vdash \llbracket M \rrbracket : A : \mathbf{go}(I) \end{array} \quad \frac{}{\Gamma, Ex[\Xi] \vdash \underline{exit} : A : \mathbf{to}(I)} \text{Id}}{\Gamma', Ex[\Xi] \vdash \mathbf{go} \llbracket M \rrbracket \text{ as } I \text{ to } \underline{exit}} \text{Cut}$$

- *End Given*

$$\frac{}{\Gamma \vdash \varepsilon \mid \mathbf{part} \Psi} \text{End}$$

the translated derivation is (where $\Gamma' = \Gamma, \Gamma_{err}, \underline{exit} : \Psi^\circ$)

$$\frac{\frac{}{\Gamma', \underline{next} : () : \mathbf{fun} \vdash () : () : \mathbf{val}} \text{OI} \quad \frac{}{\Gamma', \underline{next} : () : \mathbf{fun} \vdash \underline{next} : () : \mathbf{fun}} \text{Id}}{\Gamma', \underline{next} : () : \mathbf{fun} \vdash \mathbf{go} () \text{ as } \mathbf{cbv} \text{ to } \underline{next}} \text{Cut}$$

- *Fun Given* $\text{name}(R) = x$ and

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma, x : A : \mathbf{fun} \vdash J \mid \Xi \Psi \end{array}}{\Gamma \vdash \mathbf{fun} R; J \mid \Xi \Psi} \text{Fun}$$

the translation is

$$\llbracket \mathbf{fun} R; J \rrbracket = \mathbf{go} \llbracket \mathbf{do} R \rrbracket \text{ as } \mathbf{cbn} \text{ to } \mathbf{do} x \Rightarrow \llbracket J \rrbracket$$

with the typing derivation (where $\Gamma' = \Gamma, \Gamma_{err}, Ex[\Psi]$)

$$\frac{\begin{array}{c} \vdots IH(\mathcal{D}) \\ \Gamma', Re[A : \mathbf{fun}] \vdash \llbracket R \rrbracket \\ \vdots \text{as React above} \\ \Gamma' \vdash \llbracket \mathbf{do} R \rrbracket : A : \mathbf{fun} \\ \vdots Weak \\ \Gamma', Ex[\Xi] \vdash \llbracket \mathbf{do} R \rrbracket \end{array} \quad \begin{array}{c} \vdots IH(\mathcal{E}) \\ \Gamma', Ex[\Xi], x : A : \mathbf{fun} \vdash \llbracket J \rrbracket \\ \Gamma', Ex[\Xi] \vdash \mathbf{do} x \Rightarrow \llbracket J \rrbracket : A : \mathbf{val} \end{array}}{\Gamma', Ex[\Xi] \vdash \mathbf{go} \llbracket \mathbf{do} R \rrbracket \text{ as } \mathbf{cbn} \text{ to } \mathbf{do} x \Rightarrow \llbracket J \rrbracket} \text{Act Cut}$$

- *Val* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Delta \vdash P : A ; \quad \Gamma, \Delta \vdash J \mid \Xi \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{F} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{G} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{whole} \Psi \end{array}}{\Gamma \vdash \mathbf{val} P \Leftarrow M \mathbf{else} R; J \mid \Xi \Psi} \text{Val}$$

the inductive hypotheses

$$IH(\mathcal{D}) \text{ of } \Gamma, \Gamma_{err}, \underline{exit} : \Psi^\circ \vdash \llbracket M \rrbracket : A : \mathbf{val}$$

$$IH(\mathcal{F}) \text{ of } \Gamma, \Delta, \Gamma_{err}, \underline{exit} : \Psi^\circ, Ex[\Xi] \vdash \llbracket J \rrbracket$$

$$IH(\mathcal{G}) \text{ of } \Gamma, \Gamma_{err}, \underline{exit} : \Psi^\circ \vdash \llbracket R \rrbracket_1$$

gives us a derivation matching the type-annotated translation

$$\llbracket \mathbf{val} P \Leftarrow M \mathbf{else} R; J \rrbracket = \mathbf{go} \llbracket M \rrbracket \mathbf{as} \mathbf{cbv} : A \mathbf{to} \llbracket \mathbf{do} \{ P \Rightarrow J \mid R \} \rrbracket$$

where $\Gamma, \Gamma_{err}, \underline{exit} : \Psi^\circ \vdash \llbracket \mathbf{do} \{ P \Rightarrow J \mid R \} \rrbracket : A : \mathbf{fun}$ as in the case for *React* above.

- *Match* Given

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{val} \mid \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash R : A : \mathbf{fun} \mid \mathbf{part} \Psi \end{array} \quad \begin{array}{c} \vdots \mathcal{F} \\ \Gamma \vdash J \mid \Xi \Psi \end{array}}{\Gamma \vdash \mathbf{match} M R; J \mid \Xi \Psi} \text{Match}$$

the inductive hypotheses

$$IH(\mathcal{D}) \text{ of } \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket M \rrbracket : A : \mathbf{val}$$

$$IH(\mathcal{E}) \text{ of } \Gamma, \Gamma_{err}, Ex[\Psi], \underline{next} : () : \mathbf{fun}, Re[A : \mathbf{fun}] \vdash \llbracket R \rrbracket_1$$

$$IH(\mathcal{F}) \text{ of } \Gamma, \Gamma_{err}, Ex[\Psi] \vdash \llbracket J \rrbracket$$

gives us a derivation matching the type-annotated translation

$$\begin{aligned} \llbracket \mathbf{match} M R; J \rrbracket &= \mathbf{go} (\mathbf{do} (\underline{next} : () : \mathbf{fun}) \Rightarrow \mathbf{go} \llbracket M \rrbracket \mathbf{as} \mathbf{cbv} : A \mathbf{to} \llbracket \mathbf{do} R \rrbracket) \\ &\quad \mathbf{as} \mathbf{cbv} : () \\ &\quad \mathbf{to} \mathbf{do} \{ | () \Rightarrow \llbracket J \rrbracket \} \end{aligned} \quad \square$$

THEOREM 5.3 (TYPE COMPLETENESS). (1) If $\Gamma \vdash J$ then $\Gamma \vdash \llbracket J \rrbracket^{-1} \mid \mathbf{whole} \bullet$.

(2) If $\Gamma \vdash M : A : K$ then $\Gamma \vdash \llbracket M \rrbracket^{-1} : A : K \mid \bullet$.

(3) If $\Gamma \vdash V : A ;$ then $\Gamma \vdash \llbracket V \rrbracket^{-1} : A : \mathbf{val} \mid \bullet$.

PROOF. By induction on the given typing derivation. Most inference rules of Dual Core are either exactly the same in the source language (such as the rules for patterns), are similar but less restrictive (such as $\llbracket \cdot \rrbracket V$; to $\llbracket \cdot \rrbracket VI$) or skipped entirely (such as *Dat*). The two most interesting cases are for the *Cut* and *Cod* rules.

Cut translates back to *GoTo*, whose only difference is an extra premise $\langle A : K \mid O \mid A' : K' \rangle$ for handling additional possibilities for *O* besides just **cbv** or **cbn**. Since Dual Core always forces this annotation to be *I*, the reverse translation always uses this case for checking the consistency of the **go to**. Thus, the typing derivation

$$\frac{\begin{array}{c} \vdots \mathcal{D} \\ \Gamma \vdash M : A : \mathbf{go}(I) \end{array} \quad \begin{array}{c} \vdots \mathcal{E} \\ \Gamma \vdash N : A : \mathbf{to}(I) \end{array}}{\Gamma \vdash \mathbf{go} M \mathbf{as} I \mathbf{to} N} \text{Cut}$$

translates back to

$$\frac{\begin{array}{c} \vdots \\ IH(\mathcal{D}) \end{array} \quad \begin{array}{c} \vdots \\ IH(\mathcal{E}) \end{array}}{\Gamma \vdash \llbracket M \rrbracket^{-1} : A : \mathbf{go}(I) \mid \bullet \quad \langle A : \mathbf{go}(I) \mid I \mid A : \mathbf{to}(I) \rangle \quad \Gamma \vdash \llbracket N \rrbracket^{-1} : A : \mathbf{to}(I) \mid \bullet} \text{GoTo} \\ \Gamma \vdash \mathbf{go} \llbracket M \rrbracket^{-1} \text{ as } I \text{ to } \llbracket N \rrbracket^{-1} \mid \mathbf{whole} \bullet$$

The *Cod* rule

$$\frac{\begin{array}{c} \vdots \\ \mathcal{D}_1 \end{array} \quad \begin{array}{c} \vdots \\ \mathcal{E}_1 \end{array} \quad \begin{array}{c} \vdots \\ \mathcal{D}_n \end{array} \quad \begin{array}{c} \vdots \\ \mathcal{E}_n \end{array}}{\Delta_1 \vdash P_1 : A_1 ;; \quad \Gamma_1, x_1 : A : \mathbf{fun}, \Delta_1 \vdash J_1 \quad \dots \quad \Delta_n \vdash P_n : A_n ;; \quad \Gamma_n, x_n : A : \mathbf{fun}, \Delta_n \vdash J_n} \text{Cod} \\ \Gamma \vdash \mathbf{do} \{ \mid \text{as } x_1 \text{ to } P_1 \Rightarrow J_1 \dots \mid \text{as } x_n \text{ to } P_n \Rightarrow J_n \} : A : \mathbf{fun}$$

translates into a longer sequence of composition (via *Alt*) of two-step reactions that bind the self-parameter of a copattern and then pattern-match against the rest of the context like so (where $\Phi_0 = \mathbf{whole}, \bullet$):

$$\frac{\begin{array}{c} \mathcal{D}_i \quad IH(\mathcal{E}_i) \\ \vdots \\ IdF;;, To, Exec \\ \vdots \\ Alt, Cede \end{array} \quad \begin{array}{c} \vdots \\ \mathcal{D}_n \end{array} \quad \begin{array}{c} \vdots \\ IH(\mathcal{E}_n) \end{array}}{\Gamma \vdash \{ \varepsilon \mid \text{as } x_1 \text{ to } P_1 \Rightarrow \llbracket J_1 \rrbracket^{-1} \} \dots : A : \mathbf{fun} \mid \Phi_0 \quad \begin{array}{c} \Gamma, x_n : A : \mathbf{fun}, \Delta_n \vdash \llbracket J_n \rrbracket^{-1} \mid \mathbf{whole} \bullet \\ \vdots \\ \Gamma, x_n : A : \mathbf{fun}, \Delta_n \vdash \llbracket J_n \rrbracket^{-1} \mid \Phi_0 \end{array} \text{Exec} \quad \begin{array}{c} \vdots \\ \Gamma, x_n : A : \mathbf{fun} \vdash \text{to } P_n \Rightarrow \llbracket J_n \rrbracket^{-1} \mid \Phi_0 \\ \vdots \\ \Gamma \vdash \text{as } x_n \text{ to } P_n \Rightarrow \llbracket J_n \rrbracket^{-1} : A : \mathbf{fun} \mid \Phi_0 \end{array} \text{To} \quad \begin{array}{c} \vdots \\ IdF;; \\ \vdots \\ Alt \end{array}}{\Gamma \vdash \{ \{ \varepsilon \mid \text{as } x_1 \text{ to } P_1 \Rightarrow \llbracket J_1 \rrbracket^{-1} \} \dots \mid \text{as } x_n \text{ to } P_n \Rightarrow \llbracket J_n \rrbracket^{-1} \} : A : \mathbf{fun} \mid \Phi_0} \text{React} \\ \Gamma \vdash \mathbf{do} \{ \{ \varepsilon \mid \text{as } x_1 \text{ to } P_1 \Rightarrow J_1 \} \dots \mid \text{as } x_n \text{ to } P_n \Rightarrow J_n \} : A : \mathbf{fun} \mid \bullet$$

□

LEMMA 5.1 (POLARITY SOUNDNESS & COMPLETENESS). (1) *If a source J is well-polarized (respectively, (co)term M) then so is $\llbracket J \rrbracket$ (respectively, $\llbracket M \rrbracket$).*
 (2) *If a target J is well-polarized (respectively, M or V) so is $\llbracket J \rrbracket^{-1}$ (respectively, $\llbracket M \rrbracket^{-1}$ or $\llbracket V \rrbracket^{-1}$).*

PROOF. Note that to prove well-polarity in the source language, the left and right-hand sides of $\leadsto$ in the polarity judgement are the same. So for brevity, during simple polarity checking, the correction can be omitted and just a single expression—(co)term, command, or reaction—can be listed. For example, instead of $\Gamma \vdash M \leadsto M :: \mathbf{val} \mid \Psi$, we can write $\Gamma \vdash M :: \mathbf{val} \mid \Psi$ analogous to polarity checking in dual core.

In this way, the proofs of soundness and completeness are similar to theorems 5.2 and 5.3, but with the typing information removed and only the kind **val** or **fun** remaining. □

H Equational Soundness and Completeness

LEMMA H.1 (REVERSE EQUATIONAL SOUNDNESS). (1) *If $J = J'$ then $\llbracket J \rrbracket^{-1} = \llbracket J' \rrbracket^{-1}$.*
 (2) *If $M = M'$ then $\llbracket M \rrbracket^{-1} = \llbracket M' \rrbracket^{-1}$.*

PROOF. By definition of the source CoO equational theory, which includes all axioms of the Dual Core calculus. □

LEMMA 3.4 (VALUE RECONSTRUCTION). $\langle V, W \rangle = (V, W)$ and $\langle X[V] \rangle = X[V]$.

PROOF. For a reconstructed pair value $\langle V, W \rangle$, we have

$$\begin{aligned} \langle V, W \rangle &= \mathbf{do} \, z \Rightarrow \mathbf{go} \, V \text{ as } \mathbf{cbv} \text{ to } \mathbf{do} \, x \Rightarrow \mathbf{go} \, W \text{ as } \mathbf{cbv} \text{ to } \mathbf{do} \, y \Rightarrow \mathbf{go} \, (x, y) \text{ as } \mathbf{cbv} \text{ to } z \\ &= \mathbf{do} \, z \Rightarrow \mathbf{go} \, (V, W) \text{ as } \mathbf{cbv} \text{ to } z & (\beta_{\text{val}}) \\ &= (V, W) & (\eta_{\text{act}}) \end{aligned}$$

A reconstructed tagged value $\langle X[V] \rangle$ is similarly equal to the original value $X[V]$,

$$\begin{aligned}
 \langle X[V] \rangle &= \mathbf{do} \, z \Rightarrow \mathbf{go} \, V \text{ as } \mathbf{cbv} \text{ to } \mathbf{do} \, x \Rightarrow \mathbf{go} \, X[V] \text{ as } \mathbf{cbv} \text{ to } z \\
 &= \mathbf{do} \, z \Rightarrow \mathbf{go} \, X[V] \text{ as } \mathbf{cbv} \text{ to } z & (\beta_{val}) \\
 &= X[V] & (\eta_{act}) \quad \square
 \end{aligned}$$

LEMMA H.2 (BACKWARD-FORWARD ROUND TRIP). *In the Dual Core equational theory,*

- (1) $\llbracket \llbracket J \rrbracket^{-1} \rrbracket = J$.
- (2) $\llbracket \llbracket M \rrbracket^{-1} \rrbracket = M$.
- (3) $\llbracket \llbracket V \rrbracket^{-1} \rrbracket = V$.
- (4) $\llbracket \llbracket \mathbf{as} \, x_i \text{ to } P_i \Rightarrow J_{i..} \rrbracket^{-1} \rrbracket_1 = \mathbf{go} \, \underline{here} \text{ as } \mathbf{cbv}$
 $\quad \mathbf{to} \, \mathbf{do} \, \{ \mid P_i \Rightarrow J_i[\underline{self}/x_i].i. \}$
 $\quad \mid \underline{here} \Rightarrow \mathbf{go} \, \underline{here} \text{ as } \mathbf{cbv} \text{ to } \underline{other} \}.$

PROOF. By mutual induction on the syntax of J , M , and V , and on the list of reactions $\mid \mathbf{as} \, x_i \text{ to } P_i \Rightarrow J_{i..}$.

Commands follow immediately by definition from the inductive hypotheses:

$$\begin{aligned}
 \llbracket \llbracket \mathbf{go} \, M \text{ as } I \text{ to } N \rrbracket^{-1} \rrbracket &= \mathbf{go} \, \llbracket \llbracket M \rrbracket^{-1} \rrbracket \text{ as } I \text{ to } \llbracket \llbracket N \rrbracket^{-1} \rrbracket \\
 &= \mathbf{go} \, M \text{ as } I \text{ to } N & (IH)
 \end{aligned}$$

Values follow with the help of lemma 3.4.

- x and $()$ are immediate.
- (V, W) :

$$\begin{aligned}
 \llbracket \llbracket (V, W) \rrbracket^{-1} \rrbracket &= \langle \llbracket \llbracket V \rrbracket^{-1} \rrbracket, \llbracket \llbracket W \rrbracket^{-1} \rrbracket \rangle \\
 &= \langle V, W \rangle & (IH) \\
 &= (V, W) & (\text{lemma 3.4})
 \end{aligned}$$

- $X[V]$:

$$\begin{aligned}
 \llbracket \llbracket (X[V]) \rrbracket^{-1} \rrbracket &= \langle X[\llbracket \llbracket V \rrbracket^{-1} \rrbracket] \rangle \\
 &= \langle X[V] \rangle & (IH) \\
 &= X[V] & (\text{lemma 3.4})
 \end{aligned}$$

- $\{ M \}$:

$$\begin{aligned}
 \llbracket \llbracket \{ M \} \rrbracket^{-1} \rrbracket &= \{ \llbracket \llbracket M \rrbracket^{-1} \rrbracket \} \\
 &= \{ M \} & (IH)
 \end{aligned}$$

For general terms, most cases follow immediately from the inductive hypothesis, except for the $\mathbf{do} \, \{ \dots \}$ reactive objects of a codata type, which simplify back down to their original form.

- x and V are immediate.
- $\mathbf{do} \, x \Rightarrow J$:

$$\begin{aligned}
 \llbracket \llbracket \mathbf{do} \, x \Rightarrow J \rrbracket^{-1} \rrbracket &= \mathbf{do} \, x \Rightarrow \llbracket \llbracket J \rrbracket^{-1} \rrbracket \\
 &= \mathbf{do} \, x \Rightarrow J & (IH)
 \end{aligned}$$

- $\mathbf{do} \, \{ \mid \mathbf{as} \, x_i \text{ to } P_i \Rightarrow J_{i..} \}$: Note that the given set of pattern-command reactions $P_i \Rightarrow J_{i..}$ covers the type of the term, following the usual constraint of the Dual Core calculus. This

means we can eliminate the dead catch-all branch $_ \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}}$ via $\beta\eta$ axioms in the following equations:

$$\begin{aligned}
& \llbracket \llbracket \text{do } \{ \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \} \rrbracket^{-1} \rrbracket \\
&= \text{do } \{ \text{as } \underline{\text{self}} \text{ to } \underline{\text{here}} \Rightarrow \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket \llbracket \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \rrbracket^{-1} \rrbracket_1) \} \\
&\quad \text{as cbv} \\
&\quad \text{to } (\text{do } \underline{\text{here}} \Rightarrow \text{go } () \text{ as cbv to } \underline{\text{error}}) \\
&= \text{do } \{ \text{as } \underline{\text{self}} \text{ to } \underline{\text{here}} \Rightarrow \text{go } (\text{do } \underline{\text{other}} \Rightarrow \\
&\quad \text{go } \underline{\text{here}} \text{ as cbv} \\
&\quad \text{to do } \{ \mid P_i \Rightarrow J_i[\underline{\text{self}}/x_i].i. \\
&\quad \mid \underline{\text{here}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}} \} \} \\
&\quad \text{as cbv} \\
&\quad \text{to } (\text{do } \underline{\text{here}} \Rightarrow \text{go } () \text{ as cbv to } \underline{\text{error}}) \} \tag{IH} \\
&= \text{do } \{ \text{as } \underline{\text{self}} \text{ to } \underline{\text{here}} \Rightarrow \text{go } (\text{do } \underline{\text{other}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv} \\
&\quad \text{to do } \{ \mid P_i \Rightarrow J_i[\underline{\text{self}}/x_i].i. \}) \\
&\quad \text{as cbv} \\
&\quad \text{to } (\text{do } \underline{\text{here}} \Rightarrow \text{go } () \text{ as cbv to } \underline{\text{error}}) \} \tag{(\beta\eta)} \\
&= \text{do } \{ \text{as } \underline{\text{self}} \text{ to } \underline{\text{here}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to do } \{ \mid P_i \Rightarrow J_i[\underline{\text{self}}/x_i].i. \} \} \tag{(\beta_{fun})} \\
&= \text{do } \{ \underline{\text{here}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to do } \{ \mid \text{as } \underline{\text{self}} \text{ to } P_i \Rightarrow J_i[\underline{\text{self}}/x_i].i. \} \} \tag{(\eta_{rec})} \\
&= \text{do } \{ \mid \text{as } \underline{\text{self}} \text{ to } P_i \Rightarrow J_i[\underline{\text{self}}/x_i].i. \} \tag{(\eta_{react})} \\
&= \text{do } \{ \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \} \tag{(\alpha)}
\end{aligned}$$

Finally, we have lists of reactions, whose round-trip simplifies down to a command which matches on the $\underline{\text{here}}$ variable denoting the current value to match on, along with an additional catch-all branch which passes $\underline{\text{here}}$ to the variable $\underline{\text{other}}$ which is a reference for how to respond if everything here fails to match.

- ε

$$\begin{aligned}
\llbracket \llbracket \varepsilon \rrbracket^{-1} \rrbracket_1 &= \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}} \\
&= \text{go } \underline{\text{here}} \text{ as cbv to do } \{ \varepsilon \mid \underline{\text{here}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}} \} \tag{(\beta_{val})}
\end{aligned}$$

- $\mid \text{as } x \text{ to } P \Rightarrow J \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i.i.$

$$\begin{aligned}
& \llbracket \llbracket \text{as } x \text{ to } P \Rightarrow J \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i.i. \rrbracket^{-1} \rrbracket_1 \\
&= \llbracket \varepsilon \mid \text{as } x \text{ to } P \Rightarrow \llbracket J \rrbracket^{-1} \mid \text{as } x_i \text{ to } P_i \Rightarrow \llbracket J_i \rrbracket^{-1}.i. \rrbracket_1 \\
&= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}}) \text{ as cbv} \\
&\quad \text{to } (\text{do } \underline{\text{here}} \Rightarrow \llbracket \text{as } x \text{ to } P \Rightarrow \llbracket J \rrbracket^{-1} \mid \text{as } x_i \text{ to } P_i \Rightarrow \llbracket J_i \rrbracket^{-1}.i. \rrbracket) \\
&= \llbracket \text{as } x \text{ to } P \Rightarrow \llbracket J \rrbracket^{-1} \mid \text{as } x_i \text{ to } P_i \Rightarrow \llbracket J_i \rrbracket^{-1}.i. \rrbracket \tag{(\beta_{fun}\beta_{val})} \\
&= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket \text{as } x \text{ to } P \Rightarrow \llbracket J \rrbracket^{-1} \rrbracket_1) \text{ as cbv} \\
&\quad \text{to } (\text{do } \underline{\text{here}} \Rightarrow \llbracket \text{as } x_i \text{ to } P_i \Rightarrow \llbracket J_i \rrbracket^{-1}.i. \rrbracket_1)
\end{aligned}$$

$$\begin{aligned}
&= \text{go do } \underline{other} \Rightarrow \\
&\quad \text{go } \underline{self} \text{ as cbn to} \\
&\quad \text{to do } x \Rightarrow \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket J \rrbracket^{-1} \\
&\quad \quad \text{as cbv} \\
&\quad \quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \quad \mid \underline{here} \Rightarrow \text{go } \underline{here} \text{ as cbv to } \underline{other} \} \\
&\quad \text{as cbn to } \underline{here} \\
&\text{as cbv} \\
&\text{to do } \underline{here} \Rightarrow \llbracket \text{as } x_i \text{ to } P_i \Rightarrow \llbracket J_i \rrbracket^{-1} .i. \rrbracket_1 \\
&= \text{go do } \{ \mid P \Rightarrow \llbracket J \rrbracket^{-1} \rrbracket [\underline{self}/x] \quad (\beta_{fun}\beta_{val}) \\
&\quad \mid \underline{here} \Rightarrow \llbracket \text{as } x_i \text{ to } P_i \Rightarrow \llbracket J_i \rrbracket^{-1} .i. \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid P \Rightarrow J[\underline{self}/x] \quad (IH) \\
&\quad \mid \underline{here} \Rightarrow \text{go } \underline{here} \text{ as cbv} \\
&\quad \quad \text{to do } \{ \mid P_i \Rightarrow J_i[\underline{self}/x_i] .i. \\
&\quad \quad \mid \underline{here} \Rightarrow \text{go } \underline{here} \text{ as cbv to } \underline{other} \} \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid P \Rightarrow J[\underline{self}/x] \quad (\eta_{react}\beta_{DC}) \\
&\quad \mid P_i \Rightarrow J_i[\underline{self}/x_i] .i. \\
&\quad \mid \underline{here} \Rightarrow \text{go } \underline{here} \text{ as cbv to } \underline{other} \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go } \underline{here} \text{ as cbv} \quad (\beta^\circ) \\
&\quad \text{to do } \{ \mid P \Rightarrow J[\underline{self}/x] \\
&\quad \mid P_i \Rightarrow J_i[\underline{self}/x_i] .i. \\
&\quad \mid \underline{here} \Rightarrow \text{go } \underline{here} \text{ as cbv to } \underline{other} \} \quad \square
\end{aligned}$$

LEMMA H.3. (1) $\llbracket J[(\text{go } M \text{ as } O \text{ to } N)/(\text{go } M \text{ as } O)] \rrbracket = \llbracket J \rrbracket [\llbracket N \rrbracket / \underline{exit}]$
(2) $\llbracket M'[(\text{go } M \text{ as } O \text{ to } N)/(\text{go } M \text{ as } O)] \rrbracket = \llbracket M' \rrbracket [\llbracket N \rrbracket / \underline{exit}]$
(3) $\llbracket R[(\text{go } M \text{ as } O \text{ to } N)/(\text{go } M \text{ as } O)] \rrbracket_n = \llbracket R \rrbracket_n [\llbracket N \rrbracket / \underline{exit}]$

PROOF. By mutual induction on the syntax of J , M' , and R . The base cases are

- $(\text{go } M \text{ as } O)[(\text{go } M \text{ as } O \text{ to } N)/(\text{go } M \text{ as } O)] = \text{go } M \text{ as } O \text{ to } N$, which follows as

$$\begin{aligned}
\llbracket \text{go } M \text{ as } O \rrbracket [\llbracket N \rrbracket / \underline{exit}] &= (\text{go } \llbracket M \rrbracket \text{ as strat}(O) \text{ to } \underline{exit}) [\llbracket N \rrbracket / \underline{exit}] \\
&= \text{go } \llbracket M \rrbracket \text{ as strat}(O) \text{ to } \llbracket N \rrbracket \\
&= \llbracket \text{go } M \text{ as } O \text{ to } N \rrbracket
\end{aligned}$$

- $\{ \Rightarrow M \} [(\text{go } M \text{ as cbn to } N)/(\text{go } M \text{ as cbn})] = \{ \Rightarrow \text{go } M \text{ as cbn to } N \}$ which follows as

$$\begin{aligned}
\llbracket \Rightarrow M \rrbracket_0 [\llbracket N \rrbracket / \underline{exit}] &= (\text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{exit}) [\llbracket N \rrbracket / \underline{exit}] \\
&= \text{go } \llbracket M \rrbracket \text{ as cbn to } \llbracket N \rrbracket \\
&= \llbracket \Rightarrow \text{go } M \text{ as cbn to } N \rrbracket_0
\end{aligned}$$

These are the only two cases where the translation actually refers to a use of $\underline{exit}$ (as opposed to binding or ignoring it). All other cases follow by the definition of ordinary substitution in Dual Core and the inductive hypotheses. $\square$

LEMMA H.4. (1) $\llbracket R \text{ ++ } J \rrbracket_n = \llbracket R \rrbracket_n [\text{do } \{ \mid () \Rightarrow \llbracket J \rrbracket \} / \text{next}]$
 (2) $\llbracket J' \text{ ++ } J \rrbracket = \llbracket J' \rrbracket [\text{do } \{ \mid () \Rightarrow \llbracket J \rrbracket \} / \text{next}]$

PROOF. By induction on the syntax of R and J' . None of the cases directly involve the next variable, so the substitution is just pushed through the inductive hypotheses. At the base case for an empty command ε , where $\varepsilon \text{ ++ } J = J$, we have

$$\begin{aligned} \llbracket \varepsilon \rrbracket [\text{do } \{ \mid () \Rightarrow \llbracket J \rrbracket \} / \text{next}] &= (\text{go } () \text{ as cbv to next}) [\text{do } \{ \mid () \Rightarrow \llbracket J \rrbracket \} / \text{next}] \\ &= \text{go } () \text{ as cbv to do } \{ \mid () \Rightarrow \llbracket J \rrbracket \} \\ &= \llbracket J \rrbracket \end{aligned} \quad (\beta_{DC}) \quad \square$$

THEOREM 5.5 (EQUATIONAL SOUNDNESS). (1) If $J = J'$ then $\llbracket J \rrbracket = \llbracket J' \rrbracket$ when defined.
 (2) If $M = M'$ then $\llbracket M \rrbracket = \llbracket M' \rrbracket$ when defined.
 (3) If $R = R'$ then $\llbracket R \rrbracket_n = \llbracket R' \rrbracket_n$ when defined.

PROOF. The $\beta\eta$ axioms that come directly from Dual Core are sound after (re)translation due to lemmas H.1 and H.2. In particular, assuming that $J =_\beta J'$ in Dual Core, then by lemma H.1 we have $\llbracket J \rrbracket^{-1} = \llbracket J' \rrbracket^{-1}$ in the source equational theory, and thus

$$\begin{aligned} \llbracket \llbracket J \rrbracket^{-1} \rrbracket &= J && (\text{lemma H.2}) \\ &= J' && (\text{assumption}) \\ &= \llbracket \llbracket J' \rrbracket^{-1} \rrbracket && (\text{lemma H.2}) \end{aligned}$$

Soundness of axioms for η equivalent terms follows similarly from lemmas H.1 and H.2 as above. And more generally, given two sides matching a Dual Core axiom—but which is outside the Dual Core sub-syntax deeper in the expressions—the proof of soundness is exactly analogous to lemma H.2 except that we might have some other arbitrary sub-expression rather than the image of reverse translation $\llbracket _ \rrbracket^{-1}$.

The cases for the added axioms in fig. 10 are straightforward, as follows:

- Both sides of the *erase*, *goto_{fun}*, *goto_{val}* axioms translate exactly the same.
- The appending operation in *goto_{match}* simulates a β_{fun} substitution for the local next variable

$$\begin{aligned} \llbracket \text{match } MR; J \rrbracket &= \text{go } (\text{do next} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbv to } \llbracket \text{do } R \rrbracket) \text{ as cbv to do } \{ \mid () \Rightarrow \llbracket J \rrbracket \} \\ &= \text{go } \llbracket M \rrbracket \text{ as cbv to } \llbracket \text{do } R \rrbracket_{\text{do } \{ \mid () \Rightarrow \llbracket J \rrbracket \} / \text{next}} && (\beta_{fun}) \\ &= \text{go } \llbracket M \rrbracket \text{ as cbv to } \llbracket \text{do } (R \text{ ++ } J) \rrbracket && (\text{lemma H.4}) \\ &= \llbracket \text{go } M \text{ as cbv to do } (R \text{ ++ } J) \rrbracket \end{aligned}$$

- After translation, the only difference between the left- and right-hand sides of $\kappa_{@}$, κ_X , $\kappa_!$, $\varsigma_{@}$, and ς_X are β_{val} or β_{fun} equalities substituting for the local binding of here, and a potential β° to flip a command. For example, the two sides of $\varsigma_{@}$ are equal as

$$\begin{aligned} \llbracket \text{go } (M, M') \text{ as cbv to } N \rrbracket &= \text{go } \langle \llbracket M \rrbracket, \llbracket M' \rrbracket \rangle \text{ as cbv to } \llbracket N \rrbracket \\ &= \text{go } \llbracket M \rrbracket \text{ as cbv to do } x \Rightarrow \text{go } \llbracket M' \rrbracket \text{ as cbv to do } y \Rightarrow \text{go } \langle x, y \rangle \text{ as cbv to } \llbracket N \rrbracket && (\beta_{fun}) \\ &= \text{go } \llbracket M \rrbracket \text{ as cbv to do } x \Rightarrow \text{go } \llbracket M' \rrbracket \text{ as cbv to do } y \Rightarrow \text{go } \langle x, y \rangle \text{ as cbv to } \llbracket N \rrbracket && (\text{lemma 3.4}) \\ &= \llbracket \text{go } M \text{ as cbv to do } x \Rightarrow \text{go } M' \text{ as cbv to do } y \Rightarrow \text{go } \langle x, y \rangle \text{ as cbv to } N \rrbracket \end{aligned}$$

and the two sides of κ_X are equal as

$$\llbracket \text{go } M.X \text{ as cbn to } N \rrbracket = \text{go } (\text{do here} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } X[\text{here}]) \text{ as cbn to } \llbracket N \rrbracket$$

$$\begin{aligned}
&= \text{go } \llbracket N \rrbracket \text{ as cbv to } \underline{\text{do here}} \Rightarrow \text{go } X[\underline{\text{here}}] \text{ to as cbv } \llbracket M \rrbracket & (\beta^\circ) \\
&= \text{go } \langle X[\llbracket N \rrbracket] \rangle \text{ as cbv to } \llbracket M \rrbracket & (\beta_{fun}) \\
&= \text{go } \llbracket M \rrbracket \text{ as cbn to } \langle X[\llbracket N \rrbracket] \rangle & (\beta^\circ) \\
&= \llbracket \text{go } M \text{ as cbn to } X[N] \rrbracket
\end{aligned}$$

Finally, the cases for the added axioms in fig. 11 as follows:

- (*commit_{error}*) $\text{do } \{ R \mid _ \Rightarrow \text{go as cbv to } \underline{\text{error}} \} = \text{do } \{ R \}$ is sound as

$$\begin{aligned}
&\llbracket \text{do } \{ R \mid _ \Rightarrow \text{go to } \underline{\text{error}} \} \rrbracket \\
&= \text{do } \{ \mid \text{as } \underline{\text{self}} \text{ to } \underline{\text{here}} \Rightarrow \text{go } (\underline{\text{do other}} \Rightarrow \text{go } (\underline{\text{do other}} \Rightarrow \llbracket R \rrbracket_1) \text{ as cbv} \\
&\quad \text{to } (\underline{\text{do here}} \Rightarrow \text{go as cbv to } \underline{\text{error}})) \\
&\quad \text{as cbv to } (\underline{\text{do }} \Rightarrow \text{go as cbv to } \underline{\text{error}}) \} \\
&= \text{do } \{ \mid \text{as } \underline{\text{self}} \text{ to } \underline{\text{here}} \Rightarrow \text{go } (\underline{\text{do other}} \Rightarrow \llbracket R \rrbracket_1) \text{ as cbv} & (\beta_{fun}) \\
&\quad \text{to } (\underline{\text{do here}} \Rightarrow \text{go as cbv to } \underline{\text{error}}) \} \\
&= \llbracket \text{do } \{ R \} \rrbracket
\end{aligned}$$

- (*commit_{exec}*) $\{ \Rightarrow J \} \mid R = \{ \Rightarrow J \}$ (note that $\underline{\text{other}} \notin FV(\llbracket J \rrbracket)$) is sound as either the nullary case

$$\begin{aligned}
\llbracket \{ \Rightarrow J \} \mid R \rrbracket_0 &= \text{go } (\underline{\text{do other}} \Rightarrow \llbracket J \rrbracket) \\
&\quad \text{as cbv to } (\underline{\text{do here}} \Rightarrow \llbracket R \rrbracket_0) \\
&= \llbracket J \rrbracket & (\beta_{fun}) \\
&= \llbracket \Rightarrow J \rrbracket_0
\end{aligned}$$

or the unary case

$$\begin{aligned}
\llbracket \{ \Rightarrow J \} \mid R \rrbracket_1 &= \text{go } (\underline{\text{do other}} \Rightarrow \text{go } \underline{\text{do exit}} \Rightarrow \llbracket J \rrbracket \text{ as cbn to } \underline{\text{here}}) \\
&\quad \text{as cbv to } (\underline{\text{do here}} \Rightarrow \llbracket R \rrbracket_1) \\
&= \text{go } \underline{\text{do exit}} \Rightarrow \llbracket J \rrbracket \text{ as cbn to } \underline{\text{here}} & (\beta_{fun}) \\
&= \llbracket \Rightarrow J \rrbracket_1
\end{aligned}$$

- (*commit_{eval}*) $\{ \Rightarrow M \} \mid R = \{ \Rightarrow M \}$ (note that $\underline{\text{other}} \notin FV(\llbracket M \rrbracket)$) is sound as either the unary case

$$\begin{aligned}
\llbracket \{ \Rightarrow M \} \mid R \rrbracket_1 &= \text{go } (\underline{\text{do other}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{here}}) \\
&\quad \text{as cbv to } (\underline{\text{do here}} \Rightarrow \llbracket R \rrbracket_1) \\
&= \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{here}} & (\beta_{fun}) \\
&= \llbracket \Rightarrow M \rrbracket_1
\end{aligned}$$

or the nullary case

$$\begin{aligned}
\llbracket \{ \Rightarrow M \} \mid R \rrbracket_0 &= \text{go } (\underline{\text{do other}} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{exit}}) \\
&\quad \text{as cbv to } (\underline{\text{do here}} \Rightarrow \llbracket R \rrbracket_0) \\
&= \text{go } \llbracket M \rrbracket \text{ as cbn to } \underline{\text{exit}} & (\beta_{fun}) \\
&= \llbracket \Rightarrow M \rrbracket_0
\end{aligned}$$

- $(eval)$ $\text{to } x \Rightarrow \text{go } M \text{ as } O \text{ to } x = \{ \Rightarrow M \}$ where $x \notin FV(M)$ is sound as

$$\begin{aligned}
\llbracket \text{to } x \Rightarrow \text{go } M \text{ as } O \text{ to } x \rrbracket_1 &= \text{go do } \{ \mid x \Rightarrow \text{go do } \underline{other} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as } \text{strat}(O) \text{ to } x \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \underline{other} \Rightarrow \text{go } \llbracket M \rrbracket \text{ as } \text{strat}(O) \text{ to } \underline{here} & (\beta_{val}) \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&= \text{go } \llbracket M \rrbracket \text{ as } \text{strat}(O) \text{ to } \underline{here} & (\beta_{fun}) \\
&= \llbracket \Rightarrow M \rrbracket_1
\end{aligned}$$

- $(eval_\bullet)$ $\Rightarrow \text{go } M \text{ as } O = \{ \Rightarrow M \}$ is sound as

$$\begin{aligned}
\llbracket \Rightarrow \text{go } M \text{ as } O \rrbracket_0 &= \text{go } \llbracket M \rrbracket \text{ as } \text{strat}(O) \text{ to } \underline{exit} \\
&= \llbracket \Rightarrow M \rrbracket_0
\end{aligned}$$

- $(exec_\bullet)$ $\text{to } x \Rightarrow J[(\text{go } M \text{ as } O \text{ to } x)/(\text{go } M \text{ as } O)] = \{ \Rightarrow J \}$ (note that $\underline{other} \notin FV(\llbracket J \rrbracket)$) is sound as

$$\begin{aligned}
&\llbracket \text{to } x \Rightarrow J[(\text{go } M \text{ as } O \text{ to } x)/(\text{go } M \text{ as } O)] \rrbracket \\
&= \text{go do } \{ \mid x \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket J[(\text{go } M \text{ as } O \text{ to } x)/(\text{go } M \text{ as } O)] \rrbracket \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid x \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket J \rrbracket [x/\underline{exit}] \\
&\quad \text{as cbv} & (\text{lemma H.3}) \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \underline{other} \Rightarrow \llbracket J \rrbracket [\underline{here}/\underline{exit}] & (\beta_{val}) \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&= \llbracket J \rrbracket [\underline{here}/\underline{exit}] & (\beta_{fun}) \\
&= \text{go do } \underline{exit} \Rightarrow \llbracket J \rrbracket \text{ as cbn to } \underline{here} & (\beta_{val}) \\
&= \llbracket \Rightarrow J \rrbracket_1
\end{aligned}$$

- $(self_{anon})$ $\text{as } x \text{ } R = R$ where $x \notin FV(R)$ is sound as

$$\begin{aligned}
\llbracket \text{as } x \text{ } R \rrbracket &= \text{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1 \\
&= \llbracket R \rrbracket_1 & (\beta_{fun})
\end{aligned}$$

- $(self_{alias})$ $\text{as } Q \text{ as } x \text{ } R = \text{as } Q \text{ } R[Q/x]$ is sound by induction on the syntax of the copattern Q :
 - $Q = y$ is sound as

$$\llbracket \text{as } y \text{ as } x \text{ } R \rrbracket = \text{go } \underline{self} \text{ as cbn to do } y \Rightarrow \text{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1$$

$$\begin{aligned}
&= \llbracket R \rrbracket_1 [\underline{self}/y, \underline{self}/x] && (\beta_{fun}) \\
&= \llbracket R \rrbracket_1 [y/x] [\underline{self}/y] \\
&= \text{go } \underline{self} \text{ as cbn to do } y \Rightarrow \llbracket R \rrbracket_1 [y/x] && (\beta_{fun}) \\
&= \llbracket \text{as } y \text{ } R[y/x] \rrbracket_1
\end{aligned}$$

– The special case $Q = _ P$ is sound as

$$\begin{aligned}
&\llbracket \text{as } (_ P) \text{ as } x \text{ } R \rrbracket \\
&= \text{go do } \{ \mid (P, \underline{here}) \Rightarrow \text{go } \llbracket \underline{self} \text{ } P \rrbracket \text{ as cbn} \\
&\quad \text{to do } \underline{self} \Rightarrow \text{go do } \underline{other} \Rightarrow \text{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket R \rrbracket_1 \\
&\quad \text{as cbv to } \llbracket \underline{other} \text{ } P \rrbracket \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid (P, \underline{here}) \Rightarrow \text{go } \llbracket \underline{self} \text{ } P \rrbracket \text{ as cbn} && (\beta_{fun}) \\
&\quad \text{to do } \underline{self} \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R \rrbracket_1 [\llbracket \underline{self} \text{ } P \rrbracket / x] \\
&\quad \text{as cbv to } \llbracket \underline{other} \text{ } P \rrbracket \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid (P, \underline{here}) \Rightarrow \text{go } \llbracket \underline{self} \text{ } P \rrbracket \text{ as cbn} && (\beta_{fun}) \\
&\quad \text{to do } \underline{self} \Rightarrow \text{go do } \underline{other} \Rightarrow \text{go } \underline{self} \text{ as cbn} \\
&\quad \text{to do } y \Rightarrow \llbracket R \rrbracket_1 [\llbracket y \text{ } P \rrbracket / x] \\
&\quad \text{as cbv to } \llbracket \underline{other} \text{ } P \rrbracket \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \llbracket \text{as } (y \text{ } P) \text{ } R[(y \text{ } P/x)] \rrbracket \\
&\quad \text{– Otherwise, } Q = Q' P \text{ is sound by the inductive hypothesis and the previous special} \\
&\quad \text{case, since}
\end{aligned}$$

$$\llbracket \text{as } (Q' P) \text{ as } x \text{ } R \rrbracket = \llbracket \text{as } Q' \text{ as } (_ P) \text{ as } x \text{ } R \rrbracket$$

– $Q = Q'.X$ is sound similarly to the previous two cases for application.

- (*asto@*) In the special case $\text{as } (_ P') \text{ to } P \text{ } R = \text{as } _ \text{ to } (P', P) \text{ } R$ note that by self_{anon} ,

$$\text{as } _ \text{ to } (P', P) \text{ } R = \text{to } (P', P) \text{ } R$$

is sound as above, so it suffices to show $\text{as } (_ P') \text{ to } P \text{ } R = \text{to } (P', P) \text{ } R$ is sound as follows

$$\begin{aligned}
&\llbracket \text{as } (_ P') \text{ to } P \text{ } R \rrbracket_1 \\
&= \text{go do } \{ \mid (P', \underline{here}) \Rightarrow \llbracket \text{to } P \text{ } R \rrbracket_1 [\llbracket \underline{self} \text{ } P' \rrbracket / \underline{self}, \llbracket \underline{other} \text{ } P' \rrbracket / \underline{other}] && (\beta_{fun}) \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid (P', \underline{here}) \Rightarrow \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R \rrbracket_0 \text{ as cbv} && (\underline{self} \notin \llbracket R \rrbracket_0) \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \llbracket \underline{other} \text{ } P' \rrbracket \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here}
\end{aligned}$$

$$\begin{aligned}
&= \text{go do } \{ \mid (P', P) \Rightarrow \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_0 \text{ as cbv} \} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \llbracket \underline{\text{other}} P' \rrbracket \\
&\quad \mid \underline{\text{here}} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } P \\
&\quad \mid \underline{\text{here}} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\text{as cbn to } \underline{\text{here}} \\
&= \text{go do } \{ \mid (P', P) \Rightarrow \text{go do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_0 \text{ as cbv} \} \quad (\eta\beta) \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \llbracket \underline{\text{other}} P' \rrbracket \\
&\quad \mid \underline{\text{here}} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{\text{here}} \\
&= \text{go do } \{ \mid (P', P) \Rightarrow \text{go do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_0 \text{ as cbv} \} \quad (\beta_{DC}) \\
&\quad \text{to do } _ \Rightarrow \text{go } (P', P) \text{ as cbv } \underline{\text{other}} \\
&\quad \mid \underline{\text{here}} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{\text{here}} \\
&= \llbracket \text{to } (P', P) \Rightarrow R \rrbracket_1 \quad (\beta_{val})
\end{aligned}$$

Otherwise, the general case $\text{as } (Q P') \text{ to } P = \text{as } Q \text{ to } (P', P)$ is sound by definition.

- (asto_X) In the special case $\text{as } _ .X \text{ to } P R = \text{as } _ \text{ to } X[P] R$ note that by self_{anon} ,

$$\text{as } _ \text{ to } X[P] R = \text{to } X[P] R$$

is sound as above, so it suffices to show $\text{as } _ .X \text{ to } P R = \text{to } X[P] R$ is sound similarly to the previous special case for application $\text{as } (_ P') \text{ to } P R = \text{as } _ \text{ to } (P', P) R$.

Otherwise, the general case $\text{as } (Q.X) \text{ to } P = \text{as } Q \text{ to } X[P]$ is sound by definition.

- $(\text{id}_L) \varepsilon \mid R = R$ is sound in the unary case as

$$\begin{aligned}
\llbracket \varepsilon \mid R \rrbracket_0 &= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}}) \text{ as cbv to } (\text{do } \underline{\text{here}} \Rightarrow \llbracket R \rrbracket_0) \\
&= \text{go } \underline{\text{here}} \text{ as cbv to } (\text{do } \underline{\text{here}} \Rightarrow \llbracket R \rrbracket_0) \quad (\beta_{fun}) \\
&= \llbracket R \rrbracket_0 \quad (\beta_{val})
\end{aligned}$$

is sound in the nullary case (where $\underline{\text{here}} \notin FV(\llbracket R \rrbracket_0)$) as

$$\begin{aligned}
\llbracket \varepsilon \mid R \rrbracket_0 &= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \text{go } () \text{ as cbv to } \underline{\text{other}}) \text{ as cbv to } (\text{do } \underline{\text{here}} \Rightarrow \llbracket R \rrbracket_0) \\
&= \text{go } () \text{ as cbv to } (\text{do } \underline{\text{here}} \Rightarrow \llbracket R \rrbracket_0) \quad (\beta_{fun}) \\
&= \llbracket R \rrbracket_0 \quad (\beta_{val})
\end{aligned}$$

- $(\text{id}_R) R \mid \varepsilon = R$ is sound in the unary case as

$$\begin{aligned}
\llbracket R \mid \varepsilon \rrbracket_1 &= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_1) \text{ as cbv to } (\text{do } \underline{\text{here}} \Rightarrow \text{go } \underline{\text{here}} \text{ as cbv to } \underline{\text{other}}) \\
&= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_1) \text{ as cbv to } \underline{\text{other}} \quad (\eta_{act}) \\
&= \llbracket R \rrbracket_1 \quad (\beta_{val})
\end{aligned}$$

and is sound in the nullary case as

$$\begin{aligned}
\llbracket R \mid \varepsilon \rrbracket_0 &= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_0) \text{ as cbv to } (\text{do } \underline{\text{here}} \Rightarrow \text{go } () \text{ as cbv to } \underline{\text{other}}) \\
&= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_0) \text{ as cbv to do } \{ \mid () \Rightarrow \text{go } () \text{ as cbv to } \underline{\text{other}} \} \quad (\eta_{react}\beta_{val}) \\
&= \text{go } (\text{do } \underline{\text{other}} \Rightarrow \llbracket R \rrbracket_0) \text{ as cbv to } \underline{\text{other}} \quad (\eta_{react}) \\
&= \llbracket R \rrbracket_0 \quad (\beta_{val})
\end{aligned}$$

- (*assoc*) $R_1 \mid \{ R_2 \mid R_3 \} = \{ R_1 \mid R_2 \} \mid R_3$ is sound as

$$\begin{aligned}
\llbracket R_1 \mid \{ R_2 \mid R_3 \} \rrbracket_n &= \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_n \\
&\quad \text{as cbv} \\
&\quad \text{to do } \underline{here} \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_2 \rrbracket_n \\
&\quad \quad \text{as cbv} \\
&\quad \quad \text{to do } \underline{here} \Rightarrow \llbracket R_3 \rrbracket_n \\
&= \llbracket R_1 \rrbracket_n [(\text{do } \underline{here} \Rightarrow \llbracket R_2 \rrbracket_n [(\text{do } \underline{here} \Rightarrow \llbracket R_3 \rrbracket_n) / \underline{other}]) / \underline{other}] \quad (\beta_{fun}) \\
&= \text{go do } \underline{other} \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_n \quad (\beta_{fun}) \\
&\quad \text{as cbv} \\
&\quad \text{to do } \underline{here} \Rightarrow \llbracket R_2 \rrbracket_n \\
&\quad \text{as cbv} \\
&\quad \text{to do } \underline{here} \Rightarrow \llbracket R_3 \rrbracket_n \\
&= \llbracket \{ R_1 \mid R_2 \} \mid R_3 \rrbracket_n
\end{aligned}$$

- (*annih_{to}*) $\text{to } P \ \varepsilon = \varepsilon$ is sound as

$$\begin{aligned}
\llbracket \text{to } P \ \varepsilon \rrbracket_1 &= \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{other} \Rightarrow \text{go } () \text{ as cbv to } \underline{other} \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \mid \underline{here} \Rightarrow \text{go } \underline{here} \text{ as cbv to } \underline{other} \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid P \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \quad (\beta_{fun}\beta_{val}) \\
&\quad \mid \underline{here} \Rightarrow \text{go } \underline{here} \text{ as cbv to } \underline{other} \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go } \underline{other} \text{ as cbn to } \underline{here} \quad (\eta_{react}) \\
&= \text{go } \underline{here} \text{ as cbv to } \underline{other} \quad (\beta^\circ) \\
&= \llbracket \varepsilon \rrbracket_1
\end{aligned}$$

- (*distr_{to}*) $\text{to } P \ \{ R_1 \mid R_2 \} = \{ \text{to } P \ R_1 \} \mid \{ \text{to } P \ R_2 \}$ is sound as (where $\underline{here} \notin FV(\llbracket R_2 \rrbracket_0)$)

$$\begin{aligned}
&\llbracket \text{to } P \ \{ R_1 \mid R_2 \} \rrbracket_1 \\
&= \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{other} \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_0 \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \llbracket R_2 \rrbracket_0 \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid P \Rightarrow \llbracket R_1 \rrbracket_0 [(\text{do } _ \Rightarrow \llbracket R_2 \rrbracket_0 [(\text{do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other}) / \underline{other}]) / \underline{other}] \quad (\beta_{fun}) \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here}
\end{aligned}$$

$$\llbracket \{ \text{to } P \ R_1 \} \mid \{ \text{to } P \ R_2 \} \rrbracket_1$$

$$\begin{aligned}
&= \text{go do } \underline{other} \Rightarrow \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_0 \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&\text{as cbv} \\
&\text{to do } \underline{here} \Rightarrow \llbracket \text{to } P \ R_2 \rrbracket_1 \\
&= \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_0 \quad (\beta_{fun}) \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to do } \underline{here} \Rightarrow \llbracket \text{to } P \ R_2 \rrbracket_1 \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid P \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_0 \quad (\beta_{val}\beta_{DC}) \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_2 \rrbracket_0 \\
&\quad \text{as cbv} \\
&\quad \text{to do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other} \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here} \\
&= \text{go do } \{ \mid P \Rightarrow \llbracket R_1 \rrbracket_0 [(\text{do } _ \Rightarrow \llbracket R_2 \rrbracket_0 [(\text{do } _ \Rightarrow \text{go } P \text{ as cbv to } \underline{other}) / \underline{other}]) / \underline{other}] \quad (\beta_{fun}) \\
&\quad \mid \underline{here} \Rightarrow \llbracket \varepsilon \rrbracket_1 \} \\
&\quad \text{as cbn to } \underline{here}
\end{aligned}$$

- ($annih_{as}$) as $Q \ \varepsilon = \varepsilon$ is sound by induction on the syntax of Q . The base case is

$$\begin{aligned}
\llbracket \text{as } x \ \varepsilon \rrbracket_1 &= \text{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket \varepsilon \rrbracket_1 \\
&= \llbracket \varepsilon \rrbracket_1 \quad (\beta_{fun})
\end{aligned}$$

The two special cases for as $(_ P) \ \varepsilon$ and as $_ . X \ \varepsilon$ are similar to the $annih_{to}$ above. And the general cases follow from the inductive hypotheses and the definitions

$$\begin{aligned}
\llbracket \text{as } (Q \ P) \ \varepsilon \rrbracket_1 &= \llbracket \text{as } Q \text{ as } (_ P) \ \varepsilon \rrbracket_1 \\
\llbracket \text{as } Q . X \ \varepsilon \rrbracket_1 &= \llbracket \text{as } Q \text{ as } _ . X \ \varepsilon \rrbracket_1
\end{aligned}$$

- ($distr_{as}$) as $Q \ \{ R_1 \mid R_2 \} = \{ \text{as } Q \ R_1 \} \mid \{ \text{as } Q \ R_2 \}$ is sound by induction on the syntax of Q . The base case is

$$\begin{aligned}
\llbracket \text{as } x \ \{ R_1 \mid R_2 \} \rrbracket_1 &= \text{go } \underline{self} \text{ as cbn} \\
&\quad \text{to do } x \Rightarrow \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_1 \\
&\quad \text{as cbv} \\
&\quad \text{to do } \underline{here} \Rightarrow \llbracket R_2 \rrbracket_1 \\
&= \text{go do } \underline{other} \Rightarrow \llbracket R_1 \rrbracket_1 [\underline{self}/x] \quad (\beta_{fun}) \\
&\quad \text{as cbv} \\
&\quad \text{to do } \underline{here} \Rightarrow \llbracket R_2 \rrbracket_1 [\underline{self}/x] \\
&= \text{go do } \underline{other} \Rightarrow \text{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket R_1 \rrbracket_1 \quad (\beta_{fun}) \\
&\quad \text{as cbv} \\
&\quad \text{to do } \underline{here} \Rightarrow \text{go } \underline{self} \text{ as cbn to do } x \Rightarrow \llbracket R_2 \rrbracket_1
\end{aligned}$$

$$= \llbracket \{ \text{as } x \ R_1 \} \mid \{ \text{as } x \ R_2 \} \rrbracket$$

The two special cases for $\text{as } (_ P) \ \{ R_1 \mid R_2 \}$ and $\text{as } _ . X \ \{ R_1 \mid R_2 \}$ are similar to the distr_{to} above. And the general cases follow from the inductive hypotheses and the definitions

$$\llbracket \text{as } (Q \ P) \ R \rrbracket_1 = \llbracket \text{as } Q \ \text{as } (_ P) \ R \rrbracket_1$$

$$\llbracket \text{as } Q . X \ R \rrbracket_1 = \llbracket \text{as } Q \ \text{as } _ . X \ R \rrbracket_1$$

□

LEMMA 4.6 (REACTION ENUMERATION).

- (1) If $\Gamma \vdash R : A : \text{fun} \mid \Phi$ is guardless, then $R = \varepsilon \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \mid \dots$
- (2) If $\Gamma \vdash R : \bullet \mid \Phi$ is guardless, then either $R = \{ \Rightarrow J \}$ or $R = \varepsilon$.

PROOF. By induction on the typing derivation of $\Gamma \vdash R : \Psi \mid \Psi$.

- (Cede) The empty reaction is immediate in either case.
- (Alt) The alternative composition $R \mid R'$ proceeds in one of two ways, depending on whether Ψ is empty or not.
 - If $\Psi = A : \text{fun}$, then the inductive hypotheses are

$$R = \varepsilon \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \mid \dots \quad R' = \varepsilon \mid \text{as } x'_j \text{ to } P'_j \Rightarrow J'_j \mid \dots$$

These two reactions can be combined into a single sequence by the associativity and identity axioms,

$$\begin{aligned} R \mid R' &= \{ \varepsilon \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \mid \dots \} \mid \{ \varepsilon \mid \text{as } x'_j \text{ to } P'_j \Rightarrow J'_j \mid \dots \} \\ &= \{ \{ \{ \varepsilon \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \mid \dots \} \mid \varepsilon \} \} \mid \text{as } x'_j \text{ to } P'_j \Rightarrow J'_j \mid \dots \\ &= \{ \{ \varepsilon \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \mid \dots \} \} \mid \text{as } x'_j \text{ to } P'_j \Rightarrow J'_j \mid \dots \\ &= \varepsilon \mid \text{as } x_i \text{ to } P_i \Rightarrow J_i \mid \dots \mid \text{as } x'_j \text{ to } P'_j \Rightarrow J'_j \mid \dots \end{aligned}$$

- If $\Psi = \bullet$, the inductive hypotheses are

$$\begin{array}{lll} R = \{ \Rightarrow J \} & \text{or} & R = \varepsilon \\ R' = \{ \Rightarrow J' \} & \text{or} & R' = \varepsilon \end{array}$$

Every possible combination of $R \mid R'$ is equal to either an empty reaction or a single command right-hand side as follows:

$$\begin{aligned} \varepsilon \mid \varepsilon &= \varepsilon \\ \{ \Rightarrow J \} \mid \varepsilon &= \{ \Rightarrow J \} \\ \varepsilon \mid \{ \Rightarrow J' \} &= \{ \Rightarrow J' \} \\ \{ \Rightarrow J \} \mid \{ \Rightarrow J' \} &= \{ \Rightarrow J \} \end{aligned}$$

- (Exec) The command right-hand side $\Rightarrow J$ side is immediate.
- (Exec') The command right-hand side $\Rightarrow J$ with an implicit context binding over the body $\Gamma \vdash J \mid \Xi A : \text{fun}$, such that $\Psi = A : \text{fun}$ is non-empty and $\Phi = \Xi \Psi'$, expands to

$$\begin{aligned} \{ \Rightarrow J \} &= \text{to } x \Rightarrow J[(\text{go } M \text{ as } O \text{ to } x)/(\text{go } M \text{ as } O)] \\ &= \text{as } _ \text{ to } x \Rightarrow J[(\text{go } M \text{ as } O \text{ to } x)/(\text{go } M \text{ as } O)] \\ &= \varepsilon \mid \text{as } _ \text{ to } x \Rightarrow J[(\text{go } M \text{ as } O \text{ to } x)/(\text{go } M \text{ as } O)] \end{aligned}$$

- (Eval) The term right-hand side $\Rightarrow M$ expands to

$$\begin{aligned} \{ \Rightarrow M \} &= \text{to } x \Rightarrow \text{go } M \text{ as } \text{cbn to } x \\ &= \text{as } _ \text{ to } x \Rightarrow \text{go } M \text{ as } \text{cbn to } x \end{aligned}$$

- (*Eval'*) The term right-hand side $\Rightarrow M$ with an implicit context jump in the body $\Gamma \vdash M : A : \mathbf{fun} \mid A : \mathbf{fun}$, such that $\Psi = \bullet$ and $\Phi = \Xi A : \mathbf{fun}$, expands to

$$\{\Rightarrow M\} = \{\Rightarrow \mathbf{go} M \mathbf{as} \mathbf{cbn}\}$$

- (*To*) The context pattern abstraction $\mathbf{to} P R$ over a reaction $\Gamma \vdash R : \bullet \mid \Phi$ has the inductive hypothesis

$$R = \{\Rightarrow J\} \quad \text{or} \quad R = \varepsilon$$

In the first case, we have a singleton sequence of alternatives

$$\begin{aligned} \mathbf{to} P \Rightarrow J &= \mathbf{as_to} P \Rightarrow J \\ &= \varepsilon \mid \mathbf{as_to} P \Rightarrow J \end{aligned}$$

and in the second case, we have an empty sequence of alternatives

$$\mathbf{to} P \varepsilon = \varepsilon$$

- (*As*) The copattern abstraction $\mathbf{as} Q R$ over a reaction $\Gamma \vdash R : B : \mathbf{fun} \mid \Phi$ has the inductive hypothesis

$$R = \varepsilon \mid \mathbf{as} x_i \mathbf{to} P_i \Rightarrow J_i \mid \dots$$

The copattern $\mathbf{as}$ abstraction distributes over each alternative, combines with the inner recursive binding like so:

$$\begin{aligned} \mathbf{as} Q \{ \varepsilon \mid \mathbf{as} x_i \mathbf{to} P_i \Rightarrow J_i \mid \dots \} \\ &= \{ \mathbf{as} Q \varepsilon \} \mid \{ \mathbf{as} Q \mathbf{as} \{ \mathbf{as} x_i \mathbf{to} P_i \Rightarrow J_i \} \mid \dots \} \\ &= \varepsilon \mid \mathbf{as} Q \{ \mathbf{as} x_i \mathbf{to} P_i \Rightarrow J_i \} \mid \dots \\ &= \varepsilon \mid \mathbf{as} Q \mathbf{to} P_i \Rightarrow J_i[Q/x_i] \mid \dots \end{aligned}$$

From here, each Q reassociates over the $\mathbf{to}$ -abstraction to the form $\mathbf{as} y \mathbf{to} P'_i \Rightarrow J'_i$ (where P'_i is a new pattern and $J'_i = J_i[Q/x_i]$) by induction on the syntax of Q ,

- ($Q = y$) The base case $\mathbf{as} y \mathbf{to} P'_i \Rightarrow J'_i$ is in the form we need.
- $Q = Q' P$ The argument pattern gets pushed onto the context stack, and the final form follows from the inductive hypothesis

$$\begin{aligned} \mathbf{as} (Q' P) \mathbf{to} P'_i \Rightarrow J'_i &= \mathbf{as} Q' \mathbf{to} (P, P'_i) \Rightarrow J'_i \\ &= \mathbf{as} y \mathbf{to} P''_i \Rightarrow J'_i \end{aligned} \quad (IH)$$

- $Q = Q'.X$ The projection gets transferred as a tag on the context, and the final form follows from the inductive hypothesis

$$\begin{aligned} \mathbf{as} Q'.X \mathbf{to} P'_i \Rightarrow J'_i &= \mathbf{as} Q' \mathbf{to} X[P'_i] \Rightarrow J'_i \\ &= \mathbf{as} y \mathbf{to} P''_i \Rightarrow J'_i \end{aligned} \quad (IH)$$

- The *Other* and *Else* rules for introducing ($\mathbf{else} x R$) and *If* for introducing ($\mathbf{if} P \Leftarrow M R$) are disallowed, under the assumption the reaction is guardless. $\square$

LEMMA 5.4 (DESUGARING EQUALITY). *In the source equational theory,*

- (1) *If J is guardless, then there is a Dual Core command J' such that $J = \llbracket J' \rrbracket^{-1}$.*
- (2) *If M is guardless, then there is a Dual Core (co)term M' such that $M = \llbracket M' \rrbracket^{-1}$.*

PROOF. By induction on the syntax of J and M , using lemma 4.6 to replace every general reaction R inside a **do** with an enumeration of alternatives $\text{as } x_i \text{ to } P_i \Rightarrow J_i$, and using the axioms in fig. 10 with η_{act} to convert the source syntax which lies outside Dual Core into an equivalent Dual Core form. For example, a pair of non-values (M, N) expands to the Dual Core syntax

$$\begin{aligned} (M, N) &= \text{do } z \Rightarrow \text{go } (M, N) \text{ as cbv to } z & (\eta_{act}) \\ &= \text{do } z \Rightarrow \text{go } M \text{ as cbv to do } x \Rightarrow \text{go } N \text{ as cbv to do } y \Rightarrow \text{go } (x, y) \text{ as cbv to } z & (\zeta_{@}) \end{aligned}$$

and a function application expands to

$$\begin{aligned} M N &= \text{do } z \Rightarrow \text{go } M N \text{ as cbn to } z & (\eta_{act}) \\ &= \text{do } z \Rightarrow \text{go } M \text{ as cbn to } (N, z) & (\kappa_{@}) \\ &= \text{do } z \Rightarrow \text{go } (N, z) \text{ as cbv to } M & (\beta^{\circ}) \\ &= \text{do } z \Rightarrow \text{go } N \text{ as cbv to do } x \Rightarrow \text{go } z \text{ as cbv to do } y \Rightarrow \text{go } (x, y) \text{ as cbv to } M & (\zeta_{@}) \\ &= \text{do } z \Rightarrow \text{go } N \text{ as cbv to do } x \Rightarrow \text{go } (x, z) \text{ as cbv to } M & (\beta_{val}) \\ &= \text{do } z \Rightarrow \text{go } N \text{ as cbv to do } x \Rightarrow \text{go } M \text{ as cbn to } (x, z) & (\beta^{\circ}) \end{aligned}$$

The cases for the non-value tag introduction and eliminations are similar.

Moreover, note that the case for every form of command outside of Dual Core has an axiom whose right-hand side is immediately a Dual Core command. Together with the above rewrites, the Dual Core J' and M' which are equal to the original command or (co)term is generated by applying these steps to the inductive hypotheses in each case. $\square$

LEMMA H.5 (FORWARD-BACKWARD ROUND TRIP). *In the source equational theory,*

- (1) *If J is guardless, then $J = \llbracket J \rrbracket^{-1}$.*
- (2) *If M is guardless, then $M = \llbracket M \rrbracket^{-1}$.*

PROOF. (1) From lemma 5.4, there is some Dual Core command J' such that $J = \llbracket J' \rrbracket^{-1}$ in the source equational theory. By forward soundness theorem 5.5 and a backward-forward round trip lemma H.2, this gives us $\llbracket J \rrbracket = \llbracket \llbracket J' \rrbracket^{-1} \rrbracket = J'$ in the Dual Core calculus, and thus by reverse soundness lemma H.1, $\llbracket \llbracket J \rrbracket \rrbracket^{-1} = \llbracket J' \rrbracket^{-1} = J$ as well.

- (2) Similar to the previous part. $\square$

THEOREM 5.6 (EQUATIONAL COMPLETENESS). (1) *For guardless J, J' , if $\llbracket J \rrbracket = \llbracket J' \rrbracket$ then $J = J'$.*

- (2) *For guardless M, M' , if $\llbracket M \rrbracket = \llbracket M' \rrbracket$ then $M = M'$.*

- (3) *For guardless R, R' , if $\llbracket R \rrbracket_n = \llbracket R' \rrbracket_n$ then $R = R'$.*

PROOF. Suppose there are some guardless J and J' such that $\llbracket J \rrbracket = \llbracket J' \rrbracket$ in the Dual Core calculus. Then from the reverse equational soundness lemma H.1, the translation of that equation holds $\llbracket \llbracket J \rrbracket \rrbracket^{-1} = \llbracket \llbracket J' \rrbracket \rrbracket^{-1}$ in the source Coe equational theory, too, so by lemma H.5,

$$J = \llbracket \llbracket J \rrbracket \rrbracket^{-1} = \llbracket \llbracket J' \rrbracket \rrbracket^{-1} = J'$$

The completeness of (co)term and reaction equalities follows similarly. $\square$